操作系统

Operating Systems

刘川意 教授
liuchuanyi@hit.edu.cn

哈尔滨工业大学（深圳）

2025年10月

Module 7: 文件系统

1. 文件（File）和目录（Directory）
   a. 概述（基础）
   b. 文件操作（基础）
   c. 目录操作（基础）
   d. 硬链接和符号链接（基础）
   e. 制作与挂载文件系统（基础）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. Flash-based SSD

Disk VS Memory

	Disk	MLC NAND Flash	DRAM
Smallest write	sector	sector	byte
Atomic write	sector	sector	byte/word
Random read	8 ms	3-10 μs	50 ns
Random write	8 ms	9-11 μs*	50 ns
Sequential read	100 MB/s	550–2500 MB/s	$>$ 10 GB/s
Sequential write	100 MB/s	520–1500 MB/s*	$>$ 10 GB/s
Cost	$0.03/GB	$0.32/GB	$10/GiB
Persistence	Non-volatile	Non-volatile	Volatile

*Flash write performance degrades over time

几个发展趋势

- **Disk bandwidth and cost/bit improving exponentially**
  - Similar to CPU speed, memory size, etc.
- **Seek time and rotational delay improving *very* slowly**
  - Why? require moving physical object (disk arm)
- **Disk accesses a huge system bottleneck & getting worse**
  - Bandwidth increase lets system (pre-)fetch large chunks for about the same cost as small chunk.
  - Trade bandwidth for latency if you can get lots of related stuff.
- **Desktop memory size increasing faster than typical workloads**
  - More and more of workload fits in file cache
  - Disk traffic changes: mostly writes and new data
- **Memory and CPU resources increasing**
  - Use memory and CPU to make better decisions
  - Complex prefetching to support more IO patterns
  - Delay data placement decisions reduce random IO

Persistent Storage

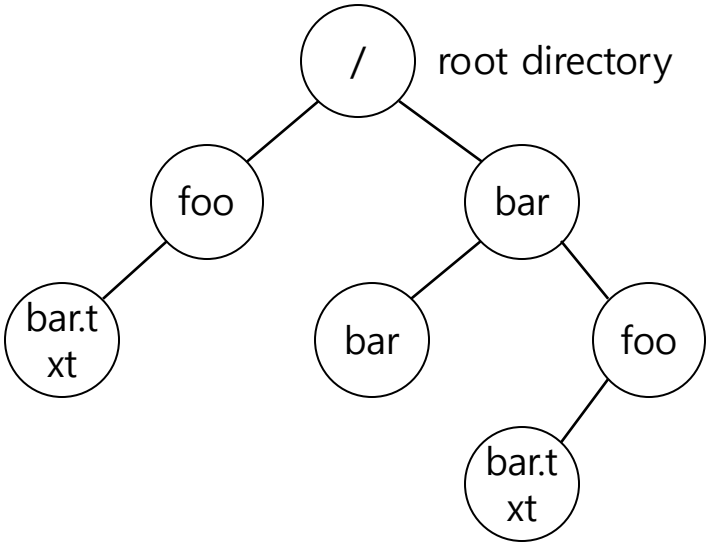
- Keep a data **intact**（完整）even if there is a power loss.
  - Hard disk drive
  - Solid-state storage device

- File System has the responsibility to store data persistently on disk.
  - 文件系统并不关心文件的结构和具体内容；FS的所有工作就是永久的存储数据，确保再次请求时，得到原来放在那里的内容；
  - **但是做到这点并不容易！**
- Two key abstractions in the virtualization of storage
  - File
  - Directory

File & Directory

- A linear array of bytes
- Each file has low-level name as **inode number**
  - The user is not aware of this name.
- Directory is like a file, also has a low-level name. Directory是一种特殊的文件
  - It contains a list of (user-readable name, low-level name) pairs.
  - Each entry in a directory refers to either *files* or other *directories*.
- Example)
  - A directory has an entry (“foo”, “10”)
    - A file “foo” with the low-level name “10”

Directory Tree (Directory Hierarchy)



An Example Directory Tree

Valid files (absolute pathname) :

/foo/bar.txt
/bar/foo/bar.txt

Valid directory :

/
/foo
/bar
/bar/bar
/bar/foo/

Sub-directories

Module 7: 文件系统

1. 文件（File）和目录（Directory）
    a. 概述（基础）
    b. 文件操作（基础）
    c. 目录操作（基础）
    d. 硬链接和符号链接（基础）
    e. 制作与挂载文件系统（基础）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. Flash-based SSD

Creating Files

- Use `open()` system call with `O_CREAT` flag.

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);
```

  - `O_CREAT` : create file.
  - `O_WRONLY` : only write to that file while opened.
  - `O_TRUNC` : make the file size zero (remove any existing content).

- `open()` system call returns **file descriptor**.
  - *File descriptor* is an integer, and is used to access files.

Reading and Writing Files

- An Example of reading and writing '`foo`' file

```
prompt> echo hello > foo
prompt> cat foo
hello
prompt>
```

  - `echo` : redirect the output of echo to the file foo
  - `cat` : dump the contents of a file to the screen

How does the `cat` program access the file foo ?

We can use `strace` to trace the system calls made by a program.

Reading and Writing Files (Cont.)


```
prompt> strace cat foo
…
open("foo", O_RDONLY|O_LARGEFILE)  = 3
read(3, "hello\n", 4096)           = 6
write(1, "hello\n", 6)             = 6 // file descriptor 1: standard out
hello
read(3, "", 4096)                  = 0 // 0: no bytes left in the file
close(3)                           = 0
…
prompt>
```

- `open` (file descriptor, flags)
  - Return file descriptor (3 in example)
  - File descriptor 0, 1, 2, is for standard input/ output/ error.
- `read` (file descriptor, buffer pointer, the size of the buffer)
  - Return the number of bytes it read
- `write` (file descriptor, buffer pointer, the size of the buffer)
  - Return the number of bytes it write

Reading and Writing Files (Cont.)

- Writing a file (A similar set of read steps)
  - A file is opened for writing (`open()`).
  - The `write()` system call is called.
    - Repeatedly called for larger files
  - `close()`

Reading And Writing, But Not Sequentially

- An open file has a **current offset**.
  - Determine **where** the next read or write will begin reading from or writing to within the file.

- Update the current offset
  - **Implicitly（隐式）**: A read or write of `N` bytes takes place, `N` is added to the current offset.
  - **Explicitly（显式）**: `lseek()`

Reading And Writing, But Not Sequentially (Cont.)

```
off_t lseek(int fildes, off_t offset, int whence);
```

- `fildes` : File descriptor
- `offset` : Position the file offset to a particular location within the file
- `whence` : Determine how the seek is performed

From the man page:

```
If whence is SEEK_SET, the offset is set to offset bytes.
If whence is SEEK_CUR, the offset is set to its current
location plus offset bytes.
If whence is SEEK_END, the offset is set to the size of the
file plus offset bytes.
```

Writing Immediately with `fsync()`

- The file system will **buffer** writes in memory for some time.
  - Ex) 5 seconds, or 30
  - Performance reasons

- At that later point in time, the write(s) will **actually be issued** to the storage device.
  - Write seem to complete quickly.
  - Data can be lost (e.g., the machine crashes).

Writing Immediately with `fsync()` (Cont.)

- However, some applications require more than eventual guarantee.
  - Ex) DBMS requires force writes to disk from time to time.

- `off_t fsync(int fd)`
  - Filesystem forces all dirty (i.e., not yet written) data to disk for the file referred to by the file description.
  - `fsync()` returns once all of theses writes are complete.

Writing Immediately with fsync() (Cont.)

- An Example of `fsync()`.

```
int fd = open("foo", O_CREAT | O_WRONLY | O_TRUNC);
assert (fd > -1)
int rc = write(fd, buffer, size);
assert (rc == size);
rc = fsync(fd);
assert (rc == 0);
```

  - In some cases, this code needs to `fsync()` the directory that contains the file `foo`.

Renaming Files

- `rename(char* old, char *new)`
  - Rename a file to different name.
  - It implemented as an **atomic call**.
    - Ex) Change from `foo` to `bar`:

```
prompt> mv foo bar   // mv uses the system call rename()
```

    - Ex) How to update a file atomically:

```
int fd = open("foo.txt.tmp", O_WRONLY|O_CREAT|O_TRUNC);
write(fd, buffer, size); // write out new version of file
fsync(fd);
close(fd);
rename("foo.txt.tmp", "foo.txt");
```

Getting Information About Files

- `stat()`, `fstat()`: Show the file metadata
  - **Metadata** is information about each file.
  - Ex) Size, Low-level name, Permission, …
  - `stat` structure is below:

```
struct stat {
    dev_t st_dev;  /* ID of device containing file */
    ino_t st_ino;  /* inode number */
    mode_t st_mode;        /* protection */
    nlink_t st_nlink;      /* number of hard links */
    uid_t st_uid;  /* user ID of owner */
    gid_t st_gid;  /* group ID of owner */
    dev_t st_rdev; /* device ID (if special file) */
    off_t st_size; /* total size, in bytes */
    blksize_t st_blksize; /* blocksize for filesystem I/O */
    blkcnt_t st_blocks;   /* number of blocks allocated */
    time_t st_atime;       /* time of last access */
    time_t st_mtime;       /* time of last modification */
    time_t st_ctime;       /* time of last status change */
};
```

Getting Information About Files (Cont.)

- To see stat information, you can use the command line tool `stat`.

```
prompt> echo hello > file
prompt> stat file

File: 'file'
Size: 6 Blocks: 8 IO Block: 4096 regular file
Device: 811h/2065d Inode: 67158084 Links: 1
Access: (0640/-rw-r-----) Uid: (30686/ root) Gid:
(30686/ remzi)
Access: 2011-05-03 15:50:20.157594748 -0500
Modify: 2011-05-03 15:50:20.157594748 -0500
Change: 2011-05-03 15:50:20.157594748 -0500
```

- File system keeps this type of information in a `inode` structure.

Removing Files

- `rm` is Linux command to remove a file
  - `rm` call `unlink()` to remove a file.

```
prompt> strace rm foo
…
unlink("foo")          = 0     // return 0 upon
success
…
prompt>
```

Why it calls `unlink()`? not "`remove` or `delete`"
We can get the answer later.

Module 7: 文件系统

1. 文件（File）和目录（Directory）
   a. 概述（基础）
   b. 文件操作（基础）
   c. 目录操作（基础）
   d. 硬链接和符号链接（基础）
   e. 制作与挂载文件系统（基础）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. Flash-based SSD

Making Directories

- `mkdir()`: Make a directory

```
prompt> strace mkdir foo
…
mkdir("foo", 0777)                = 0
prompt>
```

  - When a directory is created, it is empty.
  - Empty directory have two entries: . (itself), .. (parent)

```
prompt> ls -a
./      ../
prompt> ls -al
total 8
drwxr-x---  2 remzi remzi    6 Apr 30 16:17 ./
drwxr-x--- 26 remzi remzi 4096 Apr 30 16:17 ../
```

Reading Directories

- A sample code to read directory entries (like `ls`).

```
int main(int argc, char *argv[]) {
    DIR *dp = opendir("."); // open current directory
    assert(dp != NULL);
    struct dirent *d;
    while ((d = readdir(dp)) != NULL)//read directory entry {
        // print outthe name and inode number of each file
        printf("%d %s\n", (int) d->d_ino, d->d_name);
    }
    closedir(dp);                    // close current directory
    return 0;
}
```

- The information available within `struct dirent`

```
struct dirent {
    char            d_name[256];    /* filename */
    ino_t           d_ino;          /* inode number */
    off_t           d_off;          /* offset to the next direct */
    unsigned short  d_reclen;       /* length of this record */
    unsigned char   d_type;         /* type of file */
}
```

Deleting Directories

- `rmdir()`: Delete a directory.
  - Require that the directory be **empty**.
  - If you call `rmdir()` to a non-empty directory, it will fail.
    - I.e., Only has “.” and “..” entries.

Module 7: 文件系统

1. 文件（File）和目录（Directory）
    a. 概述（基础）
    b. 文件操作（基础）
    c. 目录操作（基础）
    d. 硬链接和符号链接（基础）
    e. 制作与挂载文件系统（基础）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. Flash-based SSD

Hard Links

- `link(old pathname, new one)`
  - **Link** a new file name to an old one
  - Create another way to refer to *the same file*
  - The command-line link program : `ln`

```
prompt> echo hello > file
prompt> cat file
hello
prompt> ln file file2// create a hard link, link file to file2
prompt> cat file2
hello
```

Hard Links (Cont.)

- The way `link` works:
  - **Create** another name in the directory.
  - **Refer** it to the same inode number of the original file.
    - The file is not copied in any way.
  - Then, we now just have two human names (`file` and `file2`) that both refer to the same file.

Hard Links (Cont.)

- The result of `link()`

```
prompt> ls -i file file2
67158084 file  /* inode value is 67158084 */
67158084 file2 /* inode value is 67158084 */
prompt>
```

  - Two files have **same inode** number, but two human name (file, file2).
  - There is **no difference** between file and file2.
    - Both just links to the underlying metadata about the file.

Hard Links (Cont.)

- Thus, to remove a file, we call `unlink()`.

```
prompt> rm file
removed 'file'
prompt> cat file2          // Still access the file
hello
```

- ***reference count***
  - Track how many different file names have been linked to this inode.
  - When `unlink()` is called, the reference count decrements.
  - If the reference count reaches zero, the filesystem free the inode and related data blocks. → truly "delete" the file

Hard Links (Cont.)

- The result of `unlink()`
  - `stat()` shows the reference count of a file.

```
prompt> echo hello > file              /* create file*/
prompt> stat file
... Inode: 67158084 Links: 1 ...       /* Link count is 1 */
prompt> ln file file2                  /* hard link file2 */
prompt> stat file
... Inode: 67158084 Links: 2 ...       /* Link count is 2 */
prompt> stat file2
... Inode: 67158084 Links: 2 ...       /* Link count is 2 */
prompt> ln file2 file3                 /* hard link file3 */
prompt> stat file
... Inode: 67158084 Links: 3 ...       /* Link count is 3 */
prompt> rm file                        /* remove file */
prompt> stat file2
... Inode: 67158084 Links: 2 ...       /* Link count is 2 */
prompt> rm file2                       /* remove file2 */
prompt> stat file3
... Inode: 67158084 Links: 1 ...       /* Link count is 1 */
prompt> rm file3
```

Symbolic Links (Soft Link)

- Symbolic link is more **useful** than Hard link.
  - Hard Link cannot create to a directory.
  - Hard Link cannot create to a file to other partition.
    - Because inode numbers are only unique within a file system.

- Create a symbolic link: `ln -s`

```
prompt> echo hello > file
prompt> ln -s file file2/* option -s: create symbolic link */
prompt> cat file2
hello
```

Symbolic Links (Cont.)

- What is different between *Symbolic link* and *Hard Link*?
  - Symbolic links are **a third type** the file system knows about.

```
prompt> stat file
 ... regular file ...
prompt> stat file2
 ... symbolic link ...      // Actually a file it self of a
different type
```

  - The size of symbolic link (`file2`) is **4 bytes**.

```
prompt> ls -al
drwxr-x---  2 remzi remzi   29 May 3 19:10 ./
drwxr-x--- 27 remzi remzi 4096 May 3 15:14 ../  // directory
-rw-r-----  1 remzi remzi    6 May 3 19:10 file1//regular file
lrwxrwxrwx  1 remzi remzi    4 May 3 19:10 file2 -> file /*
symbolic link */
```

- A symbolic link holds the pathname of the linked-to file as the data of the link file.

Symbolic Links (Cont.)

- If we link to a longer pathname, our link file would be bigger.

```
prompt> echo hello > alongerfilename
prompt> ln -s alongerfilename file3
prompt> ls -al alongerfilename file3
-rw-r----- 1 remzi remzi  6 May 3 19:17 alongerfilename
lrwxrwxrwx 1 remzi remzi 15 May 3 19:17 file3 ->
alongerfilename
```

Symbolic Links (Cont.)

- **Dangling reference (悬空引用)**
  - When remove a original file, symbolic link points noting.

```
prompt> echo hello > file
prompt> ln -s file file2
prompt> cat file2
hello
prompt> rm file              // remove the original file
prompt> cat file2
cat: file2: No such file or directory
```

例题

若文件f1的硬链接为f2，两个进程分别打开f1和f2，获得对应的文件描述符为fd1和fd2，则下列叙述中正确的是（）。

I. f1和f2的读写指针位置保持相同

II. f1和f2共享同一个内存索引结点

III. fd1和fd2分别指向各自的用户打开文件表中的一项

A. 仅III　　B. 仅II、III　　C. 仅I、II　　D. I、II和III

答案：B

解析：一个文件在物理存储器上有一个索引结点号，硬链接通过索引结点进行连接，因此f1和f2共享索引结点，II正确。两个进程各自维护自己的文件描述符，III正确，I错误。

Module 7: 文件系统

1. 文件（File）和目录（Directory）
    a. 概述（基础）
    b. 文件操作（基础）
    c. 目录操作（基础）
    d. 硬链接和符号链接（基础）
    e. 制作与挂载文件系统（基础）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. Flash-based SSD

Making and Mounting a File System

- `mkfs` tool : Make a file system
  - Write an empty file system, starting with *a root directory*, on to a disk partition.
  - Input:
    - A device (such as a disk partition, e.g., `/dev/sda1`)
    - A file system type (e.g., `ext3`)

Making and Mounting a File System (Cont.)

- `mount()`
  - Take an existing directory as a target **mount point**.
  - Essentially paste a new file system onto the directory tree at that point.
  - **Example)**

```
prompt> mount -t ext3 /dev/sda1 /home/users
prompt> ls /home/users
a b
```

    - The pathname `/home/users/` now refers to the root of the newly-mounted directory.

Making and Mounting a File System (Cont.)

- `mount` program: show **what is mounted** on a system.

```
/dev/sda1 on / type ext3 (rw)
proc on /proc type proc (rw)
sysfs on /sys type sysfs (rw)
/dev/sda5 on /tmp type ext3 (rw)
/dev/sda7 on /var/vice/cache type ext3 (rw)
tmpfs on /dev/shm type tmpfs (rw)
AFS on /afs type afs (rw)
```

  - `ext3`: A standard disk-based file system
  - `proc`: A file system for accessing information about current processes
  - `tmpfs`: A file system just for temporary files
  - `AFS`: A distributed file system

例题

若一个用户进程通过read系统调用读取一个磁盘文件中的数据，则下列关于此过程的叙述中，正确的是（ ）。
I.若该文件的数据不在内存，则该进程进入睡眠等待状态
II.请求read系统调用会导致CPU从用户态切换到核心态
III. read系统调用的参数应包含文件的名称

A.仅I、II　　B. 仅I、III　　C.仅II、III　　D.I、II和III

答案：A
对于I，当所读文件的数据不在内存时，产生中断（缺页中断），原进程进入阻塞态，直到所需数据从外存调入内存后，才将该进程唤醒。
对于Ⅱ，read系统调用通过陷入将CPU从用户态切换到核心态，从而获取操作系统提供的服务。
对于III，要读一个文件，首先要用open系统调用打开该文件。open中的参数包含文件的路径名与文件名，而read只需使用open返回的文件描述符，并不使用文件名作为参数。read要求用户提供三个输入参数：①文件描述符f；②buf缓冲区首址；③传送的字节数n。read的功能是试图从d所指示的文件中读入n个字节的数据，并将它们送至由指针buf所指示的缓冲区中。

例题

下面的说法中，正确的是（ ）。

A. 文件的构造方式仅能是树形结构

B. 物理文件的组织方式是由应用程序确定的

C. 连续分配是不适合直接存取的外存分配方式

D. 顺序存放是导致存储碎片发生可能性最大的物理文件结构

答案：D

A. 错误，文件的构造方式有很多类别，常用的有字节序列、记录序列、树。

B. 错误，文件的物理结构由操作系统的设计者根据文件存储器的特性来确定的。

C. 错误，连续分配恰恰非常适合直接存取，因为文件中的块是连续存储的，可以通过起始地址和偏移量直接访问。它的问题是容易产生外部碎片和动态存储管理的困难。

D. 正确，顺序文件占用连续的磁盘空间，因此容易导致存储碎片（外部碎片）的发生。

Module 7: 文件系统

1. 文件（File）和目录（Directory）
2. VSFS（Very Simple File System）简单文件系统
   a. Data Organization（基础）
   b. Access Methods（基础）
   c. Linux EXT2文件系统（基础）
   d. Windows FAT文件系统（基础）
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. Flash-based SSD

The Way To Think

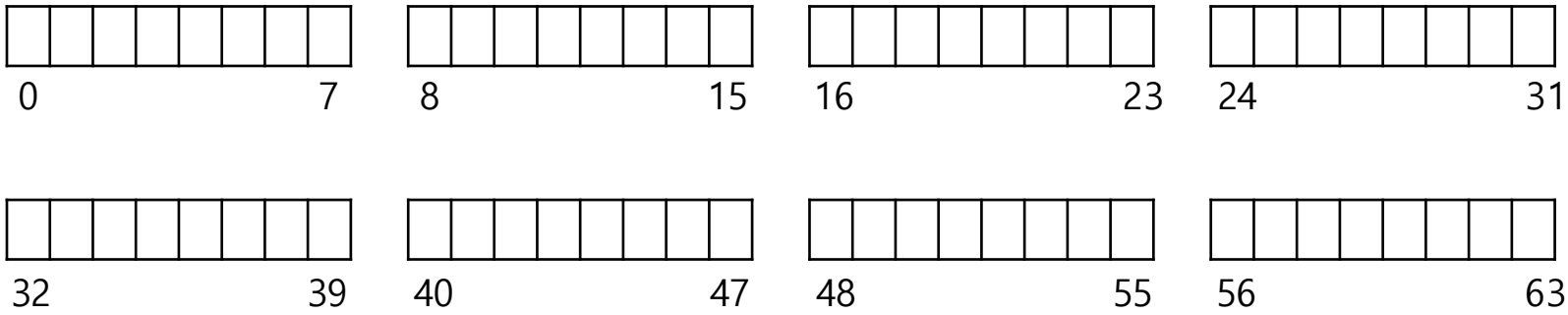
- 实现一个文件系统，需要搞清楚2个方面的Know How，并在此基础上进行设计优化：
  - **Data structures**
    - What types of on-disk structures are utilized by the file system to organize its data and metadata?
  - **Access methods**
    - How does it map the calls made by a process as `open()`, `read()`, `write()`, etc.
    - Which structures are read during the execution of a particular system call?

FS设计要点 #1:
Data Organization

Overall Organization

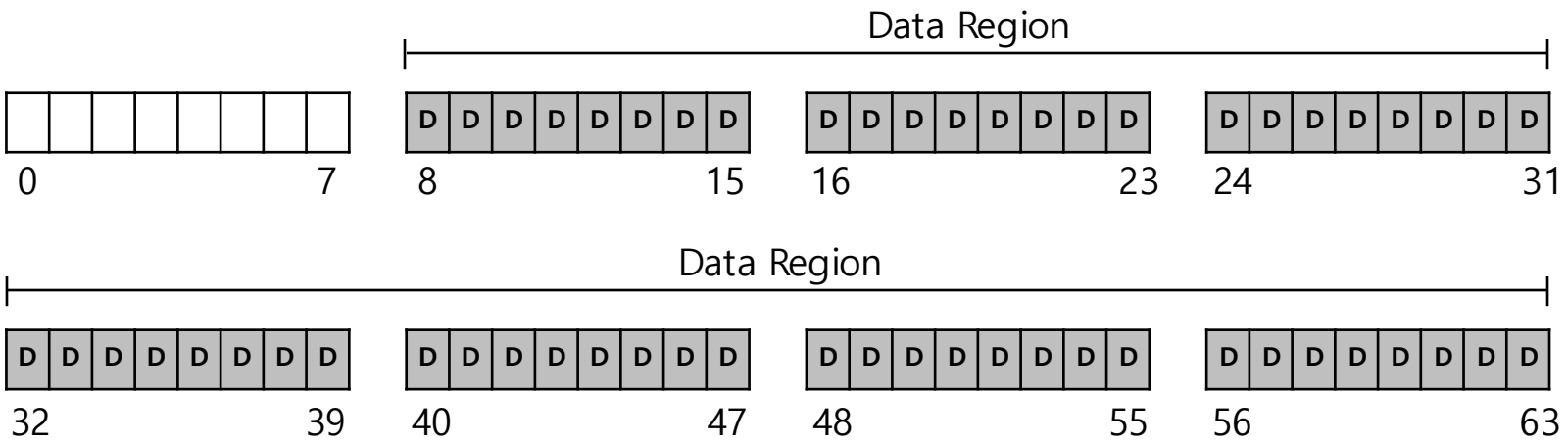
- Let's develop the overall organization of the file system data structure.

- 文件系统把数据组织成**blocks**数组
  - Block size is 4 KB.
  - The blocks are addressed from *0 to N -1.*



Data region in file system用来存储实际数据

- Reserve **data region** to store user data

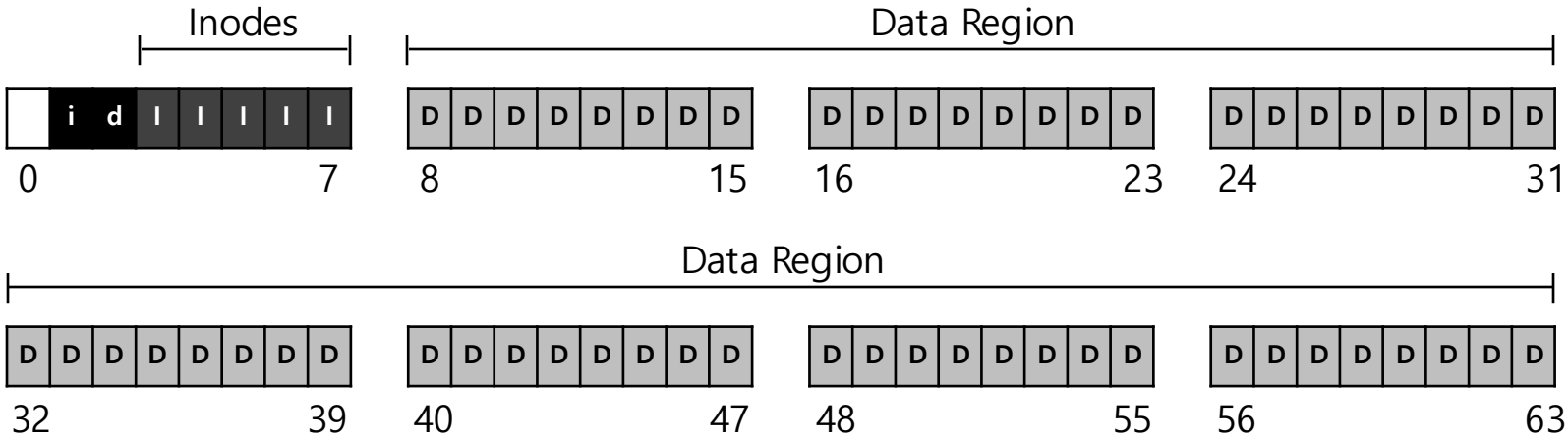


- File system has to track which data block comprise a file, the size of the file, its owner, etc.

How we store these inodes in file system?

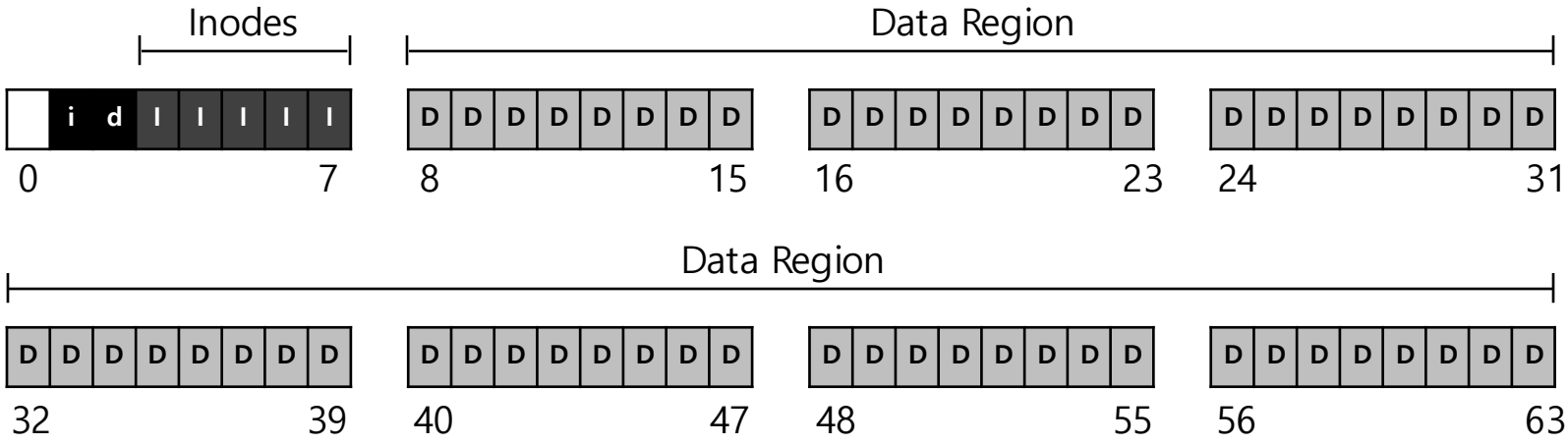
Inode table in file system

- Reserve some space for **inode table**
  - This holds an array of on-disk inodes.
  - Ex) inode tables : 3 ~ 7, inode size : 256 bytes
    - 4-KB block can hold 16 inodes.
    - The filesystem contains 80 inodes. (maximum number of files)



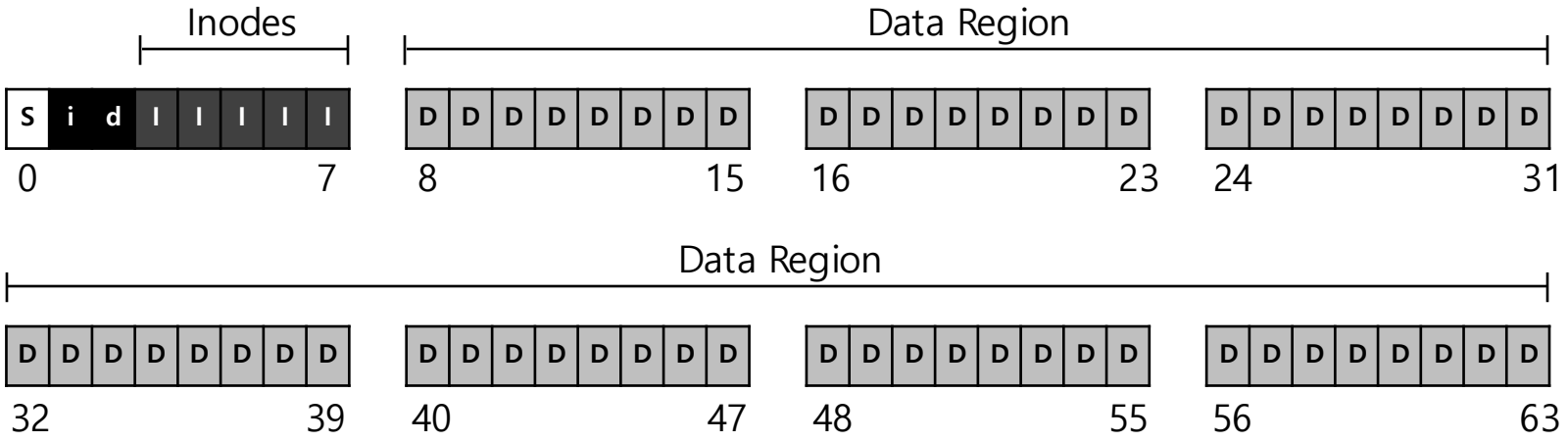
allocation structures

- This is to track whether inodes or data blocks are free or allocated.
- 使用位图bitmap表示每个block是否已分配，each bit indicates free(0) or in-use(1)
  - **data bitmap**: for data region for data region
  - **inode bitmap**: for inode table



Superblock表示FS的元数据

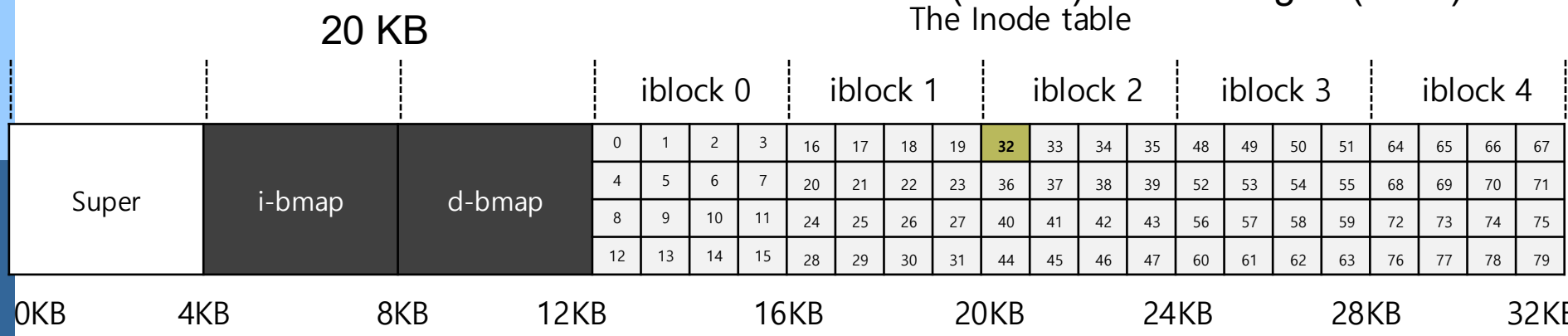
- Super block contains this **information** for **particular file system**
  - Ex) The number of inodes, begin location of inode table. etc



  - Thus, when mounting a file system, OS will read the superblock first, to initialize various information.

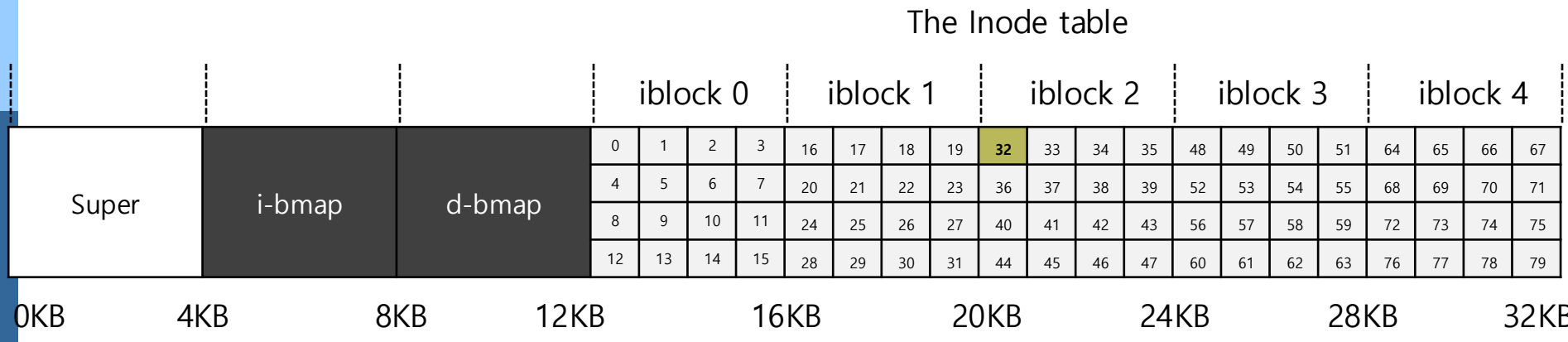
File Organization: The inode

- Inode（index node）是由Unix元老 Ken Thompson取名
- Each inode is referred to by inode number. 每个inode由inode ID唯一标志
  - **通过 inode number, FS可计算对应的inode的存储位置**
  - Ex) inode number: 32
    - Calculate the offset into the inode region (32 x sizeof(inode) (256 bytes) = 8192
    - Add start address of the inode table(12 KB) + inode region(8 KB) = 20 KB



File Organization: The inode (Cont.)

- Disk不是按byte寻址的，而是以sector为单位寻址（sector addressable）
- Disk consist of a large number of addressable sectors, (512 bytes)
  - Ex) Fetch the block of inode (inode number: 32)
    - Sector address `iaddr` of the inode block:
    - `blk : (inumber * sizeof(inode)) / blocksize`
    - `sector : (blk * blocksize) + inodeStratAddr ) /sectorsize`



File Organization: The inode (Cont.)

- `inode` have all of the information about a file
  - File type (regular file, directory, etc.),
  - Size, the number of blocks allocated to it.
  - Protection information(who ones the file, who can access, etc).
  - Time information.
  - Etc.

The EXT2 FS inode structure

Size	Name	What is this inode field for?
2	mode	can this file be read/written/executed?
2	uid	who owns this file?
4	size	how many bytes are in this file?
4	time	what time was this file last accessed?
4	ctime	what time was this file created?
4	mtime	what time was this file last modified?
4	dtime	what time was this inode deleted?
4	gid	which group does this file belong to?
2	links_count	how many hard links are there to this file?
2	blocks	how many blocks have been allocated to this file?
4	flags	how should ext2 use this inode?
4	osd1	an OS-dependent field
60	block	a set of disk pointers (15 total)
4	generation	file version (used by NFS)
4	file_acl	a new permissions model beyond mode bits
4	dir_acl	called access control lists
4	faddr	an unsupported field
12	i_osd2	another OS-dependent field

The Multi-Level Index

- To support bigger files, use multi-level index. 即多级索引是协助单个文件组织数据的
- **Indirect pointer** points to a block that contains more pointers.
  - inode have fixed number of direct pointers (12) and a single indirect pointer.
  - If a file grows large enough, an indirect block is allocated, inode's slot for an indirect pointer is set to point to it.
    - $(12 + 1024)$ x 4 K or 4144 KB
- Double indirect pointer points to a block that contains indirect blocks.
  - Allow file to grow with an additional 1024×1024 or 1 million 4KB blocks.
- Triple indirect pointer points to a block that contains double indirect blocks.
- Multi-Level Index approach to pointing to file blocks.
  - Ex) twelve direct pointers, a single and a double indirect block.
    - over 4GB in size $(12+1024+1024^2)$ x 4KB
- Many file system use a multi-level index.
  - Linux EXT2, EXT3, NetApp's WAFL, Unix file system.
  - Linux EXT4 use extents instead of simple pointers.

例题

若某文件系统索引节点（inode）中有直接地址和间接地址项，则下列选项中，与单个文件长度无关的因素是（ ）

A. 索引节点的总数

B. 间接地址索引的级数

C. 地址项的个数

D. 文件块大小

答案： A

解析：

- 地址项的个数（C选项） = 直接地址索引 + 间接地址索引（B选项）
- 单个文件长度 =地址项的个数（C选项） * 文件块大小（D选项）
- A选项，一个文件对应一个索引节点，索引节点的总数只能说明有多少个文件，与单个文件长度无关。

例题

设文件索引节点中有7个地址项，其中4个地址项为直接地址索引，2个地址项是一级间接地址索引，1个地址项是二级间接地址索引，每个地址项大小为4字节，若磁盘索引块和磁盘数据块大小均为256字节，则可表示的单个文件的最大长度是（）。

A. 33KB　　B. 519KB　　C. 1057KB　　D. 16513KB

答案：C

解析：每个磁盘索引块和磁盘数据块大小均为256B，每个磁盘索引块共有256/4=64个地址项。因此，4个直接地址索引指向的数据块大小为4*256B；2个一级间接索引包含的直接地址索引数为2*(256/4)，数据块大小为2*(256/4)*256B；1个二级间接索引指向的数据块大小为(256/4)*(256/4)*256B。因此，7个地址项所指向的数据块总大小为(4+(256/4)*2+(256/4)*(256/4)*1)*256B=1057KB。

The Multi-Level Index (Cont.)

File System Measurement Summary

Most files are small	Roughly 2K is the most common size
Average file size is growing	Almost 200K is the average
Most bytes are stored in large files	A few big files use most of the space
File systems contains lots of files	Almost 100K on average
File systems are roughly half full	Even as disks grow, file system remain -50% full
Directories are typically small	Many have few entries; most have 20 or fewer

例题

典型文件系统计算题：

某文件系统采用多级索引方式，其inode中有13个地址项，其中直接索引10项，一次间接索引1项，二次间接索引1项，三次间接索引1项。数据块为4KB，磁盘地址用4B表示，这个文件系统允许的最大文件长度是多少？

答案：

每个块可以存储的地址数为 $4KB / 4B = 1024$

最大文件的块数为 $10 + 1024 + 1024^2 + 1024^3$

故最大文件大小为

$(10 + 1024 + 1024^2 + 1024^3) * 4KB = 40KB + 4MB + 4GB + 4TB$

习题

【考研真题】某文件系统最大容量为16TB，以磁盘块为基本分配单位，磁盘块大小为4KB。文件控制块(FCB)包含一个1024B的索引表区。请回答下列问题：

(1)假设索引表区仅采用直接索引结构，索引表区存放文件占用的磁盘块号。索引表项中块号最少占多少字节？可支持的单个文件最大长度是多少字节？

(2)假设索引表区采用如下混合结构：第0～7字节采用＜起始块号，块数＞格式表示文件创建时预分配的连续存储空间，其中起始块号占5B，块数占3B；剩余1016字节采用直接索引结构，一个索引项占8B，则可支持的单个文件最大长度是多少字节？为了使单个文件的长度达到最大，请指出起始块号和块数分别所占字节数的合理值并说明理由。

Directory Organization

- Directory contains a list of (entry name, inode number) pairs.
- Each directory has two extra files ."dot" for current directory and .."dot-dot" for parent directory
  - For example, `dir` has three files (`foo, bar, foobar`)

on-disk structure for dir

inum	reclen	strlen	name
5	4	2	.
2	4	3	..
12	4	4	foo
13	4	4	bar
24	8	7	foobar

Free Space Management

- File system track which inode and data block are free or not.
- In order to manage free space, we have two simple bitmaps.
  - When file is newly created, it allocated inode by searching the inode bitmap and update on-disk bitmap.
  - Pre-allocation policy is commonly used for allocate contiguous blocks.

Module 7: 文件系统

1. 文件（File）和目录（Directory）
2. VSFS（Very Simple File System）简单文件系统
   a. Data Organization（基础）
   b. Access Methods（基础）
   c. Linux EXT2文件系统（基础）
   d. Windows FAT文件系统（基础）
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. Flash-based SSD

FS设计要点 #2:
Access Methods

Access Paths: Reading a File From Disk

- Issue an `open("/foo/bar", O_RDONLY)`,

  Traverse the pathname and thus locate the desired inode. open的目标是找到文件的inode

  - Begin at the root of the file system (/)
    - In most Unix file systems, the root inode number is 2
  - Filesystem reads in the block that contains inode number 2.
  - Look inside of it to find pointer to data blocks (contents of the root).
  - By reading in one or more directory data blocks, It will find "foo" directory.
  - Traverse recursively the path name until the desired inode ("bar")
  - Check finale permissions, allocate a file descriptor for this process and returns file descriptor to user. 确认权限，为进程分配fd，并将fd返回给应用程序

Access Paths: Reading a File From Disk (Cont.)

- Issue `read()` to read from the file.

  Read in the first block of the file, consulting the inode to find the location of such a block. Read的目标是读取文件的第一个block，通过查询inode的索引表找到第一个block

  - Update the inode with a new last accessed time.
  - Update in-memory open file table for file descriptor, the file offset.

- When file is closed:
  - File descriptor should be deallocated, but for now, that is all the file system really needs to do. No disk I/Os take place.

Access Paths: Reading a File From Disk (Cont.)

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
open(bar)			read			read				
				read						
							read			
					read					
read()					read					
								read		
					write					
read()					read					
									read	
					write					
read()					read					
										read
					write					

File Read Timeline (I/O路径： "/foo/bar")

Access Paths: Writing to Disk

- Issue `write()` to update the file with new contents.
- File may allocate a block (unless the block is being overwritten).
  - Need to update data block, data bitmap.
  - It generates five I/Os:
    - one to read the data bitmap
    - one to write the bitmap (to reflect its new state to disk)
    - two more to read and then write the inode
    - one to write the actual block itself.
  - To create file, it also allocate space for directory, causing high I/O traffic.

Access Paths: Writing to Disk (Cont.)

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data[0]	bar data[1]	bar data[2]
create (/foo/bar)		read write	read	read write	read write	read	read write			
write()	read write				read write			write		
write()	read write				read write				write	
write()	read write				read write					write

File Creation Timeline (I/O路径： "/foo/bar")

Caching and Buffering

- Reading and writing files are expensive, incurring many I/Os.
  - For example, long pathname(/1/2/3/…./100/file.txt)
    - One to read the inode of the directory and at least one read its data.
    - Literally perform hundreds of reads just to open the file.
- In order to reduce I/O traffic, file systems aggressively use system memory(DRAM) to cache.
  - Early file system use fixed-size cache to hold popular blocks.
    - Static partitioning of memory can be wasteful;
  - Modem systems use dynamic partitioning approach（动态分区方法）, unified page cache（统一页面缓存）. Linux会把空闲内存都用作cache buffer
- Read I/O can be avoided by large cache.

Caching and Buffering (Cont.)

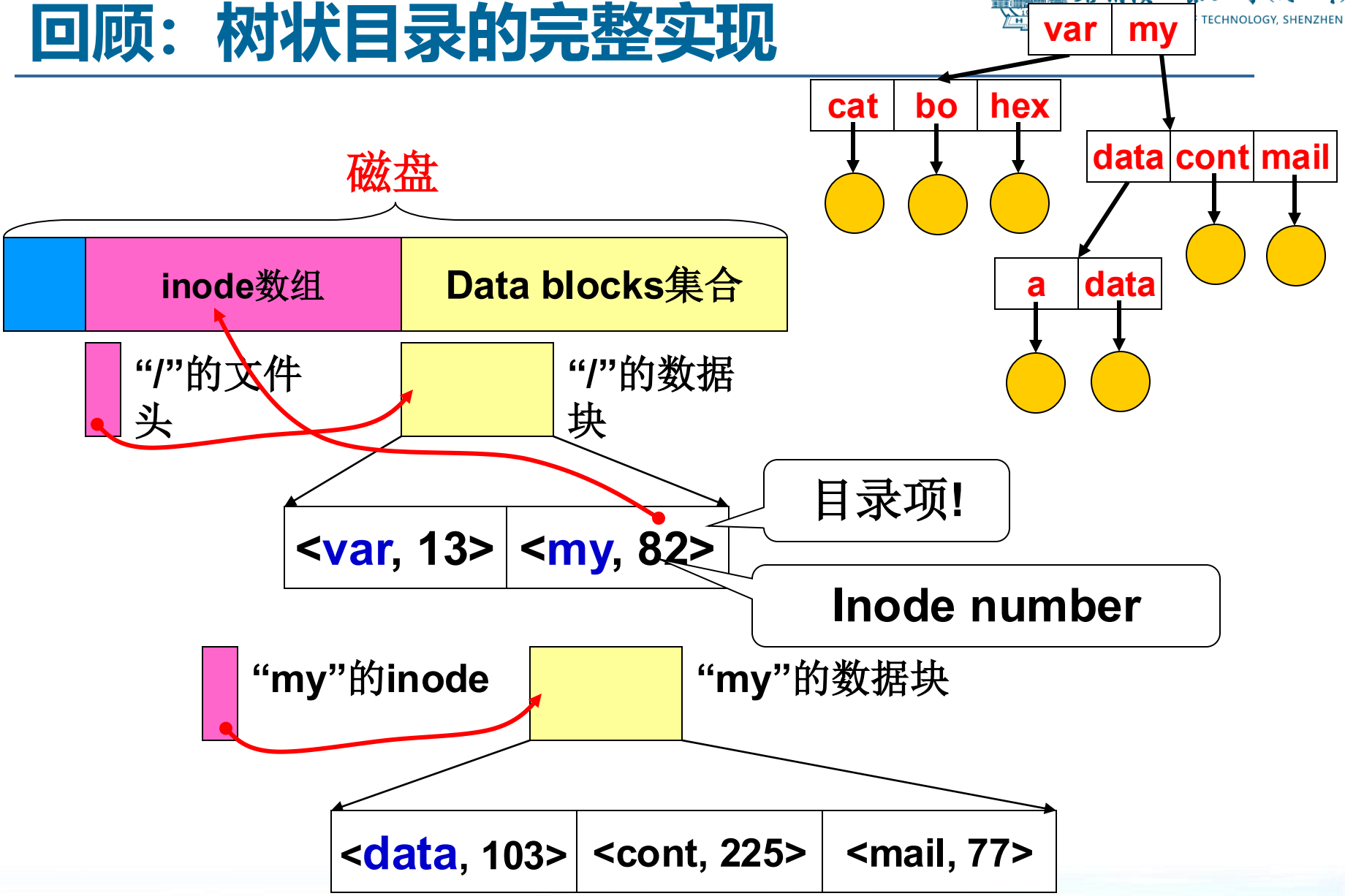
- Write traffic has to go to disk for persistent, Thus, cache does not reduce write I/Os.
- File system use write buffering for write performance benefits.
  - delaying writes (file system batch some updates into a smaller set of I/Os).
  - By buffering a number of writes in memory, the file system can then schedule the subsequent I/Os.
  - By avoiding writes
- Some application force flush data to disk by calling `fsync()` or direct I/O.

Module 7: 文件系统

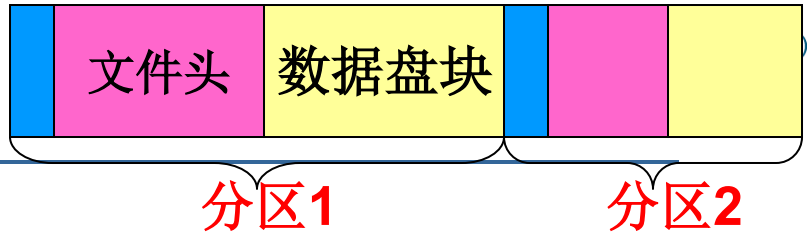
1. 文件（File）和目录（Directory）
2. VSFS（Very Simple File System）简单文件系统
   a. Data Organization（基础）
   b. Access Methods（基础）
   c. Linux EXT2文件系统（基础）
   d. Windows FAT文件系统（基础）
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. Flash-based SSD

案例分析 #1:
Linux EXT2文件系统

回顾：树状目录的完整实现



回顾：分区的详细结构



❿ 典型分区结构：UNIX分区的基本结构



- 引导块存放引导OS的信息，如果该分区中没有OS，则该块为空
- 超级块记录分区基本信息: 分区块数；块大小；空闲块数量、指针；空闲文件头数量、指针等
- 索引节点数组存放所有文件的inode，UNIX root目录的索引节点号为2
- 数据块，文件内容

EXT2文件系统

- EXT2文件系统把逻辑分区划分为块组，并且从0开始编号。每个块组包含的等量的物理块(即块组大小是相同的；物理分区最后一个块组可能小些)；在块组的数据块中存储文件或目录；

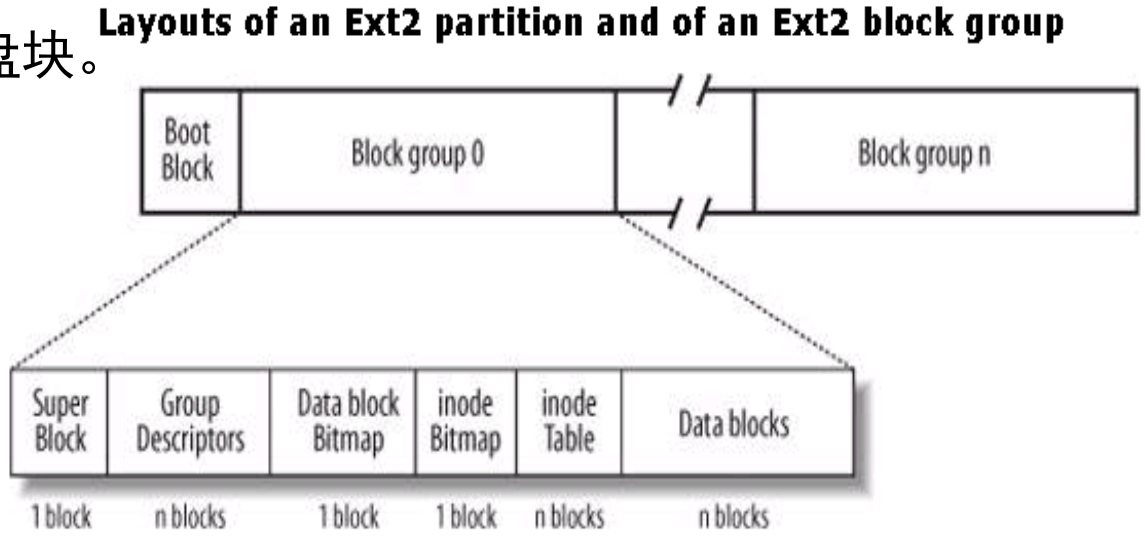
图9.2 Ext2磁盘布局在逻辑空间的映像



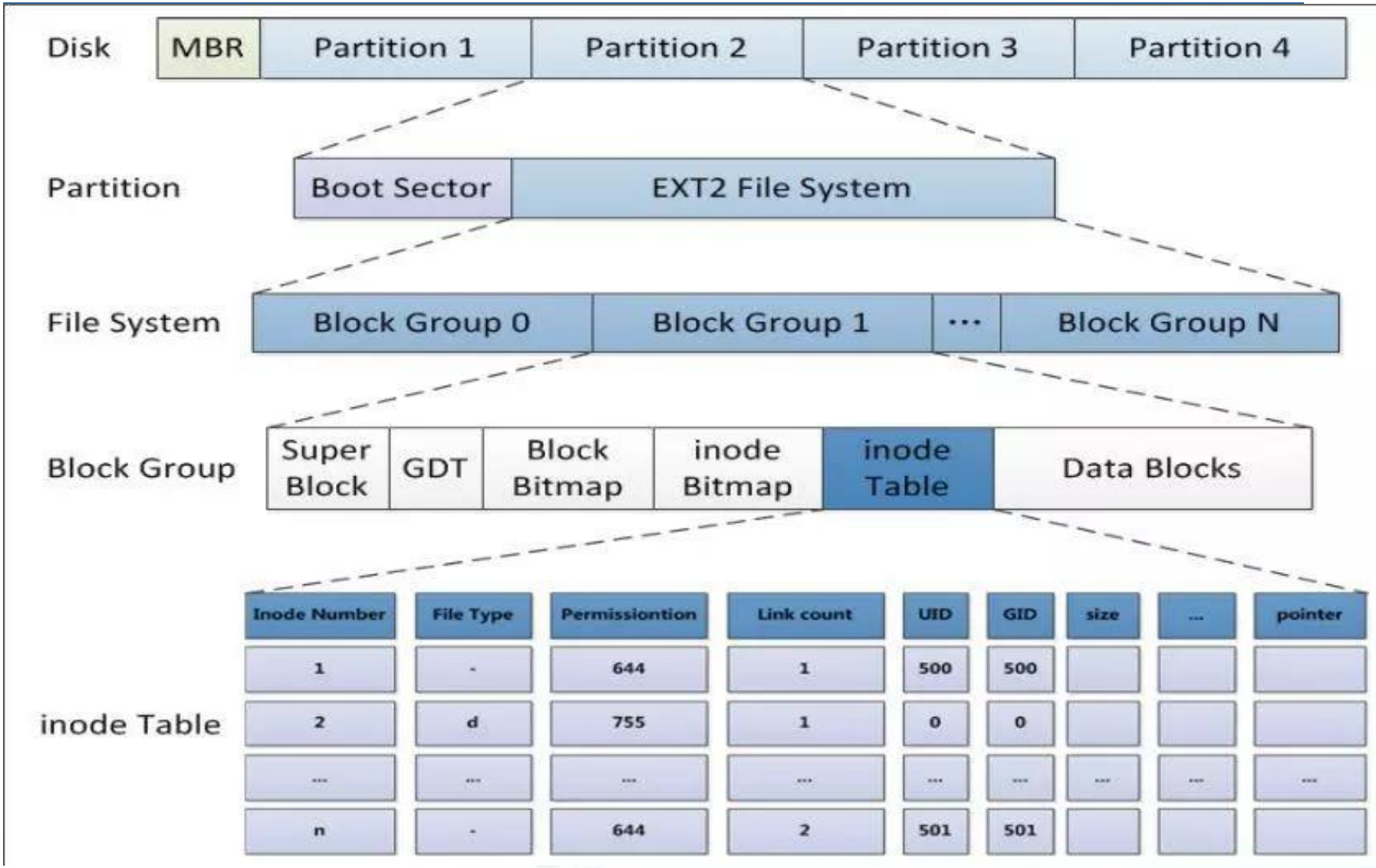
- 上图中启动块（Boot Block）的大小是确定的，用来存储磁盘分区信息和启动信息，任何文件系统都不能使用启动块。启动块之后才是ext2文件系统的开始。

EXT2文件系统的磁盘组织

- 除了引导扇区之外，EXT2磁盘分区被顺序划分为若干个**磁盘块组**（Block Group）。 每个块组按照相同的方式组织，具有相同的大小。
- EXT2磁盘块组中的磁盘块按顺序被组织成：
  - 一个用作**超级块**的磁盘块：存放了文件系统超级块的一个拷贝；
  - 多个记录**组描述符**的磁盘块；
  - 1个记录**数据块位图**的磁盘块；
  - 1个记录**索引结点位图**的磁盘块；
  - 多个用作**索引结点表**的磁盘块；
  - 多个用作**数据块**的磁盘块。



Ext2全貌图


Disk
MBR
Partition 1
Partition 2
Partition 3
Partition 4
Partition
Boot Sector
EXT2 File System
File System
Block Group 0
Block Group 1
···
Block Group N
Block Group
Super Block
GDT
Block Bitmap
inode Bitmap
inode Table
Data Blocks
inode Table
Inode Number
File Type
Permissiontion
Link count
UID
GID
size
...
pointer
1 - 644 1 500 500
2 d 755 1 0 0
... ...
n - 644 2 501 501

EXT2的超级块

- 描述整个分区的文件系统信息，如块大小、版本号、上次mount时间等。
- 每个块组的第一个磁盘块用来保存所在EXT2 fs的超级块
- 多个块组中的超级块形成冗余
  - 在某个或少数几个超级块被破坏时，可用于恢复被破坏的超级块信息。
  - 系统运行期间，把超级块复制到系统缓冲区内，只需把块组0的超级块读入内存，其它块组的超级块做为备份

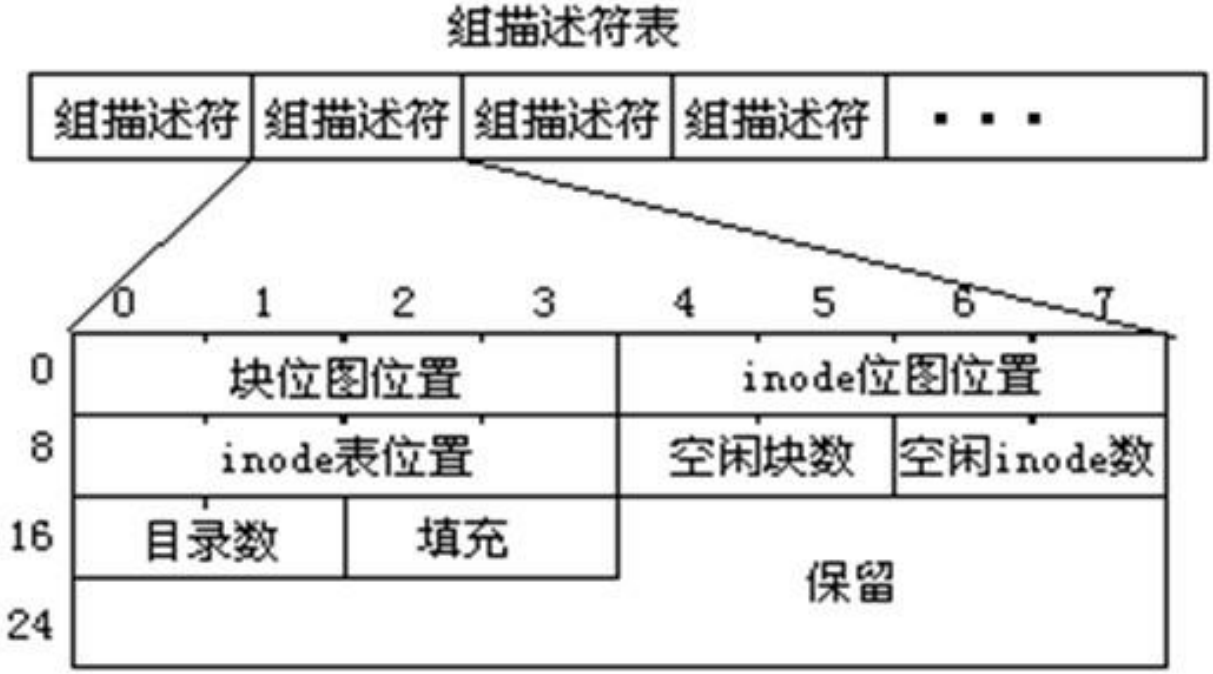
超级块数据结构



EXT2文件系统超级块

块组描述符

- 块组描述符用来描述一个磁盘块组的相关信息
- 块组描述符组由若干块组描述符组成，描述了文件系统中所有块组的属性，存放于超级块所在块的下一个块中。



EXT2文件系统组描述符

块组描述符

- 一个块组描述符的结构如下：

```
struct ext2_group_desc
{
__le32 bg_block_bitmap; // 块位图所在的第一个块的块ID
__le32 bg_inode_bitmap; // inode位图所在的第一个块的块ID
__le32 bg_inode_table; // inode表所在的第一个块的块ID
__le16 bg_free_blocks_count; // 块组中未使用的块数
__le16 bg_free_inodes_count; // 块组中未使用的inode数
__le16 bg_used_dirs_count; // 块组分配的目录的inode数
__le16 bg_pad;
__le32 bg_reserved[3];
};
```

数据块位图和索引结点块位图

- EXT2的空闲盘块分配算法采用了位图法
- 位图：为便于查找数据块或索引结点的分配信息
- 每个位（bit）都对应了一个磁盘块：
  - 0，表示对应的磁盘块（或索引结点）空闲
  - 1，表示占用。
- 2个位图（数据块和索引节点）分别占用一个专门的磁盘块；位于组描述符表之后
- 根据磁盘块的大小，可以计算出每个块组中最多能容纳的数据块个数和索引节点块个数

索引结点

- EXT2索引结点128
- inode的结构如下：

```
struct ext2_inode {
__le16 i_mode; // 文件格式和访问权限
__le16 i_uid; // 文件所有者ID的低16位
__le32 i_size; // 文件字节数
__le32 i_atime; // 文件上次被访问的时间
__le32 i_ctime; // 文件创建时间
__le32 i_mtime; // 文件被修改的时间
__le32 i_dtime; // 文件被删除的时间（如果存在则为0）
__le16 i_gid; // 文件所有组ID的低16位
__le16 i_links_count; // 此inode被连接的次数
__le32 i_blocks; // 文件已使用和保留的总块数（以512B为单位
__le32 i_flags; // 此inode访问数据时ext2的实现方式
union {
struct {
__le32 l_i_reserved1; // 保留
} linux1;
struct {
__le32 h_i_translator; // “翻译者”标签
} hurd1;
struct {
__le32 m_i_reserved1; // 保留
} masix1;
} osd1; // 操作系统相关数据
```

续

```
__le32 i_block[EXT2_N_BLOCKS]; // 定位存储文件的块的数组，前
块号，第14个为二级间接块号，第15个为三级间接块号
__le32 i_generation; // 用于NFS的文件版本
__le32 i_file_acl; // 包含扩展属性的块号，老版本中为0
__le32 i_dir_acl; // 表示文件的“High Size”，老版本中为0
__le32 i_faddr; // 文件最后一个段的地址
union {
struct {
__u8 l_i_frag; // 段号
__u8 l_i_fsize; // 段大小
__u16 i_pad1;
__le16 l_i_uid_high; // 文件所有者ID的高16位
__le16 l_i_gid_high; // 文件所有组ID的高16位
__u32 l_i_reserved2;
} linux2;
struct {
__u8 h_i_frag; // 段号
__u8 h_i_fsize; // 段大小
__le16 h_i_mode_high;
__le16 h_i_uid_high; // 文件所有者ID的高16位
__le16 h_i_gid_high; // 文件所有组ID的高16位
__le32 h_i_author;
} hurd2;
struct {
__u8 m_i_frag; // 段号
__u8 m_i_fsize; // 段大小
__u16 m_pad1;
__u32 m_i_reserved2[2];
} masix2;
} osd2; // 操作系统相关数据
};
```

关于索引节点中的i_block[]

- ext2的索引结点中使用了组合索引方式。

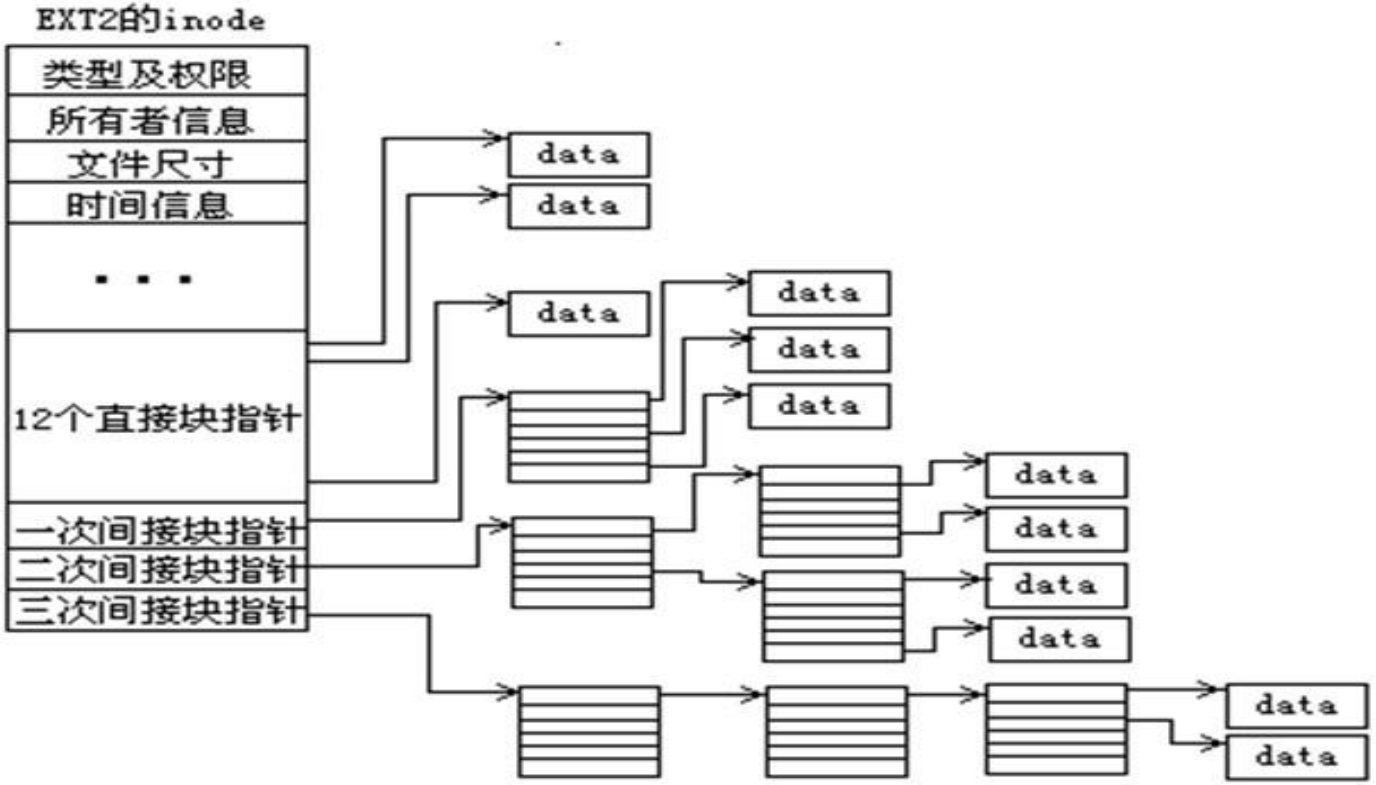
```
00241:      __le32    i_block[EXT2_N_BLOCKS];/* Pointers to blocks */
```

```
00159: /*
00160:  * Constants relative to the data blocks
00161:  */
00162: #define   EXT2_NDIR_BLOCKS        12
00163: #define   EXT2_IND_BLOCK              EXT2_NDIR_BLOCKS
00164: #define   EXT2_DIND_BLOCK             (EXT2_IND_BLOCK + 1)
00165: #define   EXT2_TIND_BLOCK             (EXT2_DIND_BLOCK + 1)
00166: #define   EXT2_N_BLOCKS           (EXT2_TIND_BLOCK + 1)
```

  - 前12项用作直接索引
  - 第13项用作间接索引
  - 第14项用作二次间接索引
  - 第15项用作三次间接索引

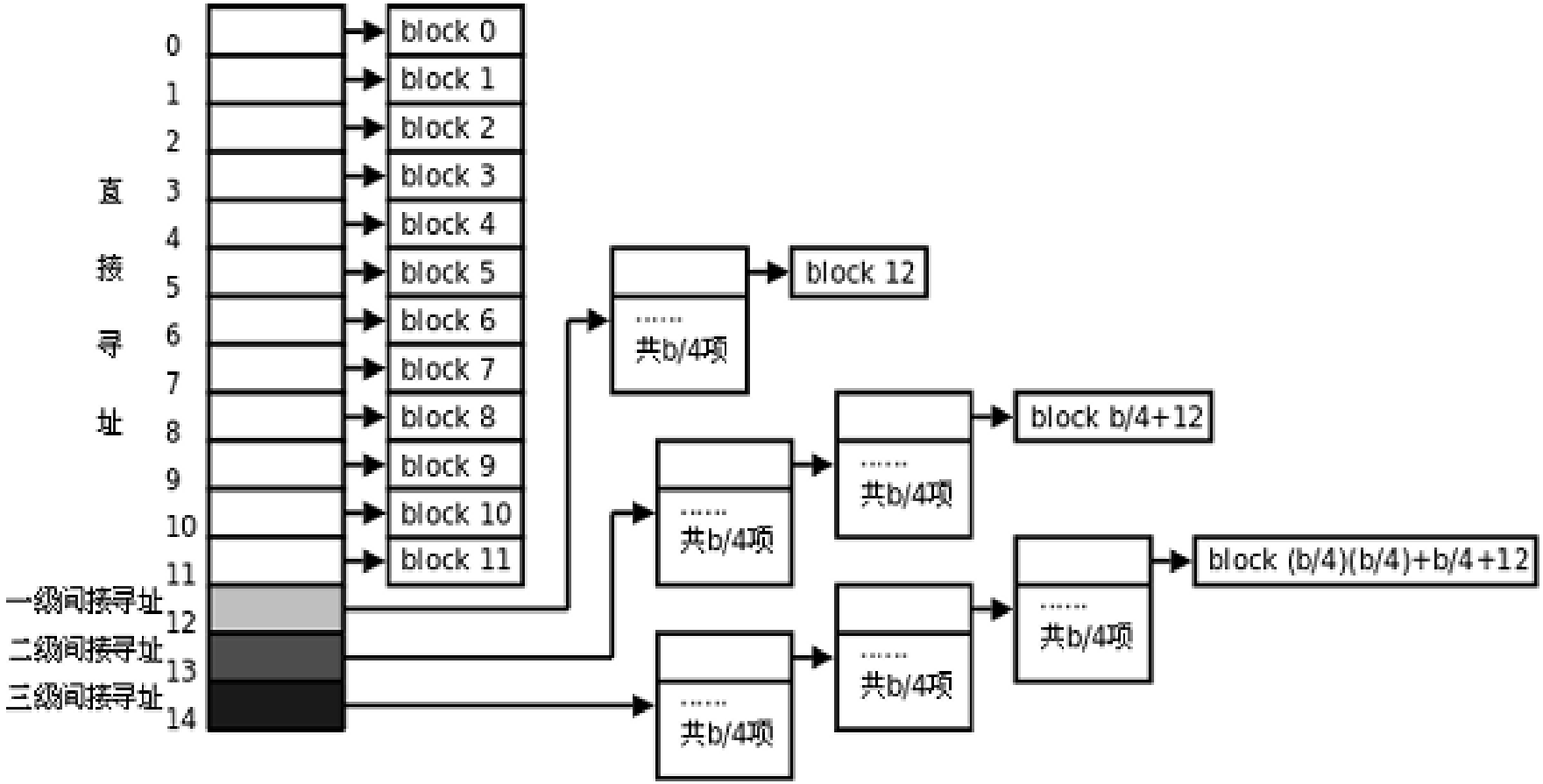
多级索引

- EXT2文件系统中的每个文件由一个inode描述，且只能由一个inode描述。
- inode与文件一起存放在外存，系统运行时，把inode写入内存建立映像，加快文件系统速度。



EXT2的inode中物理块指针示意图

EXT2中的数据块寻址



假如采用磁盘块大小为4KB，索引表项中块号信息占四个字节，那么三级索引能够支持的最大文件多大？

EXT2中的目录项和文件类型

- 在ext2文件系统中，目录是作为文件存储的。
- 这种文件的数据块中存放了该目录下的所有目录项

```
struct ext2_dir_entry_2 {
__le32 inode; // 文件入口的inode号，0表示该项未使用
__le16 rec_len; // 目录项长度
__u8 name_len; // 文件名包含的字符数
__u8 file_type; // 文件类型
char name[255]; // 文件名
};
```

目录项在include/linux/ext2_fs.h文件中定义。

EXT2支持的文件类型

- EXT2在目录项中存放了文件的类型信息。文件类型可以是0～7中的任意一个整数。它们分别代表如下含义：
  - 0：文件类型未知；
  - 1：普通文件类型；
  - 2：目录；
  - 3：字符设备；
  - 4：块设备；
  - 5：有名管道FIFO；
  - 6：套接字；
  - 7：符号链接

```
00539: /*
00540:  * Ext2 directory file types.  Only the low 3 bits are used.  The
00541:  * other bits are reserved for now.
00542:  */
00543: enum {
00544:     EXT2_FT_UNKNOWN,
00545:     EXT2_FT_REG_FILE,
00546:     EXT2_FT_DIR,
00547:     EXT2_FT_CHRDEV,
00548:     EXT2_FT_BLKDEV,
00549:     EXT2_FT_FIFO,
00550:     EXT2_FT_SOCK,
00551:     EXT2_FT_SYMLINK,
00552:     EXT2_FT_MAX
00553: };
```

管理ext2的磁盘空间

- 存储在磁盘上的文件与用户所“看到”的文件有所不同：
  - 用户感觉，文件在逻辑上是连续的
  - 而在磁盘上，存储文件数据的磁盘块可能分散在磁盘各处
- 磁盘碎片整理

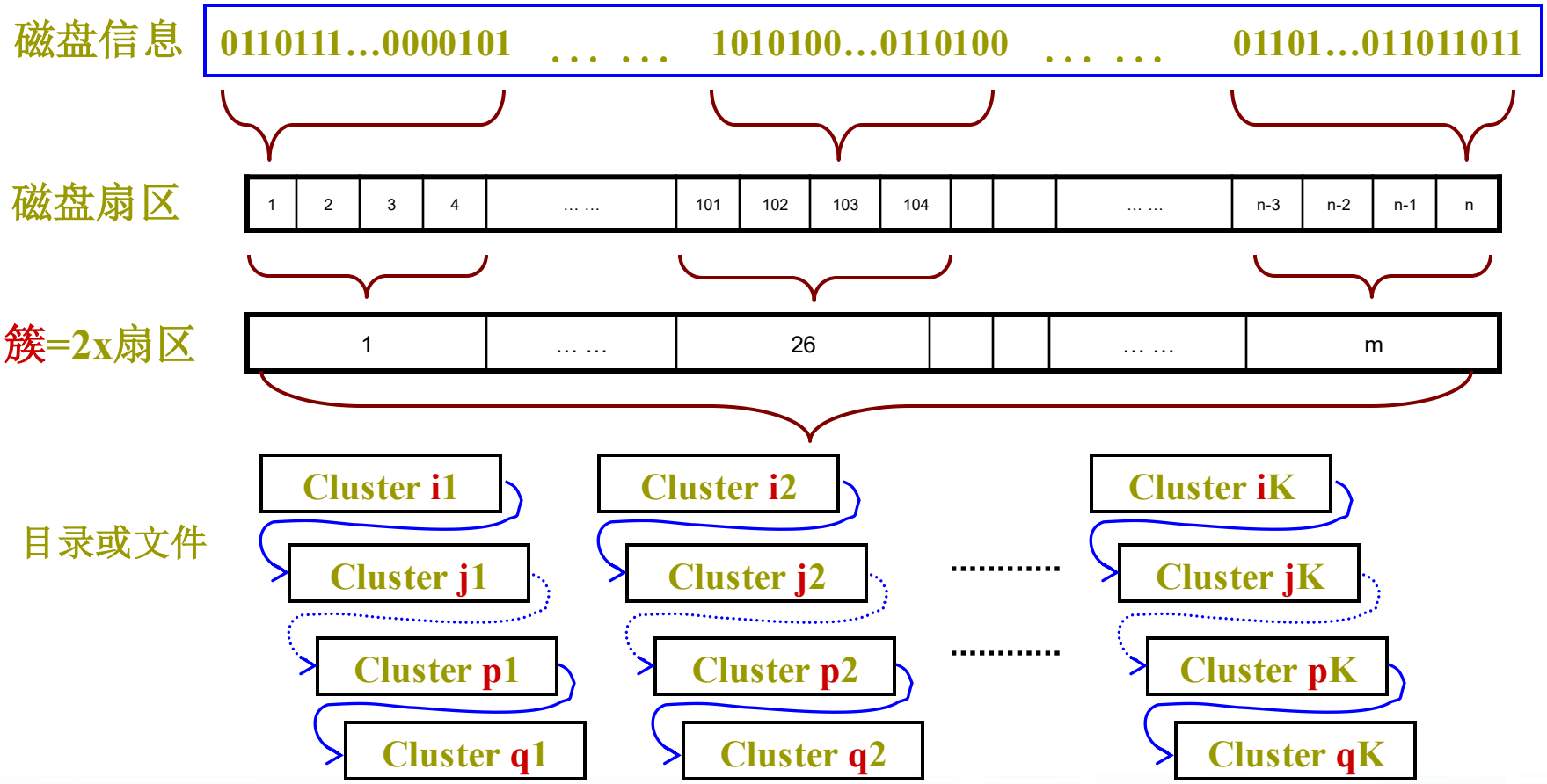
Module 7: 文件系统

1. 文件（File）和目录（Directory）
2. VSFS（Very Simple File System）简单文件系统
   a. Data Organization（基础）
   b. Access Methods（基础）
   c. Linux EXT2文件系统（基础）
   d. Windows FAT文件系统（基础）
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. Flash-based SSD

案例分析 #2:
Windows FAT文件系统

Windows的FAT文件系统实现

数据组织逻辑结结构



Windows的FAT文件系统实现

FAT卷结构示意图



引导代码

介质标志
磁头数
每磁道扇区数
扇区大小
扇区数/簇

总扇区数
保留扇区数
隐藏扇区数

FAT数目
每个FAT占扇区数
根目录项数

如果是引导分区,则存在该代码,否则空闲

软盘或硬盘等

该参数不是固定不变的, 一般卷越大簇越大



在FAT32中，引导扇区有备份

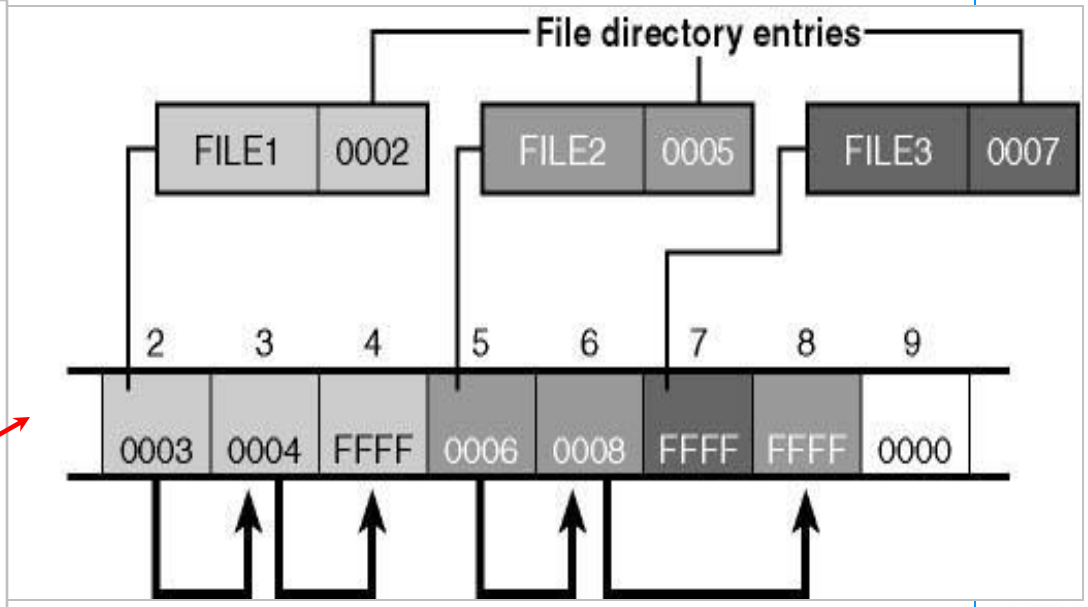
Windows的FAT文件系统实现

FAT卷结构示意图



功能： 记录和描述整个卷使用情况

1. FAT文件系统格式信息：
   FAT12/FAT16/FAT32
2. 卷上每一簇对应FAT中一项
   记录该簇使用情况
   包含：簇地址号和使用标志信息
   使用标志信息＝0 — 该簇空闲未用
   ≠0 — 该簇被占用
3. 每个目录/文件的文件分配链(簇链)
   链尾标志信息：
   0xFFF/0xFFFF/0xFFFFFFFF



Windows的FAT文件系统实现

FAT卷结构示意图



FAT2是FAT1的“镜像”备份

文件分配表对卷非常重要
它的内容破坏会导致部分文件无法访问，甚至导致整卷瘫痪

对FAT表备份是十分必要的
若FSD（文件系统驱动程序）不能正常访问FAT1，则会访问FAT2

Windows的FAT文件系统实现

FAT卷结构示意图



根目录区保存卷中根目录项内容

FAT12和FAT16卷中预留256个目录项空间(是1个文件)
即指定了根目录可以容纳的文件和目录数上限
FAT32卷中没有预留根目录空间，也无文件/目录数限制

FAT目录项大小为32字节(文件名遵循8.3命名规则)，保存：
文件名、文件尺寸、文件属性、起始簇号、
创建日期和时间、最后访问日期、最后修改日期和时间

如果目录/文件名为长文件名（非8.3规则），则通过增加若干个目录项的方法解决

Windows的FAT文件系统实现

FAT卷结构示意图

引导区	文件分配表 FAT1	文件分配表 FAT2	根目录	其他目录和所有文件

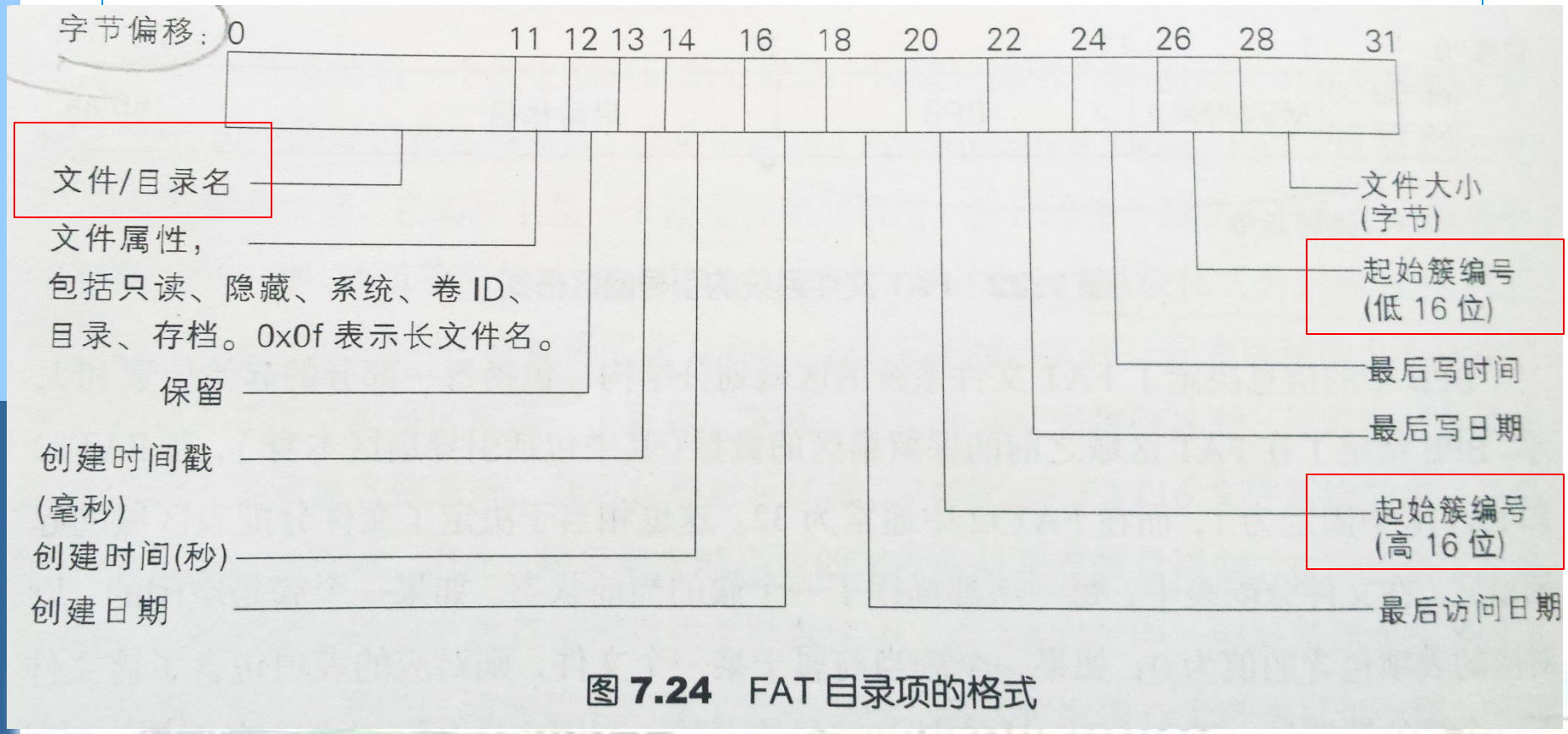
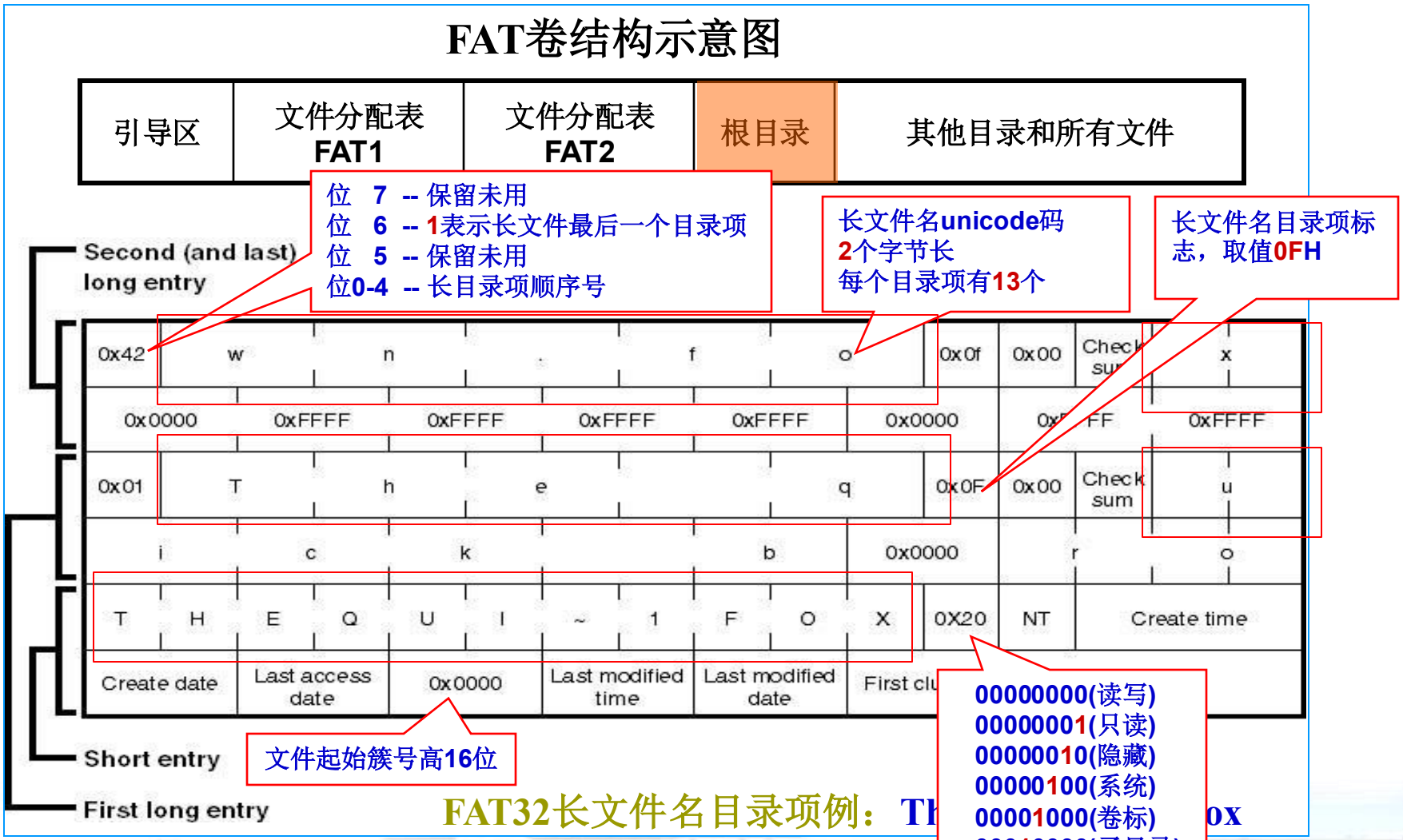


图 7.24 FAT 目录项的格式

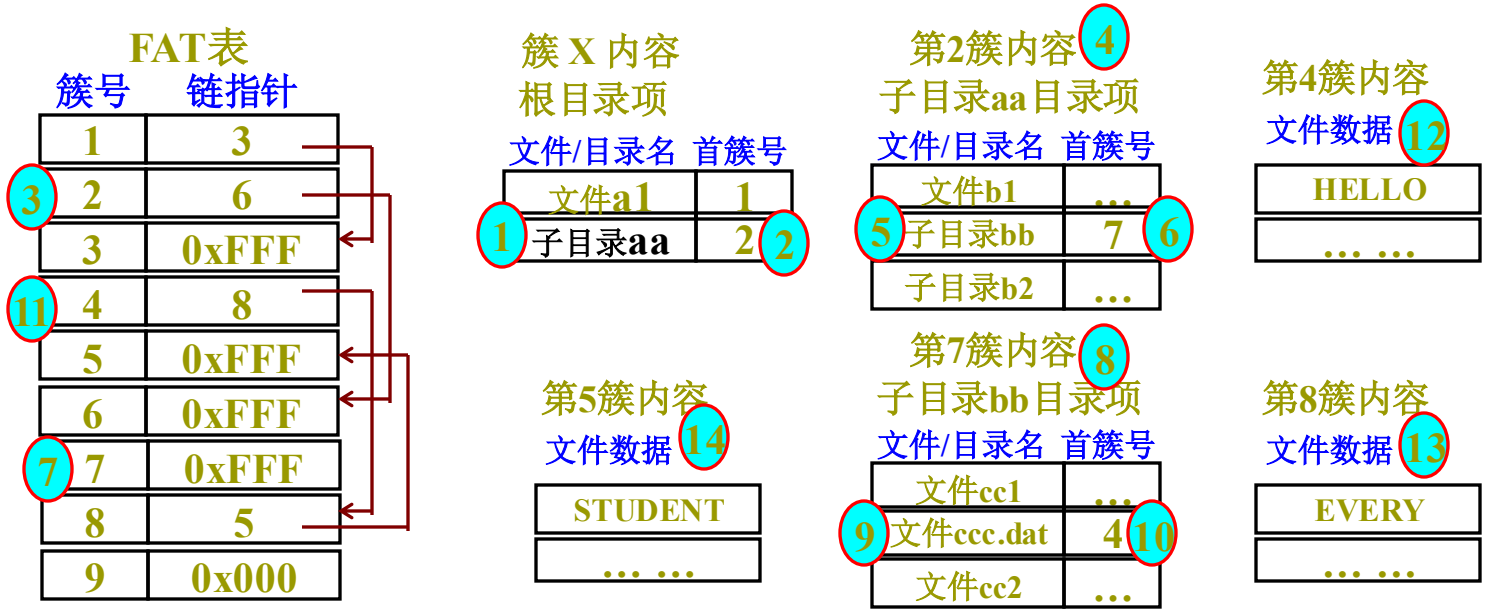
Windows的FAT文件系统实现

FAT卷结构示意图



FAT32长文件名目录项例：Th ox

Windows的FAT文件系统实现



文件访问过程：读出文件\aa\bb\ccc.dat的内容

1. 查“根目录”中目录项：找到含目录名=“aa”的目录项
2. 从aa目录项中查出该目录文件的首簇号＝2
3. 查FAT1中以第2簇为头的文件分配簇链，检索相应簇内容，找出含目录名=“bb”的目录项
4. 从bb目录项中查出该目录文件的首簇号=7
5. 查FAT1中以第7簇为头的文件分配簇链，检索相应簇内容，找出含文件名=“ccc.dat”的目录项
6. 从ccc.dat目录项中查出该文件的首簇号=4
7. 查FAT1中以第4簇为头的文件分配簇链，读出相应簇内容，即得到了文件ccc.dat的内容

Windows的FAT文件系统实现

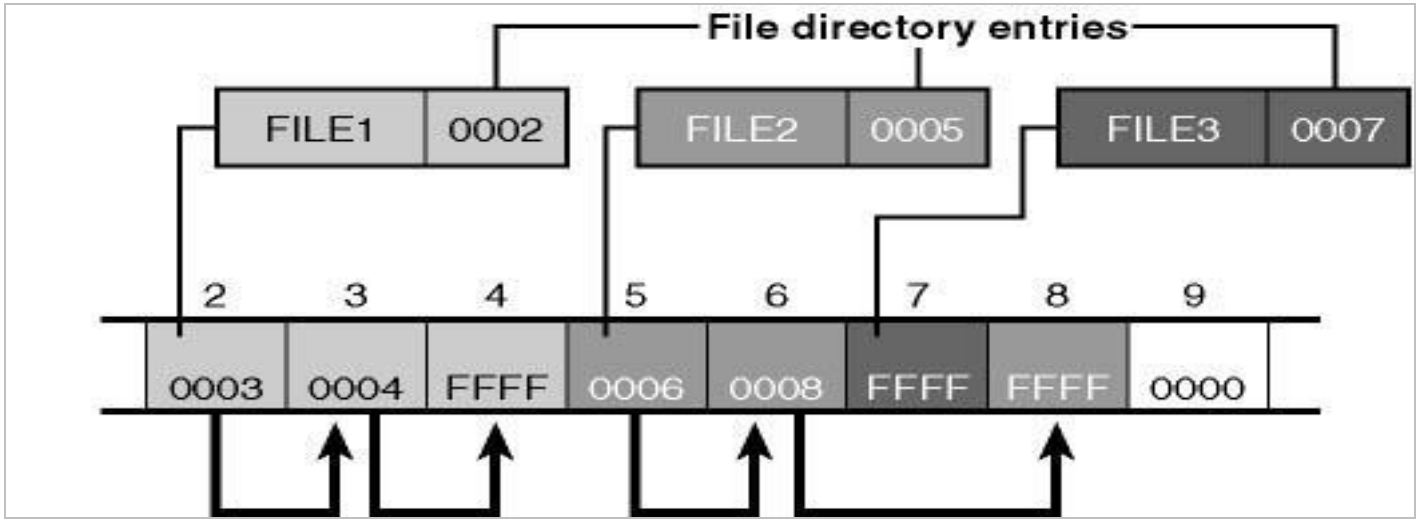


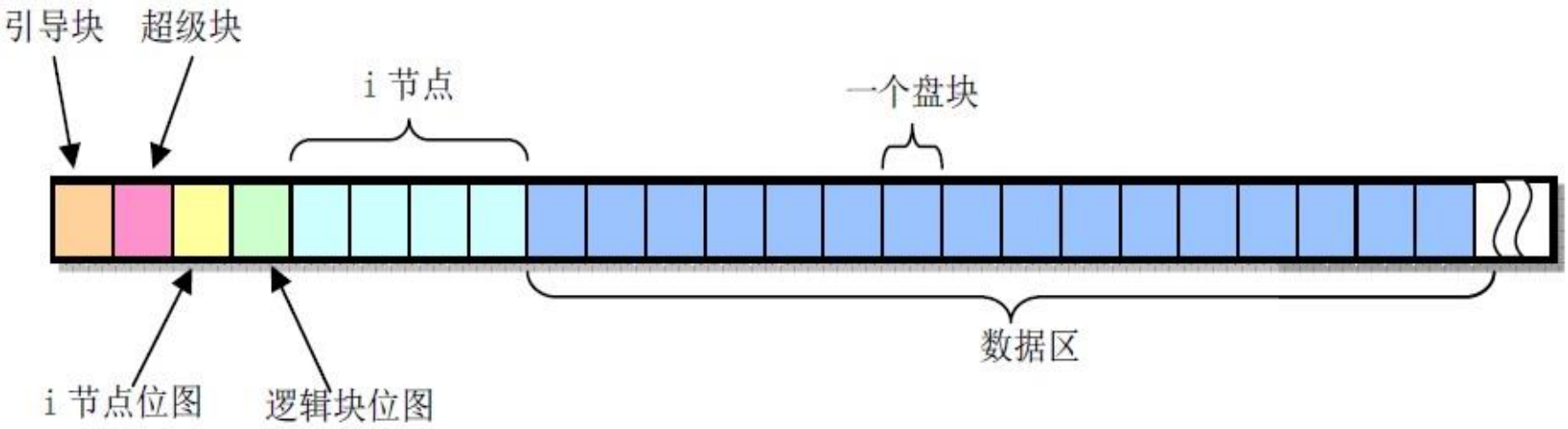


文件删除过程：删除文件\aa\bb\ccc.dat
文件建立过程：建立新文件\aa\fgx.txt

比较两种文件系统

引导区	文件分配表 **FAT1**	文件分配表 **FAT2**	根目录	其他目录和所有文件





Windows的FAT文件系统实现

FAT卷容量与簇大小的关系

- **FAT12** 最大簇数 = 2^{12}个=4K个
  规定簇大小=0.5KB～8KB
  FAT12卷大小≤32MB
- **FAT16** 最大簇数 = 2^{16}个=64K个
  规定簇大小=0.5KB～64KB
  FAT16卷大小≤4GB
- **FAT32** 最大簇数 = 2^{32}个=4G个
  限定使用2^{28}个=0.25G个
  规定簇大小=4KB～32KB
  FAT32卷大小 ≤ 1TG~8TB

FAT16/FAT32限制单个文件最大为4GB



W2K系统默认FAT16卷缺省簇大小

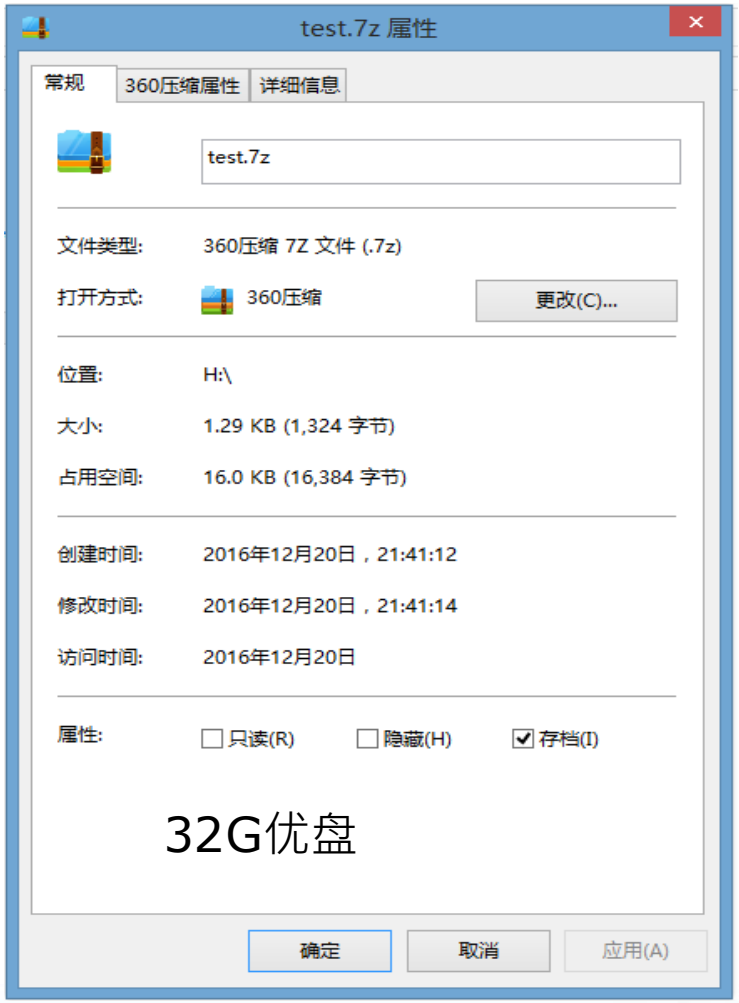
Volume Size	Cluster Size
0-32MB	512bytes
33MB-64MB	1KB
65MB-128MB	2KB
129MB-256MB	4KB
257MB-511MB	8KB
512MB-1023MB	16KB
1024MB-2047MB	32KB
2048MB-4095MB	64KB

FAT32卷缺省簇大小

Volume Size	Cluster Size
32MB-8GB	4KB
8GB-16GB	8KB
16GB-32GB	16KB
>32GB	32KB

为什么在同一种文件系统中簇的大小不统一？

Windows的FAT文件系统实现



32G优盘



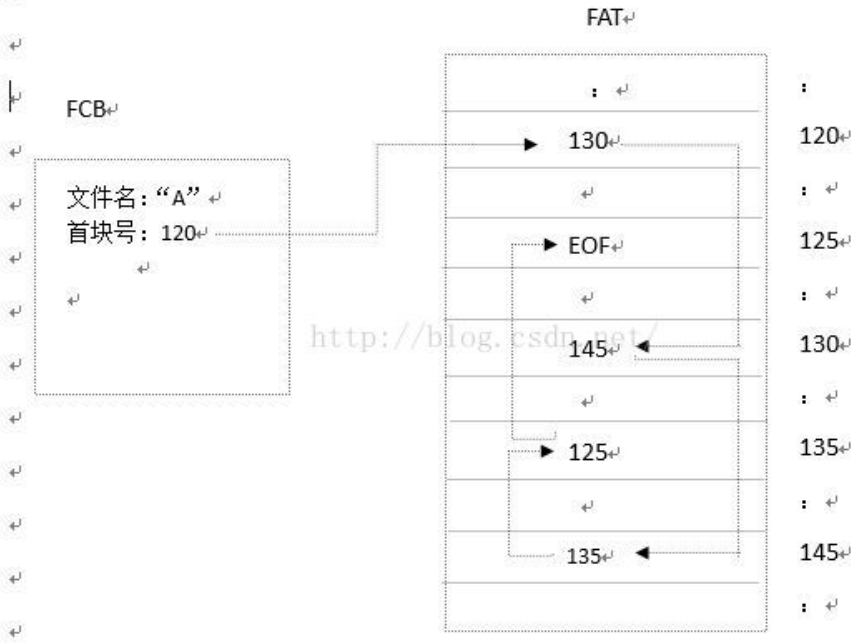
128G优盘

例题

假设某文件系统的硬盘空间为500MB，盘块大小为1KB，采用显示链接分配，请回答以下问题：

(1)如果约定FAT表项字节数需要与0.5字节对齐，那么其FAT表（文件分配表）需占用多少存储空间?

(2)如果文件A占用硬盘的盘块号依次为120、130、145、135、125共五个盘块，请画图示意文件A的FCB与FAT表的关系以及FAT表中各盘块间的链接情况。



http://blog.csdn.net/

例题

典型文件系统计算题：

1）假定磁盘块的簇大小为1KB，对于540MB的硬盘，其文件分配表（FAT）最少需要占用多少存储空间?

2）某文件系统采用多级索引方式，其inode中有13个地址项，其中直接索引10项，一次间接索引1项，二次间接索引1项，三次间接索引1项。数据块为4KB，磁盘地址用4B表示，这个文件系统允许的最大文件长度是多少?

答案：

1）总簇数：540MB/1KB=540K个

因为540K正好小于2^{20}，所以FAT的每个表目占20位，即2.5B，因此FAT的空间为2.5B*540K=1350KB.

2）每个块可以存储的地址数为4KB/4B=1024

最大文件的块数为$10+1024+1024^2+1024^3$

故最大文件大小为 $(10+1024+1024^2+1024^3)*4KB=4TB+4GB+4MB+40KB$

Module 7: 文件系统

1. 文件（File）和目录（Directory）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)（进阶）
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. Flash-based SSD

“The Old Unix FS” By Unix Wizard Ken Thompson



Data structures

- The Good Thing
  - Simple and supports the basic abstractions（抽象）.
  - Easy to use file system.
- The Problem
  - Terrible performance

Problem of Unix operating system

- Unix file system treated the disk as a **random-access memory.**
  - Example of random-access blocks with Four files.
    - Data blocks for each file can accessed by going back and forth the disk, because they are are **contiguous**（毗连的）**.**

A1	A2	B1	B2	C1	C2	D1	D2

    - File b and d is deleted.

A1	A2			C1	C2		

    - File E is created with free blocks. (**spread across** the block)

A1	A2	E1	E2	C1	C2	E3	E4

- Other Problem is the original block size was too small (512 bytes)

FFS: Disk Awareness is the solution

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

- FFS is **Fast File system** designed by a group at Berkeley.
- The design of FFS is that file system structures and allocation polices to be "disk aware" and improve performance.
  - Keep same API with file system. (`open()`, `read()`, `write()`, etc)
  - Changing the internal implementation.

Organizing Structure: The Cylinder Group

- FFS divides the disk into a bunch of groups. **(Cylinder Group)**
  - Modern file system call cylinder group as block group.

G0	G1	G2	G3	G4	G5	G6	G7	G8	G9

- These groups are uses to improve seek performance.
  - By placing two files within the same group.
  - Accessing one after the other **will not be long seeks** across the disk.
  - FFS needs to allocate files and directories within each of these groups.

Organizing Structure: The Cylinder Group (Cont.)



- Data structure for each cylinder group.
  - A copy of the **super block(S)** for reliability reason.
  - **inode bitmap(ib)** and **data bitmap(db)** to track free inode and data block.
  - **inodes** and **data block** are same to the previous very-simple file system(VSFS).

How To Allocate Files and Directories?

哈尔滨工业大学(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

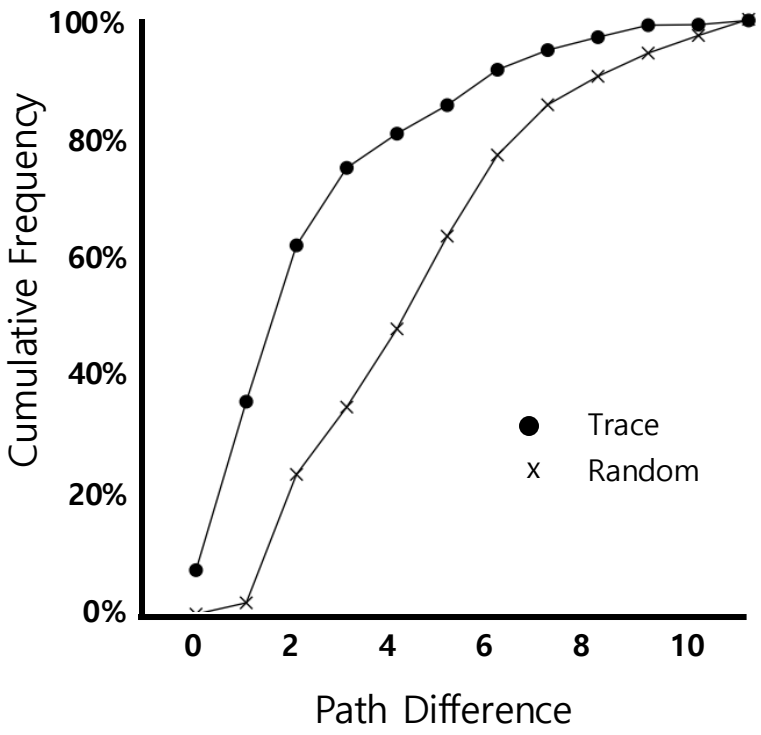
- Policy is "**keep related stuff together**"
- The placement of directories
  - Find the cylinder group with a low number of allocated directories and a high number of free inodes.
  - Put the directory data and inode in that group.
- The placement of files.
  - Allocate data blocks of a file in the same group as its inode
  - It places all files in the same group as their directory

FFS Locality for SEER Traces.

- How "**far away**" file accesses were from one another in the directory tree.

```
            proc/src/foo.c
            proc/src/bar.c
the distance of two file access is 1

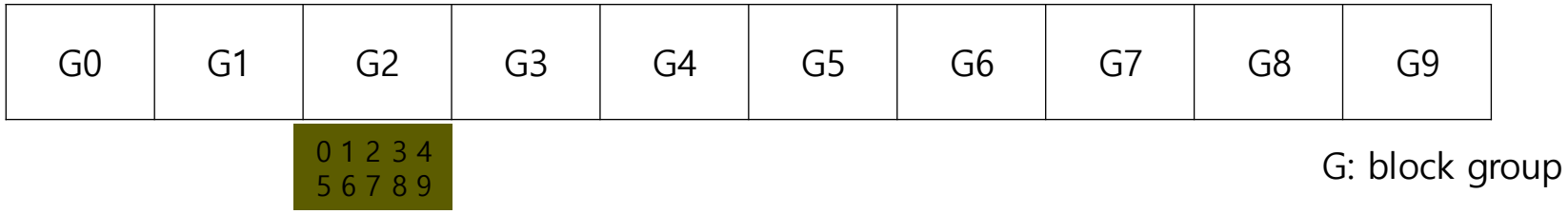
            proc/src/foo.c
            proc/obj/foo.o
the distance of two file access is 2
```



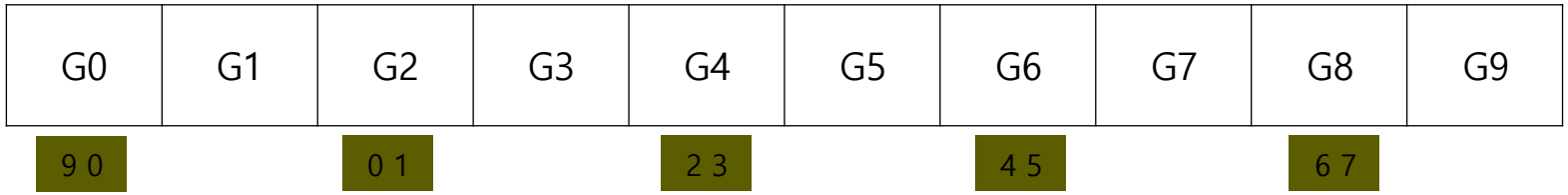
- 7% of file accesses to the same file
- Nearly 40% of file accesses in the same directory
- 25% of file accesses were two distances

The Large-File Exception

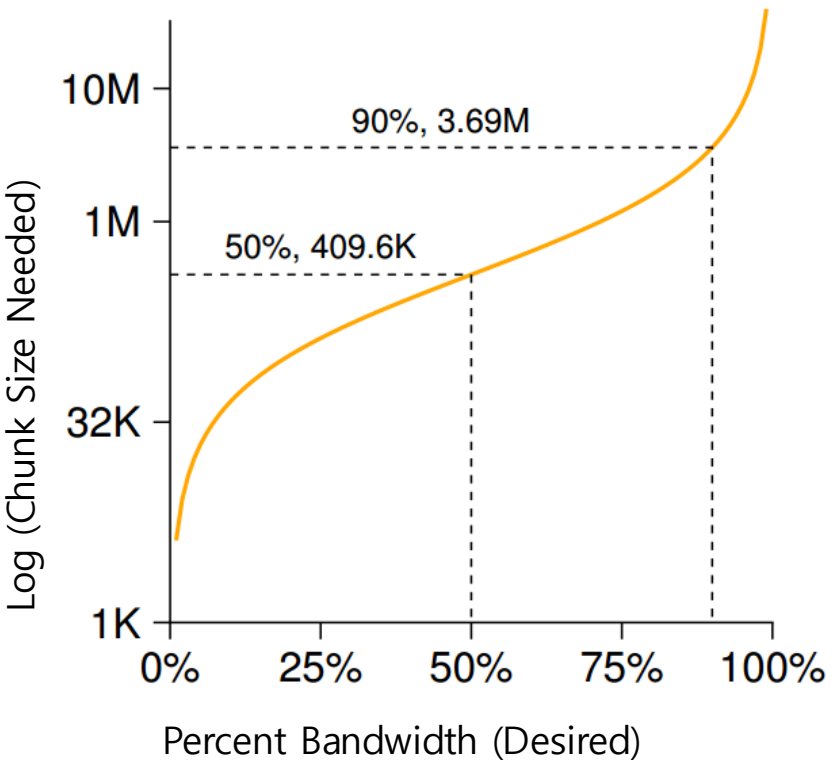
- General policy of file placement
  - Entirely fill the block group it is first place within
  - Hurt file-access locality from “related” file being placed



- For large files, chunks are spread across the disk
  - Hurt performance, but it can be addressed by choosing chunk size
  - **Amortization**: reducing overhead by doing more work



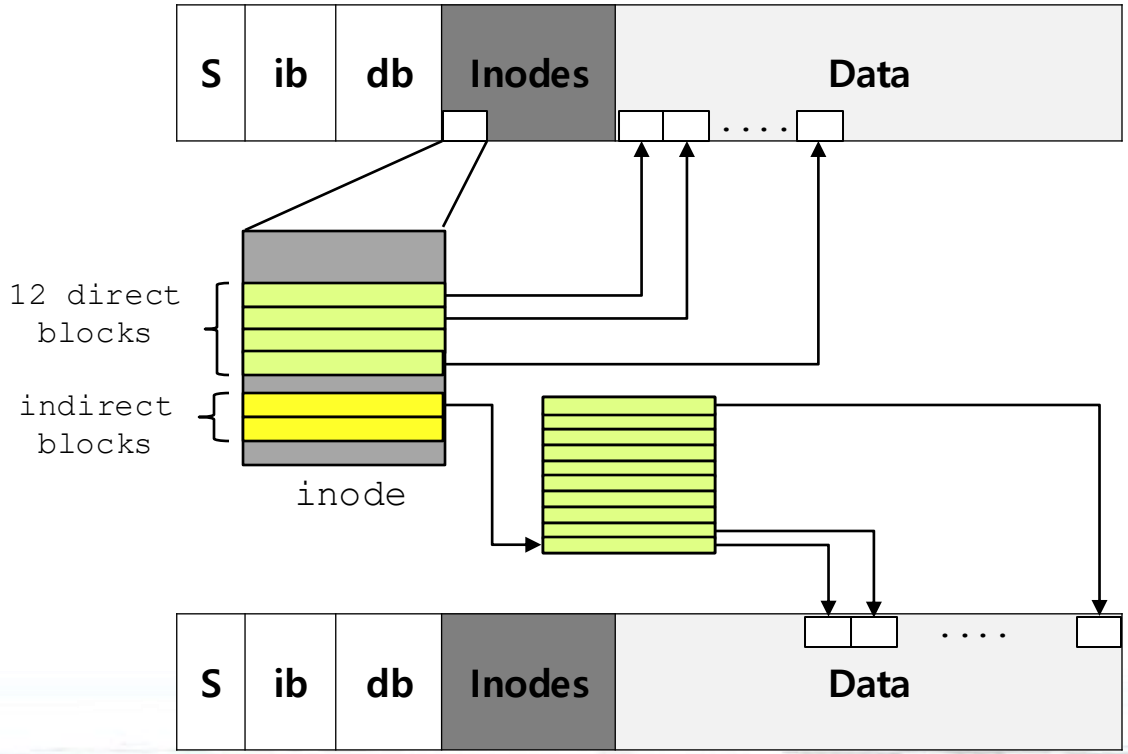
Amortization: How Big Do Chunks Have To Be?



- Computation of the size of chunk
  - Desire 50% of peak disk performance
    - half of time seeking and
      half of time transferring
  - Disk bandwidth: 40 MB/s
  - Positioning time: 10ms
  - $\frac{40\ \cancel{MB}}{\cancel{sec}} \cdot \frac{1024\ KB}{1\ \cancel{MB}} \cdot \frac{1\ \cancel{sec}}{1000\ \cancel{ms}} \cdot 10\ \cancel{ms} = 409.6\ KB$
    - Transfer only 409.6 KB every time seeking
  - 99% of peak performance on 3.69MB chunk size

The Large-File Exception in FSS

- A simple approach based on the structure of inode
  - Each subsequent **indirect blocks**, and all the **blocks it pointed to**, placed in **a different block group.**
  - Every **1024 blocks (4MB)** of the **file in a separate group**



A few other Things about FFS

- Internal fragmentation（内部碎片）
- Sub-blocks
  - Ex) Create a file with 1 KB : use two sub-blocks, not an entire 4-KB blocks
- Parameterization
- Track buffer
- Long file names
  - Enabling more expressive names in the file system
- Symbolic link

Module 7: 文件系统

1. 文件（File）和目录（Directory）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
   a. 概述（进阶）
   b. FSCK：文件系统检查器（进阶）
   c. Journaling：Write-Ahead Logging（进阶）
5. LFS日志结构FS (Log-structured File Systems)
6. Flash-based SSD

Crash Consistency

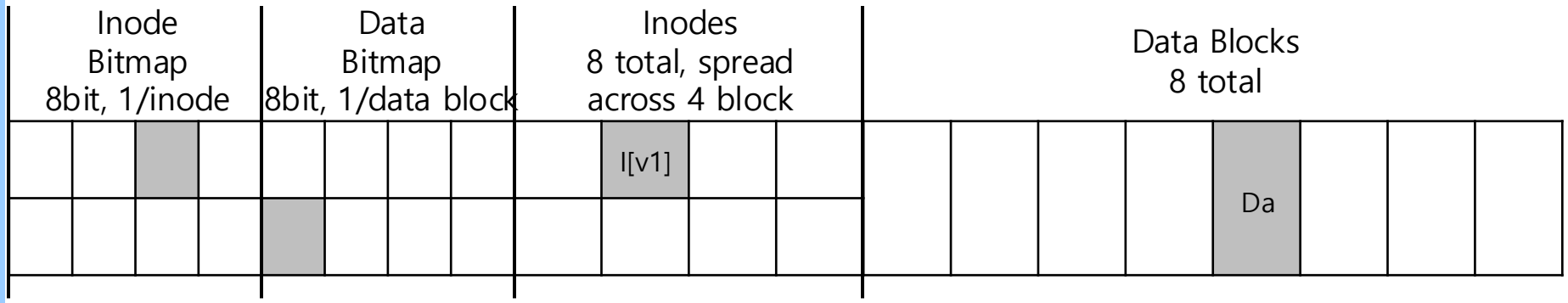
- Unlike most data structure, file system data structures must **persist**
  - They must survive over the long haul, stored on devices that retain data despite power loss.

- **FS的重要挑战**: How to update persistent data structure despite the presence of a **power loss** or **system crash**.

- We'll begin by examining the approach taken by older file systems.
  - **FSCK** (file system checker)
  - **Journaling** (write-ahead logging)

A Detailed Example (1)

- Workload
  - Append of a single data block(4KB) to an existing file
  - open() → lseek() → write() → close()

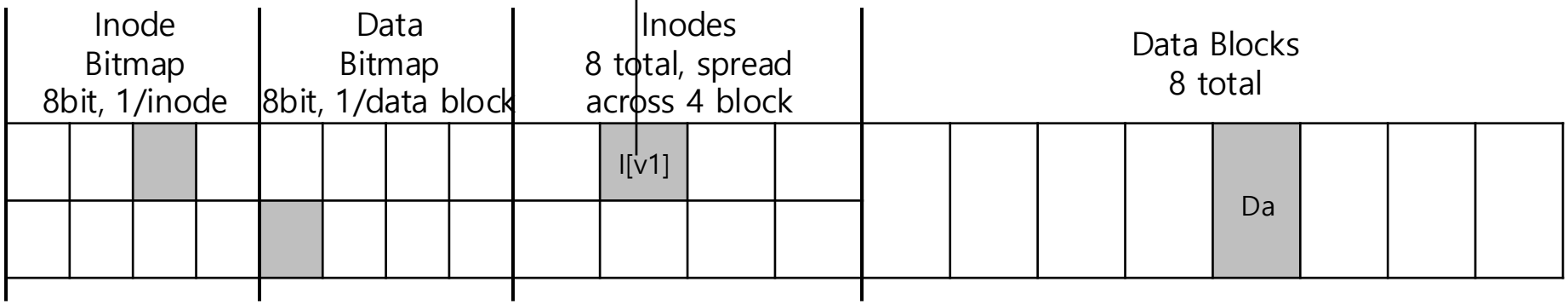


  - single inode is allocated (inode number 2)
  - single allocated data block (data block 4)
  - The inode is denoted I[v1]

A Detailed Example (2)

- Inside of I[v1] (inode, before update)

```
owner           : HIT
permissions     : read-only
size            : 1
pointer         : 4
pointer         : null
pointer         : null
pointer         : null
```

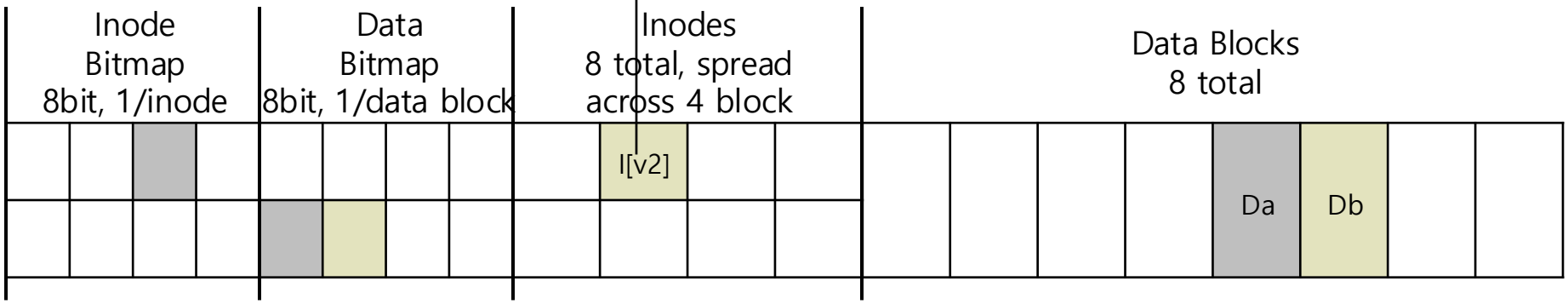


- Size of the file is 1 (one block allocated)
- First direct pointer points to block4
- All 3 other direct pointers are set to `null`(unused)

A Detailed Example (3)

- After update

```
owner           : HIT
permissions     : read-only
size            : 2
pointer         : 4
pointer         : 5
pointer         : null
pointer         : null
```



- Data bitmap is updated
- Inode is updated (I[v2])
- New data block is allocated (Db)

A Detailed Example (end)

- To achieve the transition, the system perform three separate writes to the disk.
  - One each of inode I[v2]
  - Data bitmap B[v2]
  - Data block (Db)
- These writes usually don't happen immediately
  - dirty inode, bitmap, and new data will sit in main memory
  - **page cache** or **buffer cache**
- If a crash happens after one or two of these write have taken place, but not all three, the file system could be left in a inconsistent state

Crash Scenario (1)

- Imagine only a single write succeeds; there are thus three possible outcomes
  1. Just the data block(Db) is written to disk
     - The data is on disk, but there is no inode
     - Thus, it is as if the write never occurred
     - This case is not a problem at all
  2. Just the updated inode(I[v2]) is written to disk
     - The inode points to the disk address (5, Db)
     - But, the Db has not yet been written there
     - We will read **garbage** data(old contents of address 5) from the disk
     - **Problem : file-system inconsistency**
  3. Just the updated bitmap (B[v2]) is written to disk
     - The bitmap indicates that block 5 is allocated
     - But there is no inode that points to it
     - Thus, the file system is inconsistent again
     - **Problem : space leak,** as block 5 would never be used by the file system

Crash Scenario (2)

- There are also three more crash scenarios. In these cases, two writes succeed and the last one fails
  1. The inode(I[v2]) and bitmap(B[v2]) are written to disk, but not data(Db)
     - The file system metadata is completely consistent
     - **Problem : Block 5 has garbage in it**
  2. The inode(I[v2]) and the data block(Db) are written, but not the bitmap(B[v2])
     - We have the inode pointing to the correct data on disk
     - **Problem : inconsistency between the inode and the old version of the bitmap(B1)**
  3. The bitmap(B[v2]) and data block(Db) are written, but not the inode(I[v2])
     - **Problem : inconsistency between the inode and the data bitmap**
     - We have no idea which file it belongs to

The Crash Consistency Problem

- What we'd like to do ideally is move the file system from on consistent state to another **atomically** 即：解决I/O操作的原子化问题
- Unfortunately, we can't do this easily
  - The disk only commits one write at a time
  - Crashes or power loss may occur between these updates
- We call this general problem the **crash-consistency（崩溃一致性）problem**

Module 7: 文件系统

1. 文件（File）和目录（Directory）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
   a. 概述（进阶）
   b. FSCK：文件系统检查器（进阶）
   c. Journaling：Write-Ahead Logging（进阶）
5. LFS日志结构FS (Log-structured File Systems)
6. Flash-based SSD

Solution #1:
FSCK: the File System Checker

The File System Checker (1)

- The File System Checker (**`fsck`**)
  - `fsck` is a Unix tool for finding inconsistencies and repairing them.
  - `fsck` check **super block, Free block, Inode state, Inode links**, etc.
  - Such an approach can't fix all problems
    - example : The file system looks consistent but the inode points to garbage data.
  - The only real goal is to make sure the file system metadata is internally consistent, 即FS数据结构之间的关系是符合逻辑的

The File System Checker (2)

- Basic summary of what `fsck` does:
  - **Superblock**
    - `fsck` first checks if the superblock looks reasonable
      - Sanity checks : file system size > number of blocks allocated
    - Goal : to find suspect superblock
    - In this case, the system may decide to use an alternate copy of the superblock
  - **Free blocks**
    - `fsck` scans the inodes, indirect blocks, dobule indirect blocks,
    - The only real goal is to make sure the file system metadata is internally consistent.
  - **Inode state**
    - Each inode is checked for corruption or other problem
      - Example : type checking(regular file, directory, symbolic link, etc)
    - If there are problems with the inode fields that are not easily fixed.
      - The inode is considered suspect and cleared by `fsck`

The File System Checker (3)

- **Inode Links**
  - `fsck` also verifies the link count of each allocated inode
    - To verify the link count, `fsck` scans through the entire directory tree
  - If there is a mismatch between the newly–calculated count and that found within an inode, corrective action must be taken
    - Usually by fixing the count with in the inode
  - If an allocated inode is discovered but no directory refers to it, it is moved to the lost+found directory
- **Duplicates**
  - fsck also checks for duplicated pointers
  - Example : Two different inodes refer to the same block
    - If on inode is obviously bad, it may cleared
    - Alternately, the pointed-to block could be copied

The File System Checker (4)

- **Bad blocks**
  - A check for bad block pointers is also performed while scanning through the list of all pointers
  - A pointer is considered “bad” if it obviously points to something outside it valid range
  - Example : It has an address that refers to a block greater than the partition size
    - In this case, `fsck` can’t do anything too intelligent; it just removes the pointer
- **Directory checks**
  - `fsck` does not understand the contents of user files
    - However, directories hold specifically formatted information created by the file system itself
    - Thus, `fsck` performs additional integrity checks on the contents of each directory
  - Example
    - making sure that “.” and “..” are the first entries
    - each inode referred to in a directory entry is allocated?
    - ensuring that no directory is linked to more than once in the entire hierarchy

The File System Checker (end)

- Buliding a working `fsck` requires intricate knowledge of the filesystem
- `fsck` have a bigger and fundamental problem: **too slow**
  - scanning the entire disk may take many minutes or hours
  - Performance of `fsck` became prohibitive.
    - as disk grew in capacity and RAIDs grew in popularity
- At a higher level, the basic premise of fsck seems just a tad irrational.
  - It is incredibly expensive to scan the entire disk
  - It works but is wasteful
  - Thus, as disk(and RAIDs) grew, researchers started to look for other solutions

Module 7: 文件系统

1. 文件（File）和目录（Directory）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
   a. 概述（进阶）
   b. FSCK：文件系统检查器（进阶）
   c. Journaling：Write-Ahead Logging（进阶）
5. LFS日志结构FS (Log-structured File Systems)
6. Flash-based SSD

Solution #2: Journaling: Write-Ahead Logging

Journaling (1)

- **Journaling** (**Write-Ahead Logging**)
  - When updating the disk, before over writing the structures in place, first write down a little note describing what you are about to do
  - Writing this note is the “write ahead” part, and we write it to a structure that we organize as a “log”
  - By writing the note to disk, you are guaranteeing that if a crash takes places during the update of the structures you are updating, you can go back and look at the note you made and try again
  - Thus, you will know exactly what to fix after a crash, instead of having to scan the entire disk

Journaling (Cont.)

- We'll describe how Linux ext3 incorporates journaling into the file system.
  - Most of the on-disk structures are identical to Linux ext2
  - The new key structure is the journal itself, 即：ext3引入的核心数据结构是journal的设计
  - It occupies some small amount of space within the partition or on another device, journal是per FS的，一般位于superblock以后，data group之前



Fig.1 Ext2 File system structure



Fig.2 Ext3 File system structure

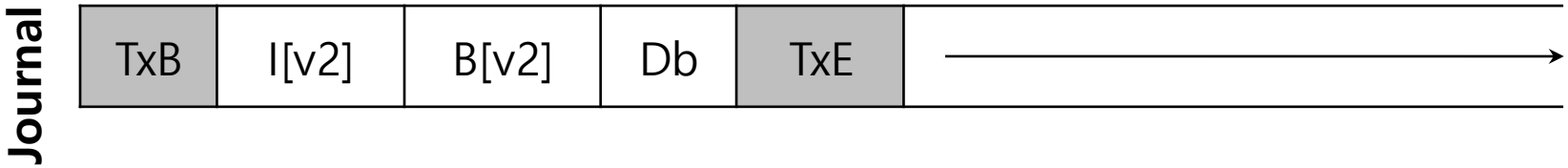
Data Journaling Principle

- Data journaling is available as a mode with the ext3 file system

- Example : our canonical update again
  - We wish to update inode (I[v2]), bitmap (B[v2]), and data block (Db) to disk
  - Before writing them to their final disk locations, we are now first going to write them to the log(a.k.a. journal), 即：两阶段提交原则

Data Journaling Structure

- Example : our canonical update again (Cont.)



- TxB: Transaction begin block
  - It contains some kind of **transaction**（事务）**identifier(TID)**
- The middle three blocks just contain the exact content of the blocks themselves
  - This is known as **physical logging**
- TxE: Transaction end block
  - Marker of the end of this transaction
  - It also contain the TID

Data Journaling Checkpoint

- **Checkpoint** 即：可恢复点
  - Once this transaction is safely on disk, we are ready to overwrite the old structures in the file system
  - This process is called **checkpointing**, Thus, to checkpoint the file system, we issue the writes I[v2], B[v2], and Db to their disk locations

Data Journaling Procedure

- Our initial sequence of operations:
  1. **Journal write**
     - Write the transaction to the log and wait for these writes to complete
     - TxB, all pending data, metadata updates, TxE
  2. **Checkpoint**
     - Write the pending metadata and data updates to their final locations

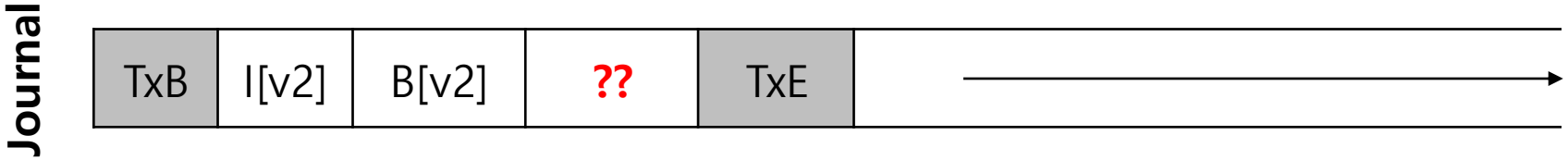
如何应对写journal阶段的crash

- When a crash occurs during the writes to the journal
  1. **Transaction each one at a time**
     - 5 transactions (TxB, I[v2], B[v2], Dnb, TxE)
     - This is slow because of waiting for each to complete

如何应对写journal阶段的crash (Cont.)

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

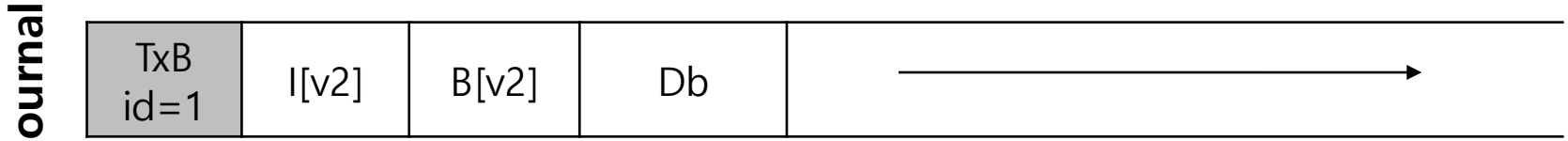
2. **Transaction all block writes at once**
   - Five writes -> a single sequential write : Faster way
   - However, this is unsafe
   - Given such a big write, the disk internally may perform scheduling and complete small pieces of the big write in any order，大写请求可能被设备内部切分成若干小请求
   - Thus, the disk internally may (1) write TxB, I[v2], B[v2], and TxE and only later (2) write Db
   - Unfortunately, if the disk loses power between (1) and (2)



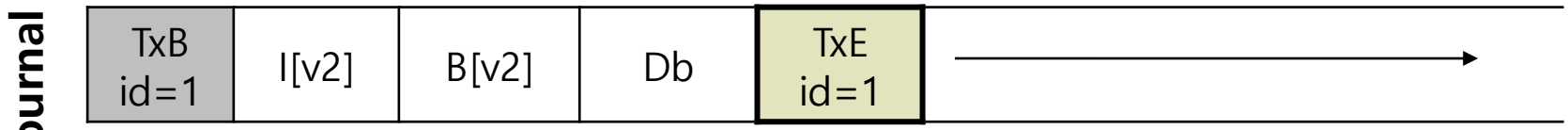
   - Transaction looks like a valid transaction.
     - Further, the file system can't look at that forth block and know it is wrong.
     - It is much worse if it happens to a critical piece of file system, such as superblock.

如何应对写journal阶段的crash (Cont.)

2. **Transaction all block writes at once (Cont.)**
   - To avoid this problem, the file system issues the transactional write in two steps
   - First, writes all blokes **except the TxE block** to journal



   - Second, The file system issues the write of the TxE



   - An important aspect of this process is the atomicity guarantee provided by the disk.
     - The disk guarantees that any 512-byte write either happen or not
     - Thus, **TxE should be a single 512-byte block**

因此，把Data Journaling协议优化成:

1. Journal write : write the contents of the transaction to the log
2. **Journal commit (added) :** write the transaction commit block
3. Checkpoint **:** write the contents of the update to their locations

Data Journaling 恢复

- Recovery
  - If the crash happens **before the transactions** is written to the log
    - The pending update is **skipped**
  - If the crash happens **after the transactions** is written to the log, but **before the checkpoint**
    - **Recover** the update as follow:
      - Scan the log and lock for transactions that have committed to the disk
      - Transactions are replayed

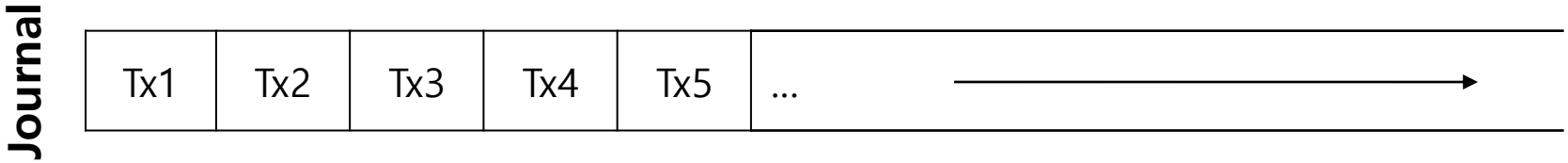
Batching Log Updates

- If we create two files in same directory, same inode, directory entry block is to the log and committed twice.

- To reduce excessive write traffic to disk, journaling manage the **global transaction**.
  - Write the content of the global transaction forced by synchronous request.
  - Write the content of the global transaction after timeout of 5 seconds.

Making The log Finite：问题描述

- The log is of a finite size
  - Two problems arise when the log becomes full



1. The larger the log, the longer recovery will take
   - Simpler but less critical
2. No further transactions can be committed to the disk
   - Thus making the file system “less than useful”

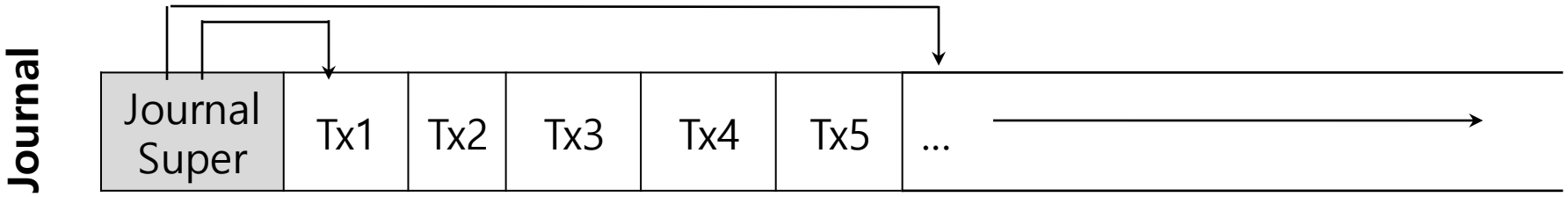
Making The log Finite：基本思路

- To address these problems, journaling file systems treat the log as a circular data structure, re-using it over and over
  - This is why the journal is referred to as a circular log.

- To do so, the file system must take action some time after a checkpoint
  - Specifically, once a transaction has been checkpointed, the file system should free the space

Journal Super Block

- journal super block
  - Mark the oldest and newest transactions in the log.
  - The journaling system records which transactions have not been check pointed.



- **Data Journaling协议最终版：**
  1. Journal write
  2. Journal commit
  3. checkpoint
  4. **Free**
     - Some time later, mark the transaction free in the journal by updating the journal Superblock

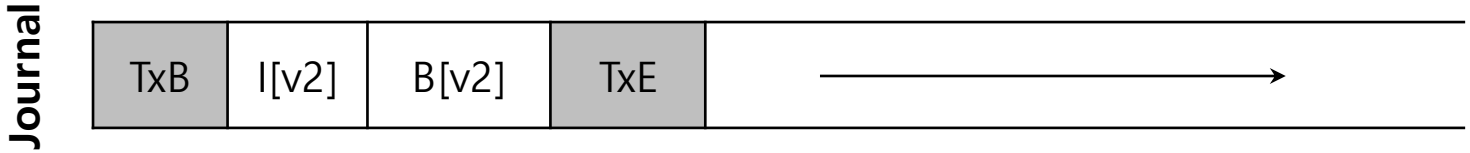
Metadata Journaling (1)

- There is a still problem : writing every data block to disk **twice**
  - Commit to log (journal file)
  - Checkpoint to on-disk location.

- People have tried a few different things in order to speed up performance.
  - Example : A simpler form of journaling is called **ordered journaling (metadata journaling)**
    - **User data is not written to the journal**

Metadata Journaling (2)

- Thus, the following information would be written to the journal



- The data block Db, previously written to the log, would instead be written to the file system proper

Metadata Journaling (3)

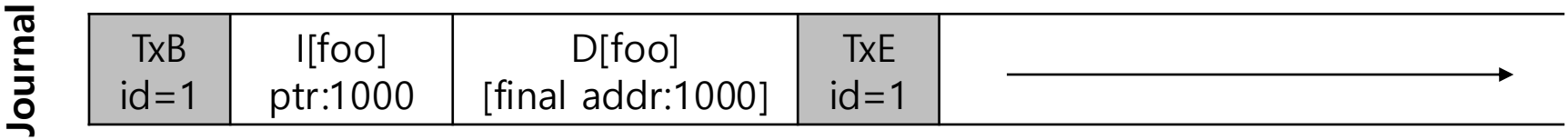
- The modification does raise an interesting question: **when should we write data blocks to disk?**
- Let's consider an example
  1. Write Data to disk after the transaction
     - Unfortunately, this approach has a problem
     - The file system is consistent but I[v2] may end up pointing to garbage data
  2. Write Data to disk before the transaction
     - It ensures the problems

Metadata Journaling (end)

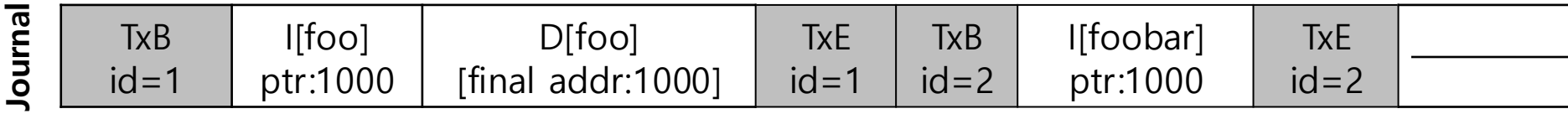
- Specifically, the protocol is as follows:
  1. **Data Write(added)**: Write data to final location
  2. **Journal metadata write(added)**: Write the begin and metadata to the log
  3. Journal commit
  4. Checkpoint metadata
  5. Free

Tricky case: Block Reuse (1)

- Some metadatas should not be replayed.
- Example
  1. Directory “foo” is updated.

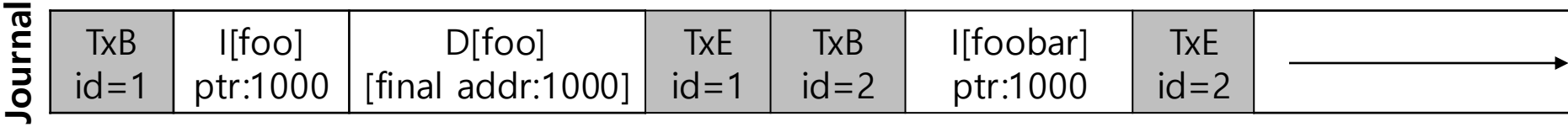


  2. Directory “foo” id deleted. block 1000 is freed up for reuse
  3. User Create a file “foobar”, reusing block 1000 for data



Tricky case: Block Reuse (2)

4. Now assume a crash occurs and all of this information is still in the log.



5. During replay, the recovery process replays everything in the log
   - Including the write of directory data in block 1000
6. The replay thus overwrites the user data of current file `foobar` with old directory contents

Tricky case: Block Reuse (2)

- Solution
  - What Linux ext3 does instead is to add a **new type** of record to the journal, Known as a **revoke**（撤销）record
  - When replaying the journal, the system first scans for such revoke records
  - Any such revoked data is never replayed

Data Journaling Timeline

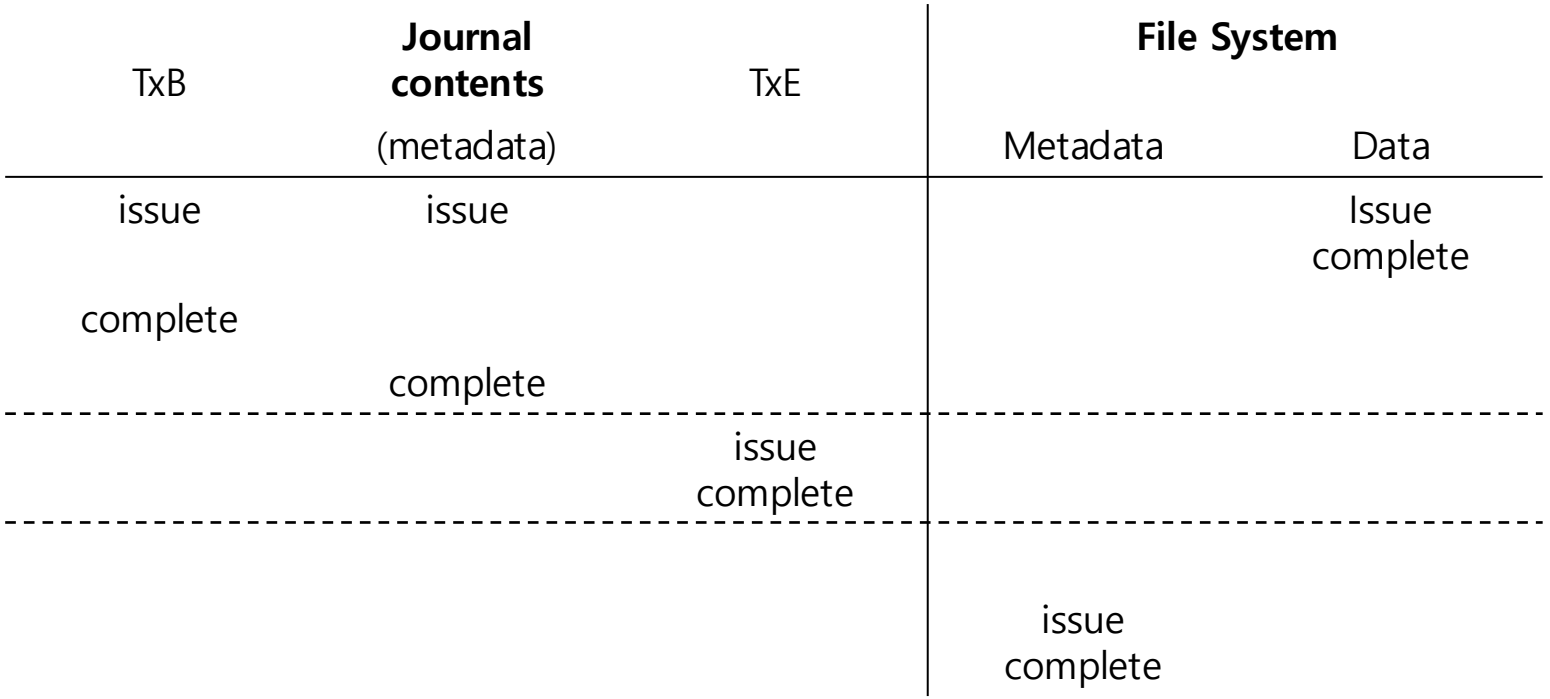
Journal contents				File System	
TxB	(metadata)	(data)	TxE	Metadata	Data
issue complete	issue complete	issue			
		complete	issue complete		
				issue complete	issue complete

protocol when journaling data and metadata

Note: Horizontal dashed lines show where write-ordering requirements must be obeyed

- E.g. the writes of the transaction begin block (TxB) and the contents of the transaction can logically be issued at the same time, and thus can be completed in any order
- the write to the transaction end block (TxE) must not be issued until said previous writes complete.
- check-point writes to data and metadata blocks cannot begin until the TxE has committed

Metadata Journaling Timeline



protocol when journaling only metadata

Note: Horizontal dashed lines show where write-ordering requirements must be obeyed

- E.g. the writes of the transaction begin block (TxB) and the contents of the transaction can logically be issued at the same time, and thus can be completed in any order
- the write to the transaction end block (TxE) must not be issued until said previous writes complete.
- check-point writes to data and metadata blocks cannot begin until the TxE has committed

Module 7: 文件系统

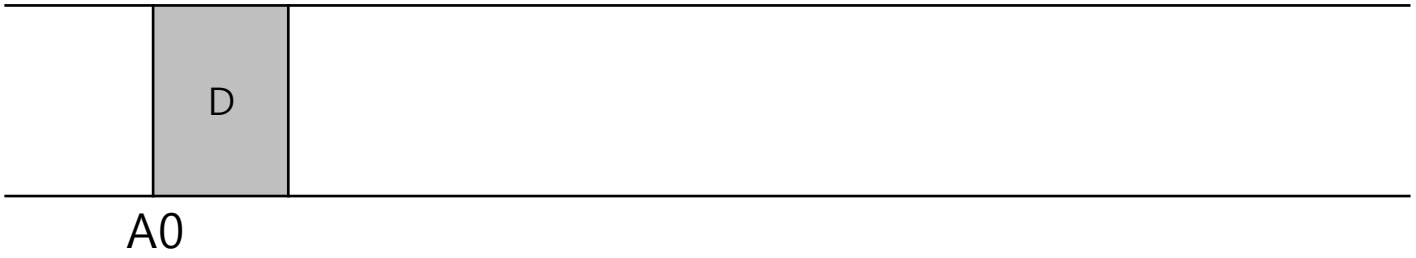
1. 文件（File）和目录（Directory）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)（进阶）
6. Flash-based SSD

LFS: Log-structured File System

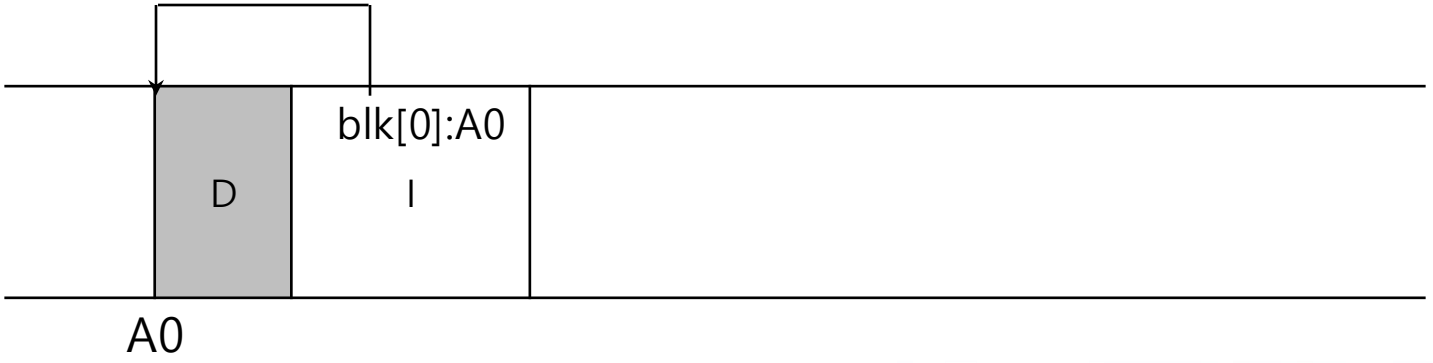
- Memory sizes were growing.
- Large gap between random IO and sequential IO performance.
- Existing File System perform poorly on common workloads.
- File System were not RAID-aware.

Writing to Disk Sequentially

- How do we transform all updates to file-system state into a series of sequntial writes to disk?
  - data update

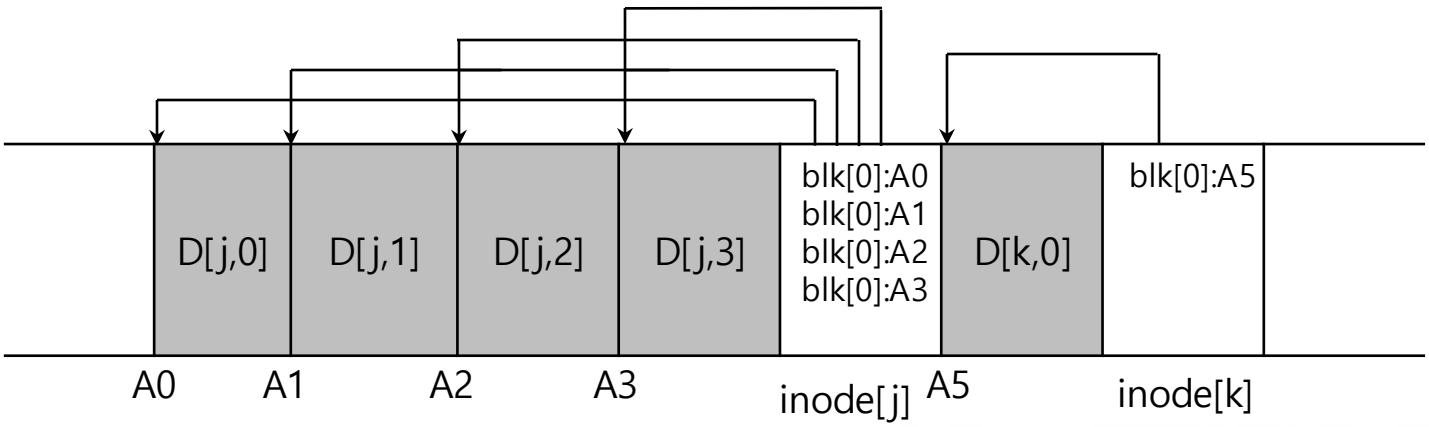


  - metadata needs to be updated too. (Ex. inode)



Writing to Disk Sequentially and Effectively

- Writing single blocks sequentially does not guarantee efficient writes
  - After writing into A0, next write to A1 will be delayed by disk rotation
- Write buffering for effectiveness
  - Keeps track of updates in **memory buffer** (also called **segment**)
  - Writes them to disk all at once, when it has sufficient number of updates.



How Much to Buffer?

- Each write to disk has fixed overhead of positioning
  - Time to write out *D* MB

$$T_{write} = T_{position} + \frac{D}{R_{peak}} \quad (43.1)$$

($T_{position}$: positioning time, R_{peak}: disk transfer rate)

- To amortize the cost, how much should LFS buffer before writing?
  - Effective rate of writing can be denoted as follows

$$R_{effecitve} = \frac{D}{T_{write}} = \frac{D}{T_{position} + \frac{D}{R_{peak}}} \quad (43.2)$$

- Assume that $R_{effecitve} = F \times R_{peak}$ (F: fraction of peak rate, 0 < F < 1), then

$$R_{effecitve} = \frac{D}{T_{position} + \frac{D}{R_{peak}}} = F \times R_{peak} \quad (43.3)$$

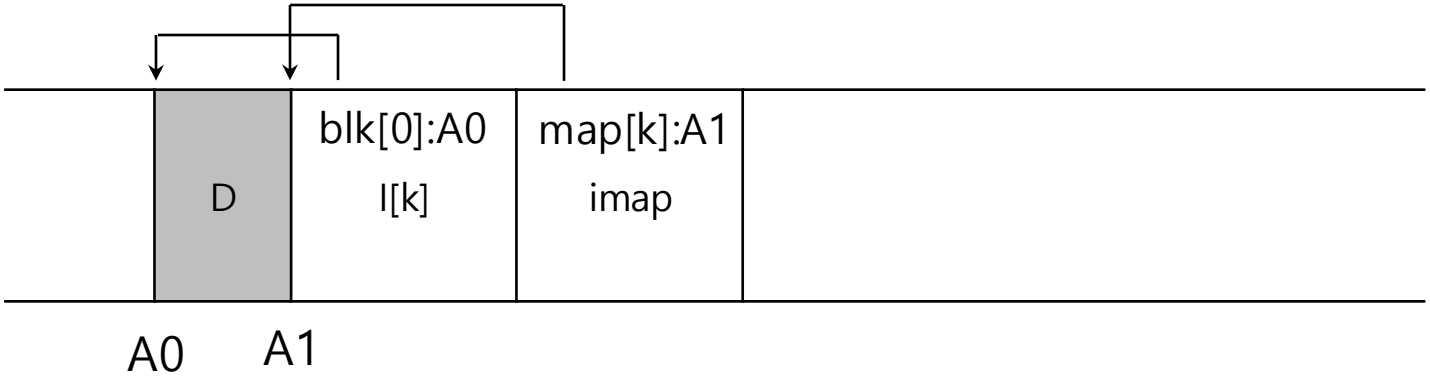
- Solve for *D*

$$\mathrm{D} = \frac{F}{1-F} \times R_{peak} \times T_{position} \quad (43.6)$$

- If we want F to be 0.9 when $T_{position} = 10msec$ and $R_{peak} = 100MB/s$, then $D = 9MB$ by the equation.
  - Segment size should be 9MB at least.

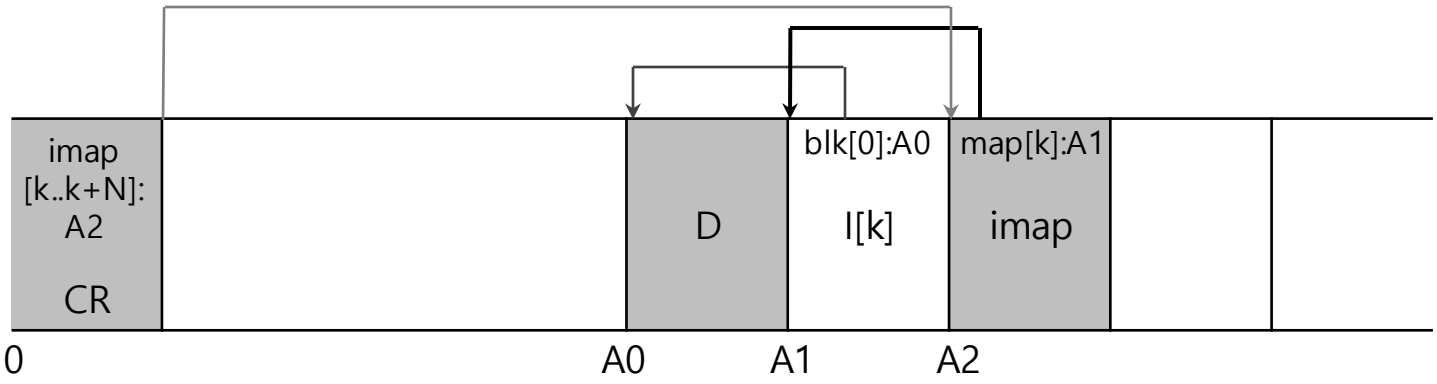
Finding Inode in LFS

- Inodes are scattered throughout the disk!
- Solution is through indirection “Inode Map” (imap)
- LFS place the chunks of the inode map right next to where it is writing all of the other new information, 即把imap区域放在inode的后续存储位置



The CR（Checkpoint Region）

- How to find the inode map, spread across the disk?
  - The LFS File system have fixed location on disk to begin a file lookup
- **Checkpoint Region** contains pointers to the latest of the inode map
  - Only updated periodically (ex. Every 30 seconds)
    → performance is not ill-affected

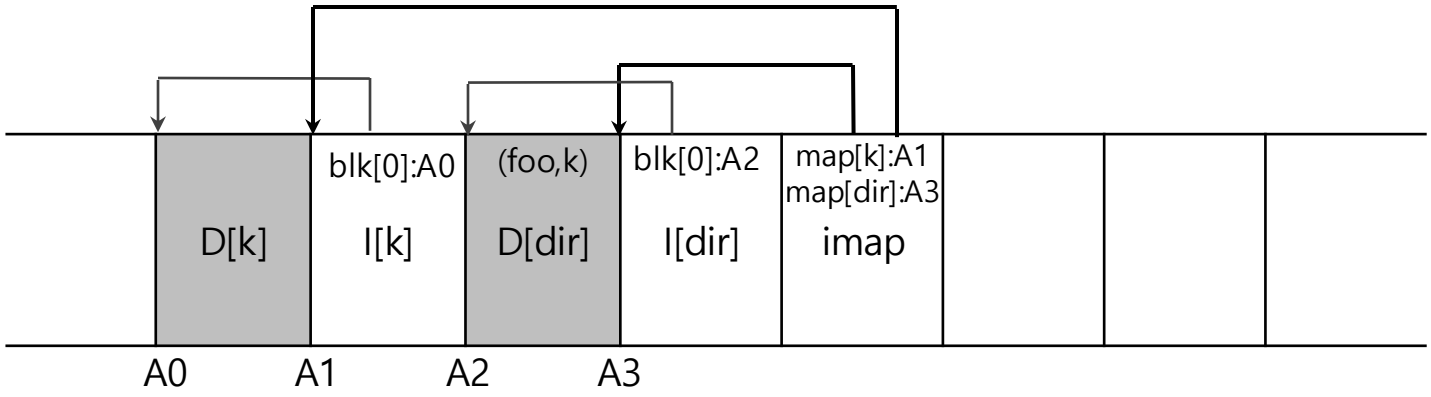


Reading a File from Disk: A Recap

- Read checkpoint region
- Read entire inode map and cache it in memory
- Read the most recent inode
- Read a block from file by using direct or indirect or doubly-indirect pointers

What About Directories?

- Directory structure of LFS is basically identical to classic UNIX file systems.
  - Directory is a file which data blocks consist of directory information

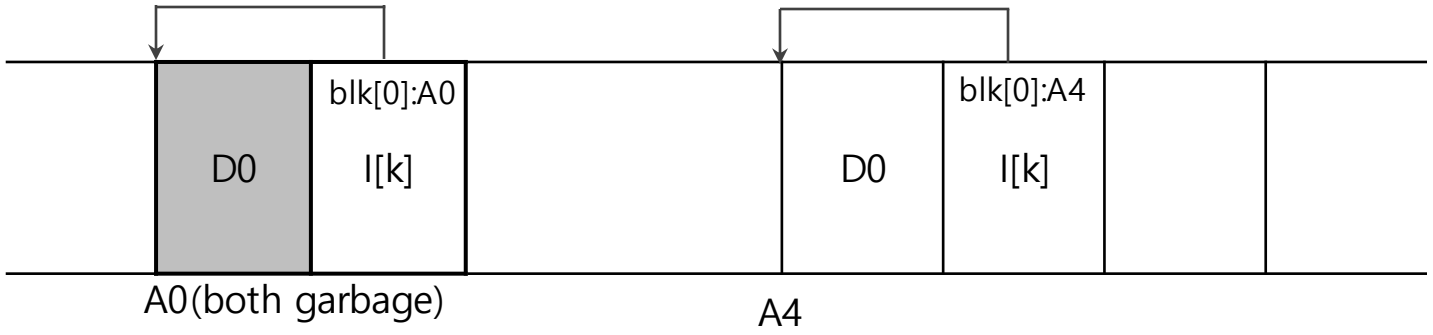


Garbage Collection

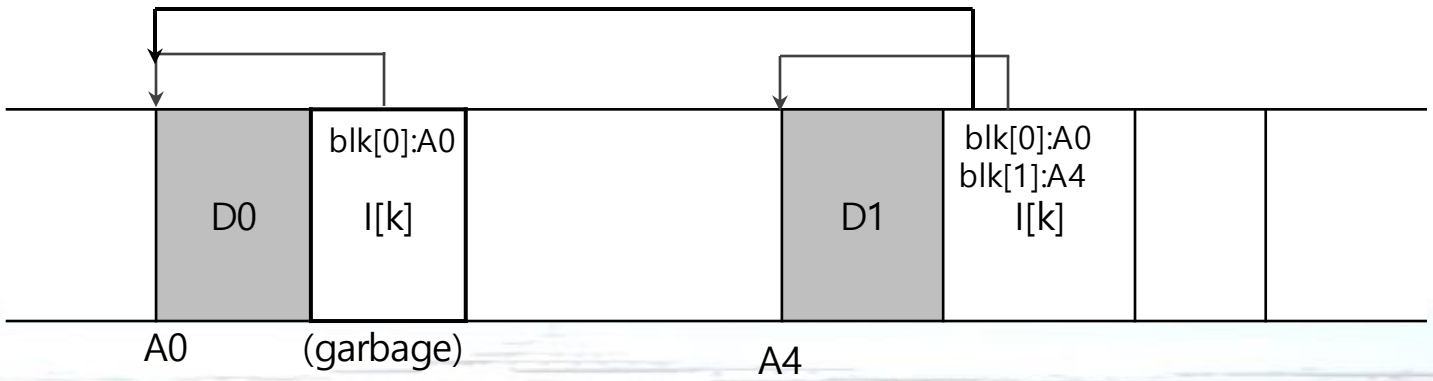
- LFS keeps writing newer version of file to new locations.
- Thus, LFS leaves the older versions of file structures all over the disk, call as garbage.

Examples: Garbage

- For a file with a singe data block
  - Overwrite the data block: both old data block and inode become garbage



  - Append a block to that original file k: old inode becomes garbage



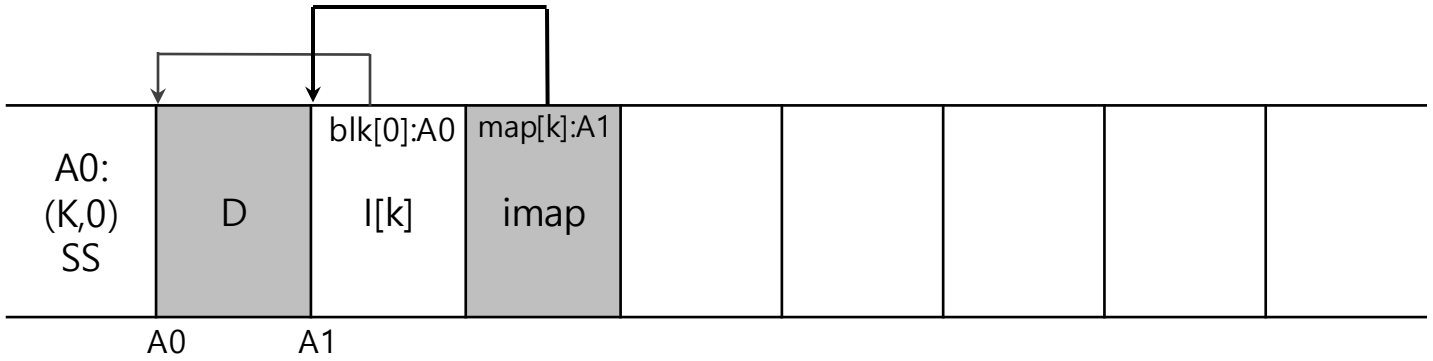
Handling older versions of inodes and data blocks

- One possibility: **Versioning file system**
  - keep the older versions around
  - Users can restore old file versions

- LFS approach: **Garbage Collection**
  - Keep only the latest live version and periodically clean old dead versions
  - Segment-by-segment basis
    - Block-by-block basis cleaner eventually make free holes in random location
      → Writes can not be sequential anymore

Determining Block Liveness

- **Segment summary block (SS)**
  - Located in each segment
  - Inode number and offset for each data block are recorded
- Determining Liveness
  - The block is live if the latest inode indicates the block



  - **Version number** can be used for efficient liveness determining

Which Blocks to Clean, and When?

- When to clean
  - Periodically
  - During idle time
  - When the disk is full
- Which blocks to clean
  - Segregate hot/cold segments
    - Hot segment: frequently over-written
      → more blocks are getting over-written if we wait a long time before cleaning
    - Cold segment: relatively stable
      → May have a few dead blocks, but the other blocks are stable
  - Clean cold segment sooner and hot segment later

Crash Recovery and the Log

- Log organization in LFS
  - CR points to a head and tail segment
  - Each segment points to next segment
- LFS can easily recover by simply reading latest valid CR
  - The latest consistent snapshot may be quite old
- To ensuring atomicity of CR update
  - Keep two CRs
  - CR update protocol: timestamp → CR → timestamp
- Roll forward
  - Start from end of the log (pointed by the lastest CR)
  - Read next segments and adopt any valid updates to the file system

Module 7: 文件系统

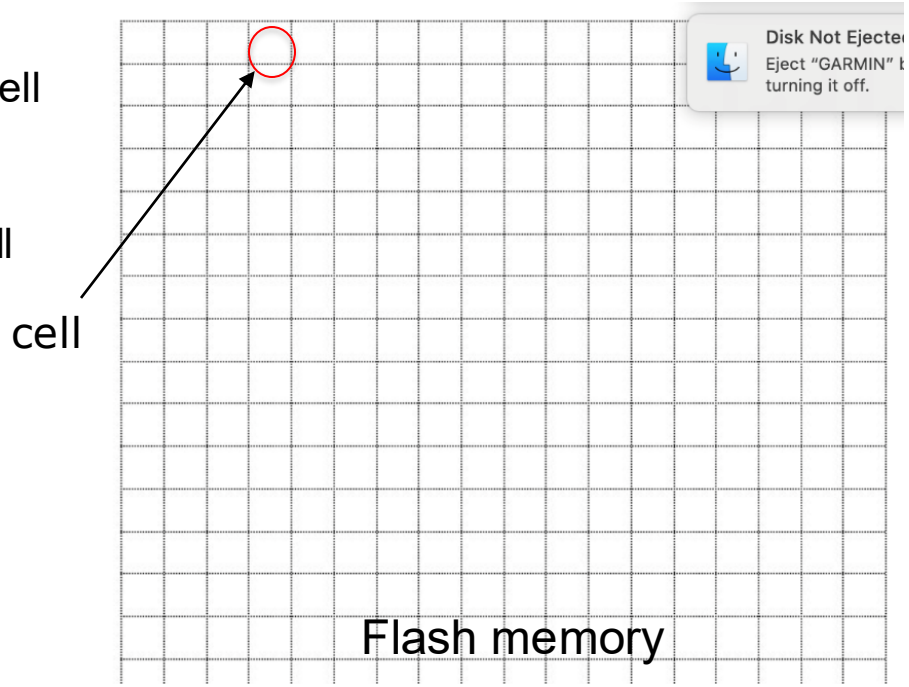
1. 文件（File）和目录（Directory）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. **Flash-based SSD**
   a. **Flash基本原理（进阶）**
   b. **垃圾回收（进阶）**
   c. **地址映射（进阶）**
   d. **合并（进阶）**

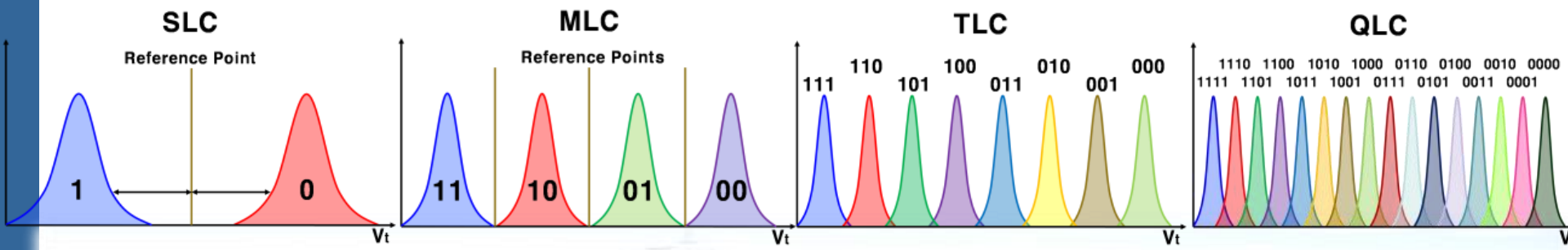
Overview

- Solid-state storage device
  - No mechanical or moving parts like HDD.
    再也没有类似HDD的机械部分或机械运动组件了
  - Built out of transistors (like memory and processors). 完全以晶体管为存储介质
  - Retain information despite power loss unlike typical RAM. 非易失性存储
- NAND Flash based SSD特性:
  - **To write** to a given chunk of it, you first **have to erase** a bigger chunk.
    写操作的基本单位是page
  - The number of erase/write is limited. 擦除和写入的次数是有限制的

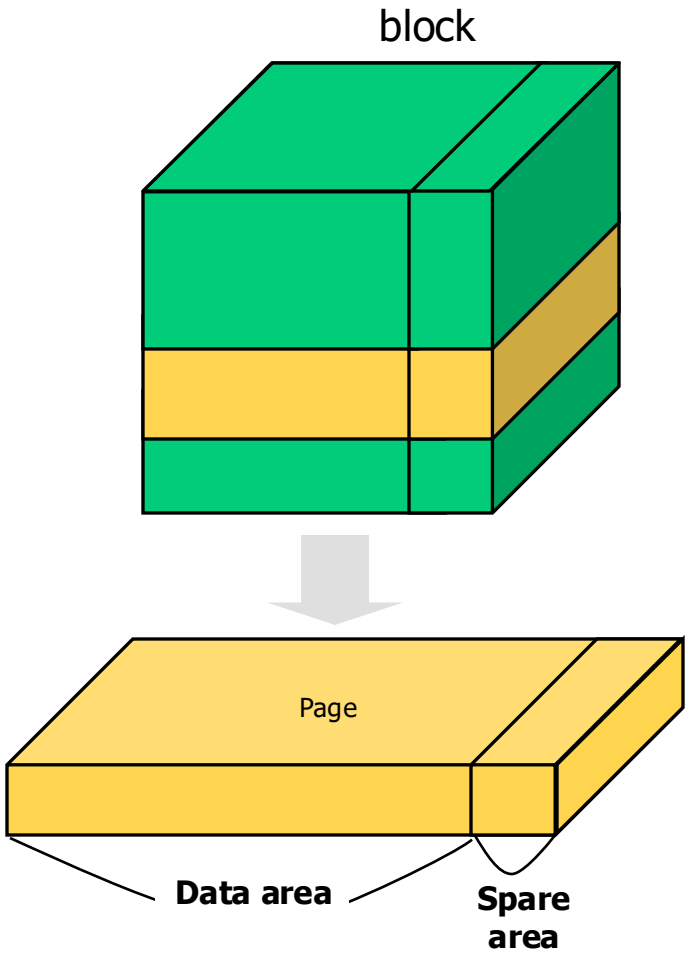
Storing Bit

- Single-level cell (SLC): a single bit per cell
- Multi-level cell (MLC): two bits per cell
- Triple-level cell (TLC): three bits per cell
- QLC, PLC

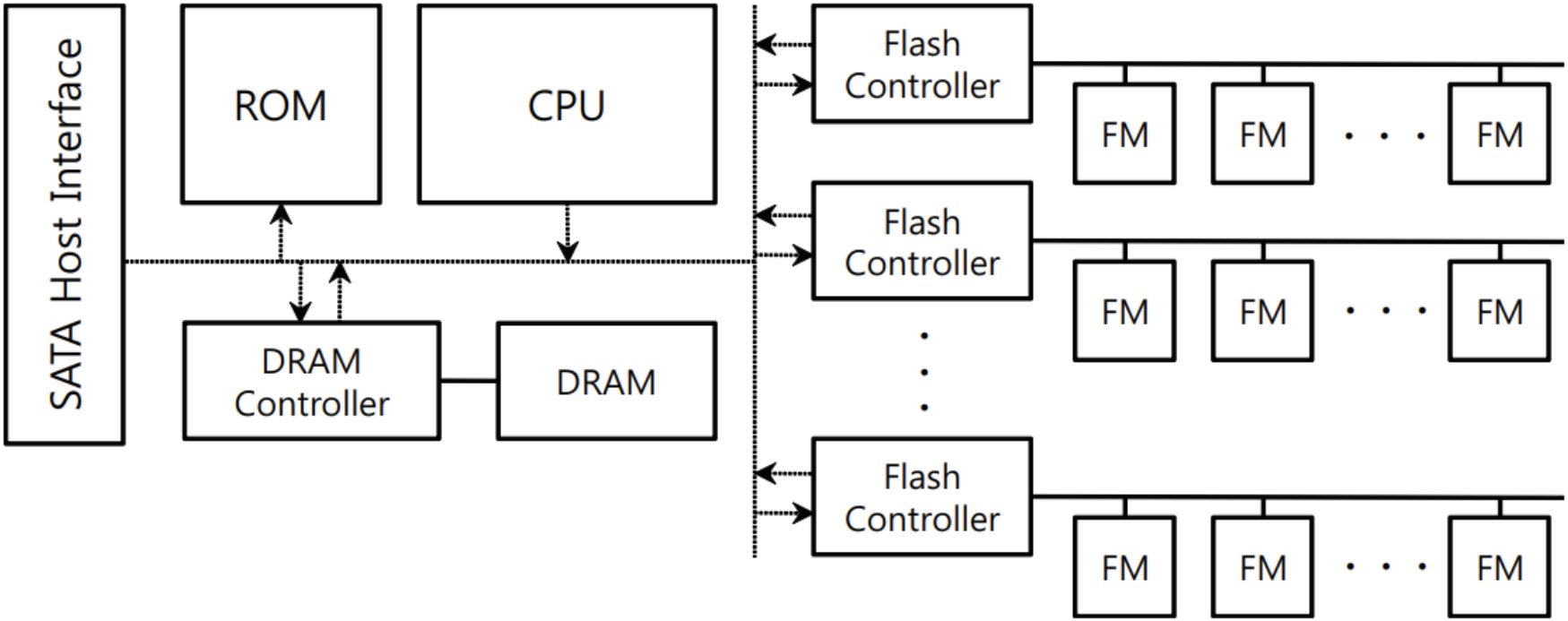




Structure of Flash

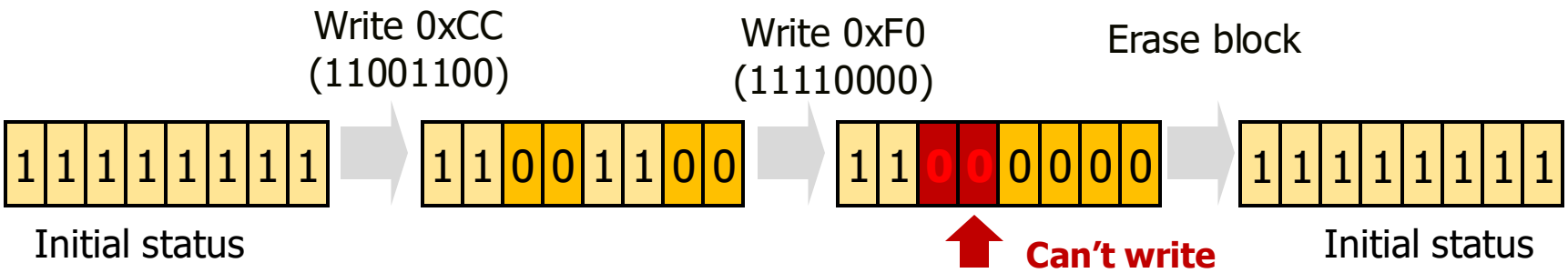


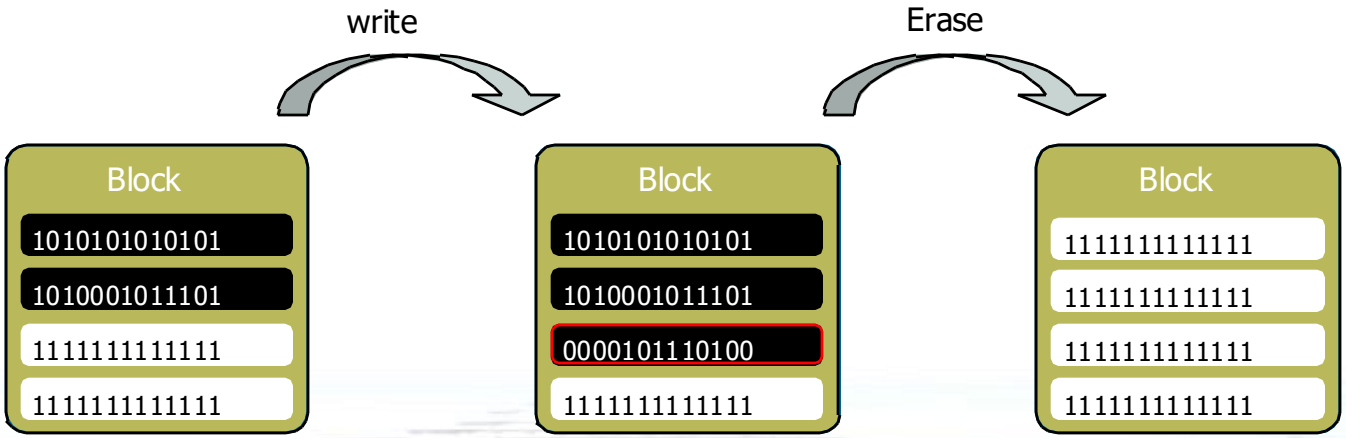
Structure of Flash SSD



Basic Operations

- Read : in page unit
- Write(program): 1 → 0: in page unit.
- Erase: 0 → 1: in block unit



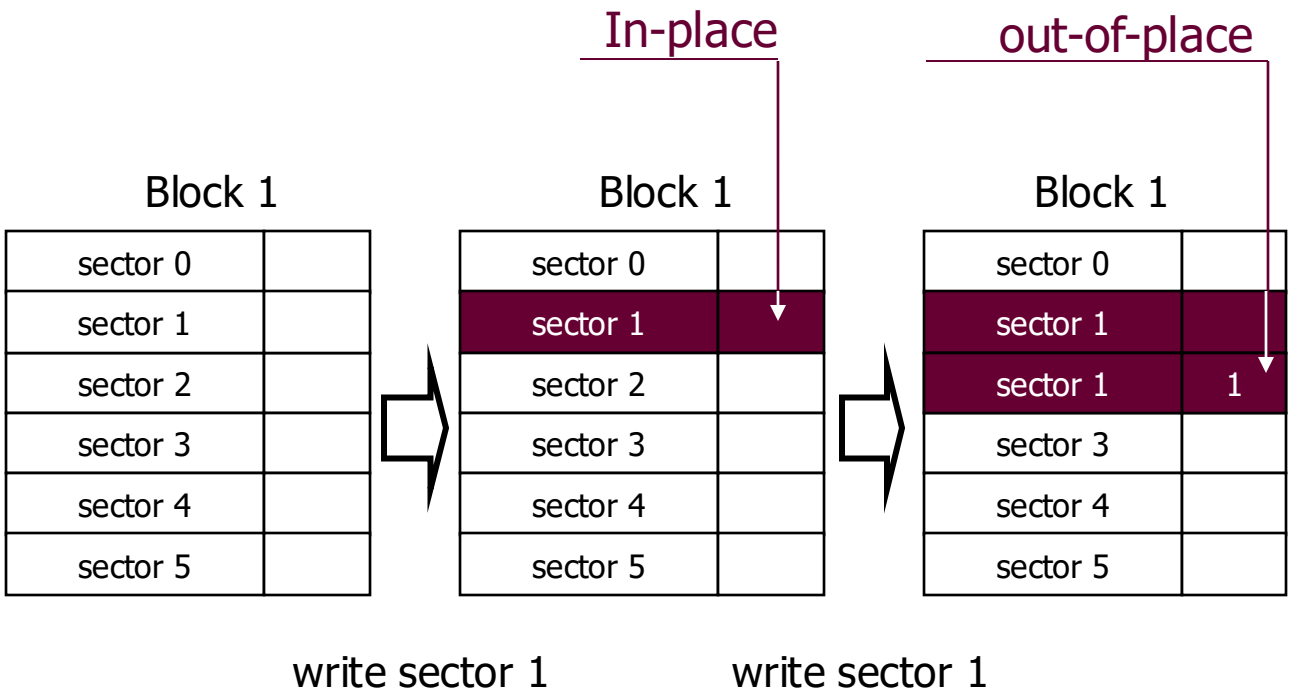


Reliability of Flash

- Wear out
  - Flash cells wear out as we program/erase it.
  - Finally the block becomes unusable.
  - Typical erase/wear out cycle
    - MLC-based block: 10,000 P/E (Program/Erase)
    - SLC-based block: 100,000 P/E
- Disturbance
  - When accessing a particular page within a flash, it is possible that some bits get flipped in neighboring pages.
  - It is called read disturbance or program disturbance.

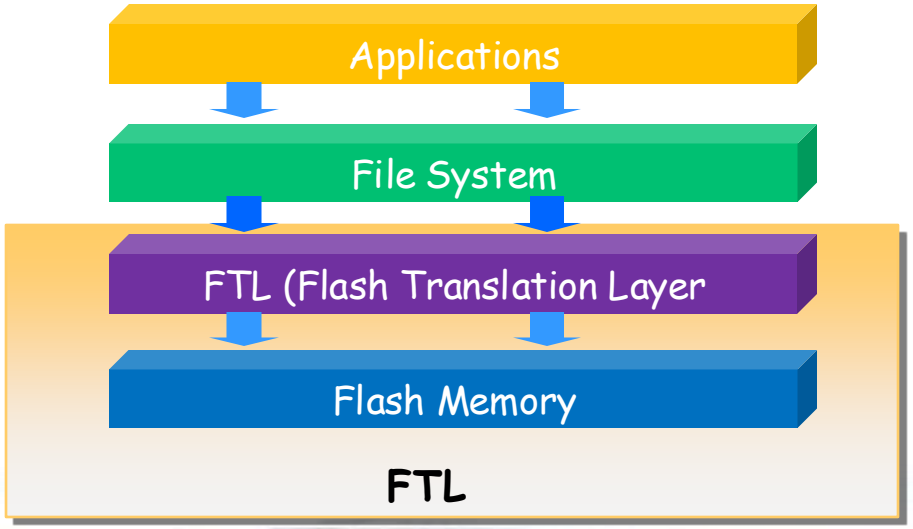
Out-of-place update in Flash memory

- Flash memory should be erased before written.
- Flash SSD uses out-of-place update for write operation.

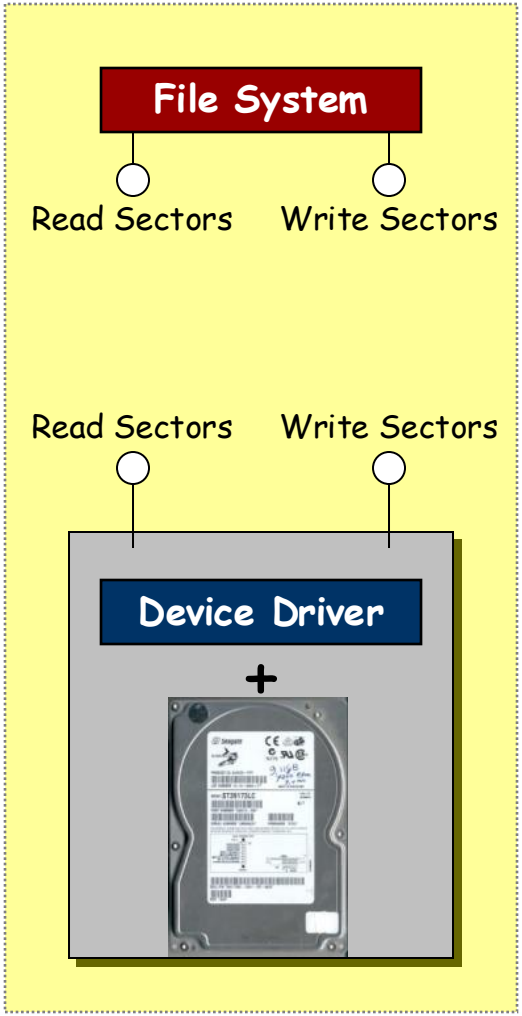


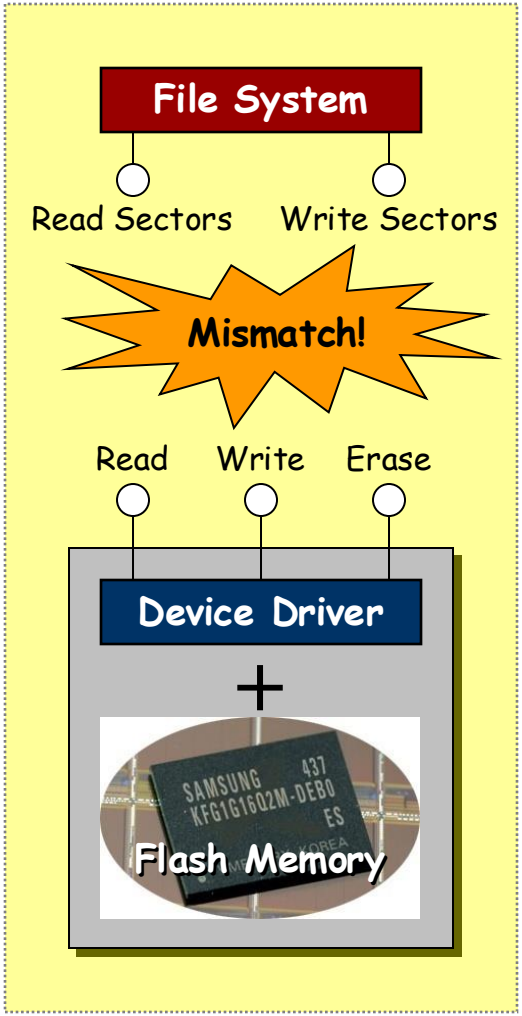
Flash Translation Layer

- A software that make the SSD look HDD，为什么SSD对FS和OS是透明的？是因为有FTL的工作
  - Address translation
    - Program pages within an erased block in order (from low page to high page)
  - Wear leveling
    - FTL should try to spread writes across the blocks of the flash
    - Ensuring that all of the blocks of the device wear out at roughly the same time.
  - Garbage collection



Hard Disk and Flash SSD

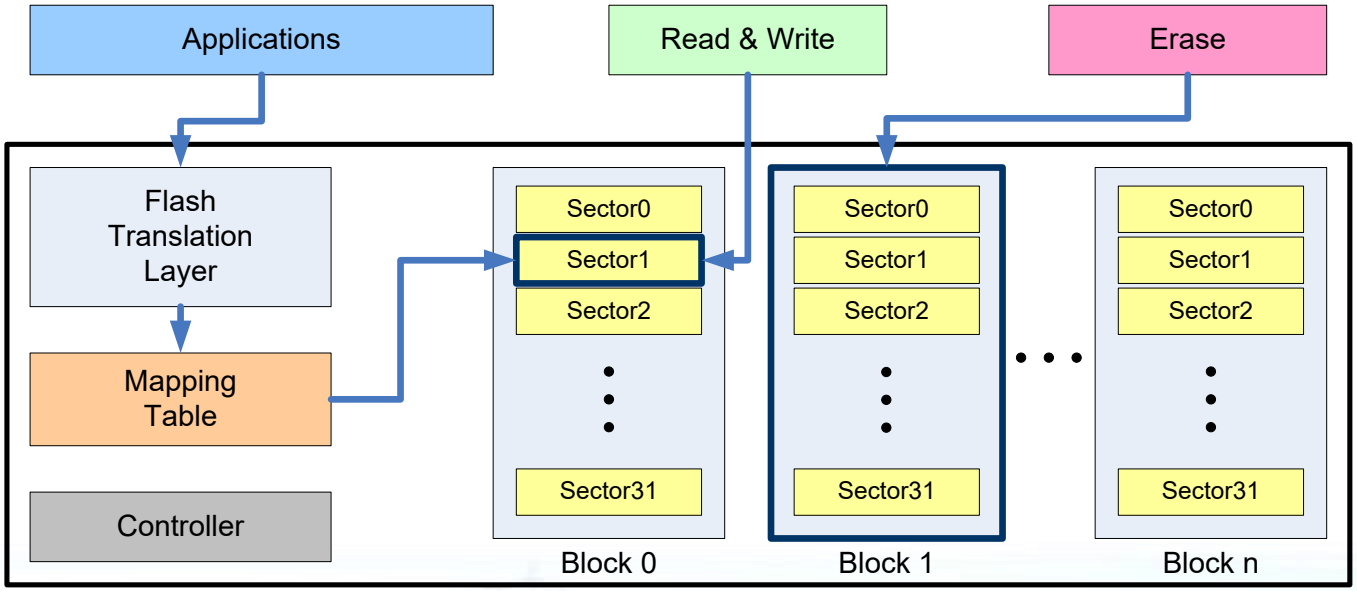




Flash Translation Layer

- Append the write to the next free spot in the currently-being-written-to block.

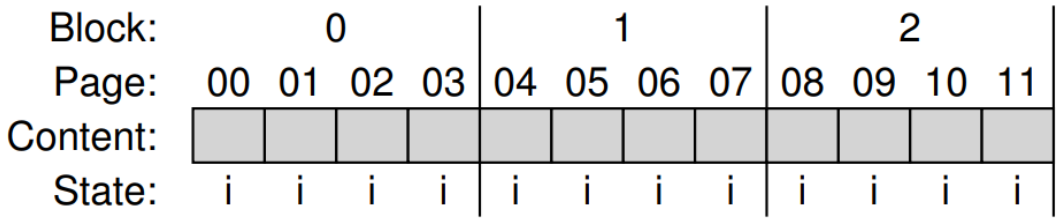
Block:	0				1				2			
Page:	00	01	02	03	04	05	06	07	08	09	10	11
Content:	a1	a2	b1	b2								
State:	V	V	V	V	i	i	i	i	i	i	i	i



Example

- Assume that a page size is 4 Kbyte and a block consists of four pages.
- Write(page number)
  - Write(100) with contents a1
  - Write(101) with contents a2
  - Write(2000) with contents b1
  - Write(2001) with contents b2

- The initial state of SSD, with all pages marked `INVALID(i)`.



189

Example

- Erase block 0.

Block:	0				1				2			
Page:	00	01	02	03	04	05	06	07	08	09	10	11
Content:												
State:	E	E	E	E	i	i	i	i	i	i	i	i

- Program pages in order and update mapping information.

Table: 100 → 0 Memory

Block:	0				1				2			
Page:	00	01	02	03	04	05	06	07	08	09	10	11
Content:	a1											
State:	V	E	E	E	i	i	i	i	i	i	i	i

Flash Chip

- After performing four writes.

Table: 100 → 0 101 → 1 2000 → 2 2001 → 3 Memory

Block:	0				1				2			
Page:	00	01	02	03	04	05	06	07	08	09	10	11
Content:	a1	a2	b1	b2								
State:	V	V	V	V	i	i	i	i	i	i	i	i

Flash Chip

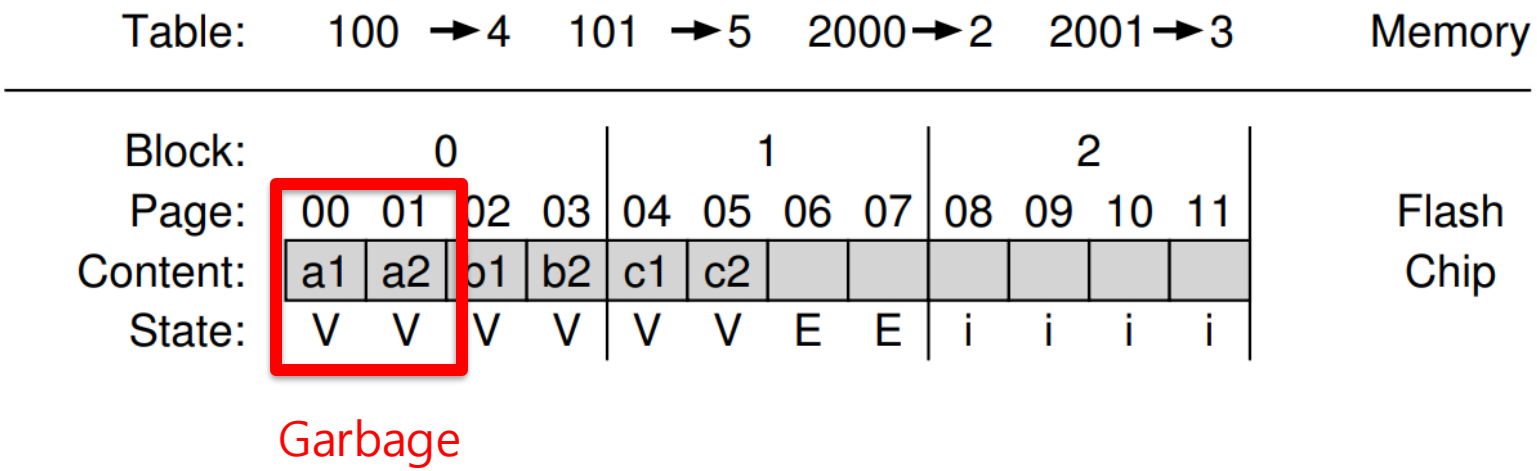
190

Module 7: 文件系统

1. 文件（File）和目录（Directory）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. **Flash-based SSD**
   a. **Flash基本原理（进阶）**
   b. **垃圾回收（进阶）**
   c. **地址映射（进阶）**
   d. **合并（进阶）**

Garbage collection

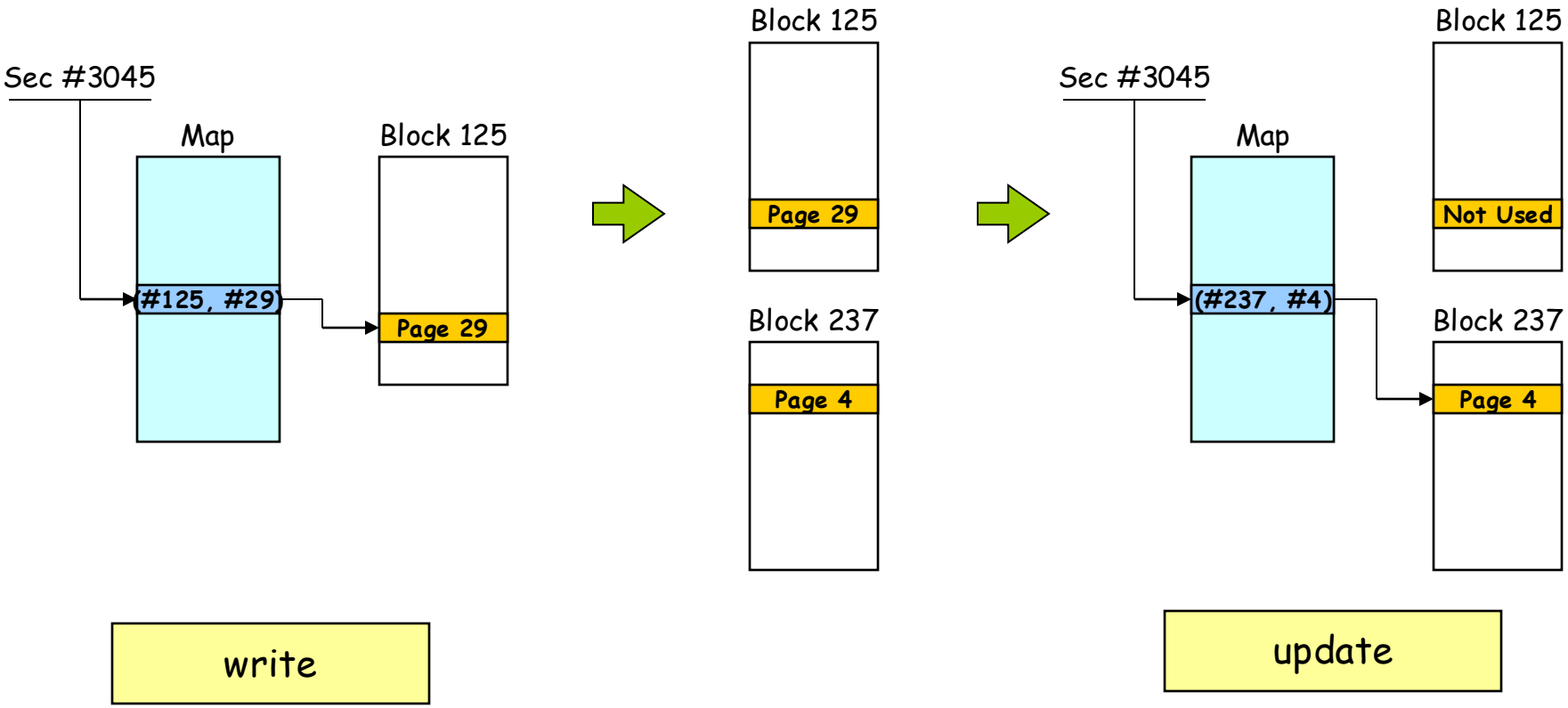
- Update the existing page → old version of data becomes obsolete.
- Update blocks 100 and 101 with contents c1 and c2.



- The pages 0 and 1 in block 0 is old version data and these are called garbage.
- These garbage pages must be reclaimed for new writes to take place.

192

Address Translation

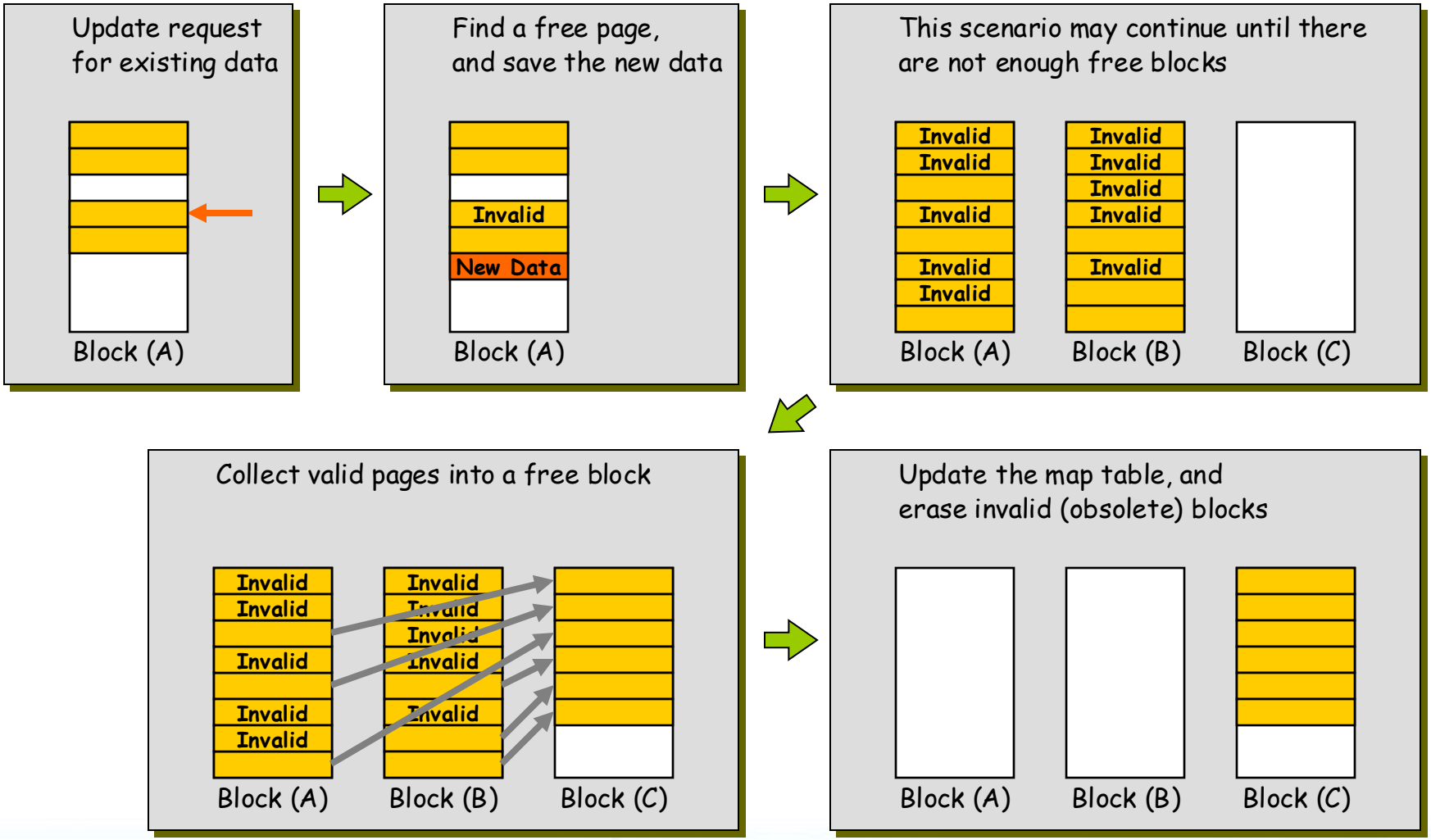


Garbage collection

- the process of finding garbage blocks and reclaiming them.
- Basic process
  - Find a block that contains one or more garbage pages.
  - Read in the live(non-garbage) pages from that block.
  - Write out those live pages to the next writable pages.
  - Reclaim the entire block for use in writing.

194

Garbage collection



Garbage collection

- Garbage collection is expensive
  - Require reading and rewriting of live data.
  - Ideal garbage collection is reclamation of a block that consists of only dead pages.

- Cost of Garbage Collection
  - Amount of data blocks that are migrated.
  - Overprovision the device by adding extra flash capacity→ Cleaning can be delayed.
  - Run the GC in the background.

196

Module 7: 文件系统

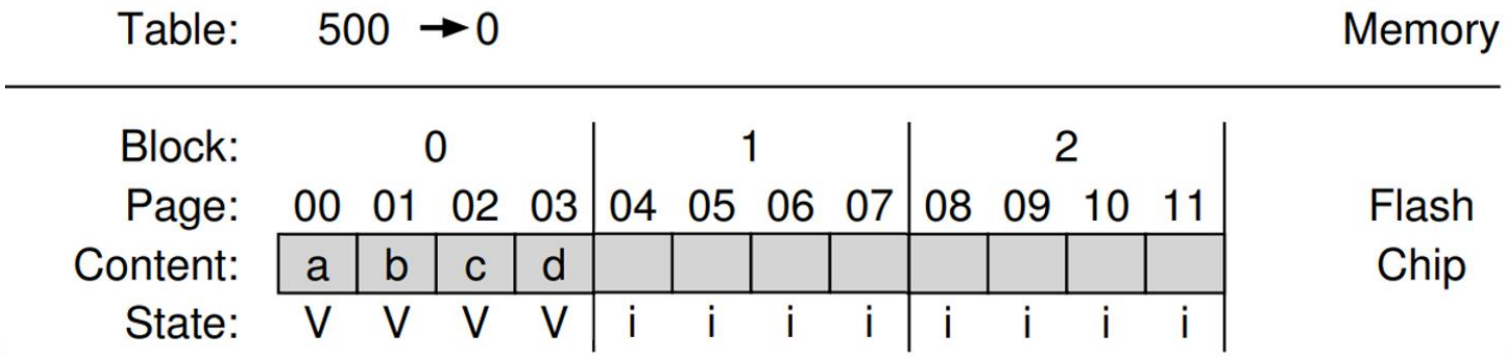
1. 文件（File）和目录（Directory）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. **Flash-based SSD**
   a. **Flash基本原理（进阶）**
   b. **垃圾回收（进阶）**
   c. **地址映射（进阶）**
   d. **合并（进阶）**

Mapping Table Size

- Size of page-level mapping table is too large
  - With a 1TB SSD with a 4byte entry per 4KB page, 1GB of DRAM is needed for mapping.
- Some approaches to reduce the costs of mapping
  - Block-based mapping
  - Hybrid mapping
  - Page mapping plus caching

Block Mapping

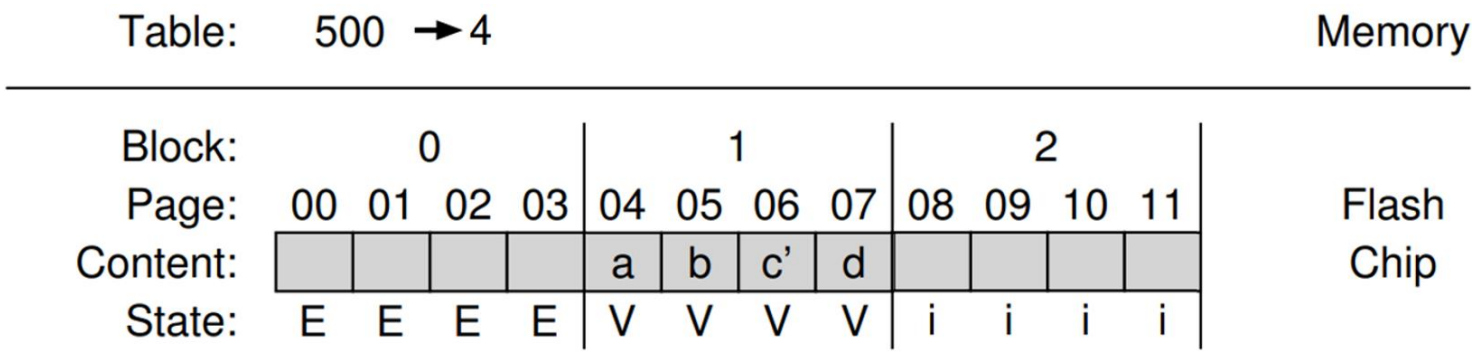
- Mapping at block granularity.
  - To reduce the size of a mapping table.
- Small write problem: When a small write occurs, the FTL must read a large amount of live data from the old block and copy them into a new one.
- Logical blocks 2000, 2001, 2002, and 2003 are at the same Flash block (500), and have different offsets (0, 1, 2, and 3).
- The FTL records that chunk 500 maps to block 0.



199

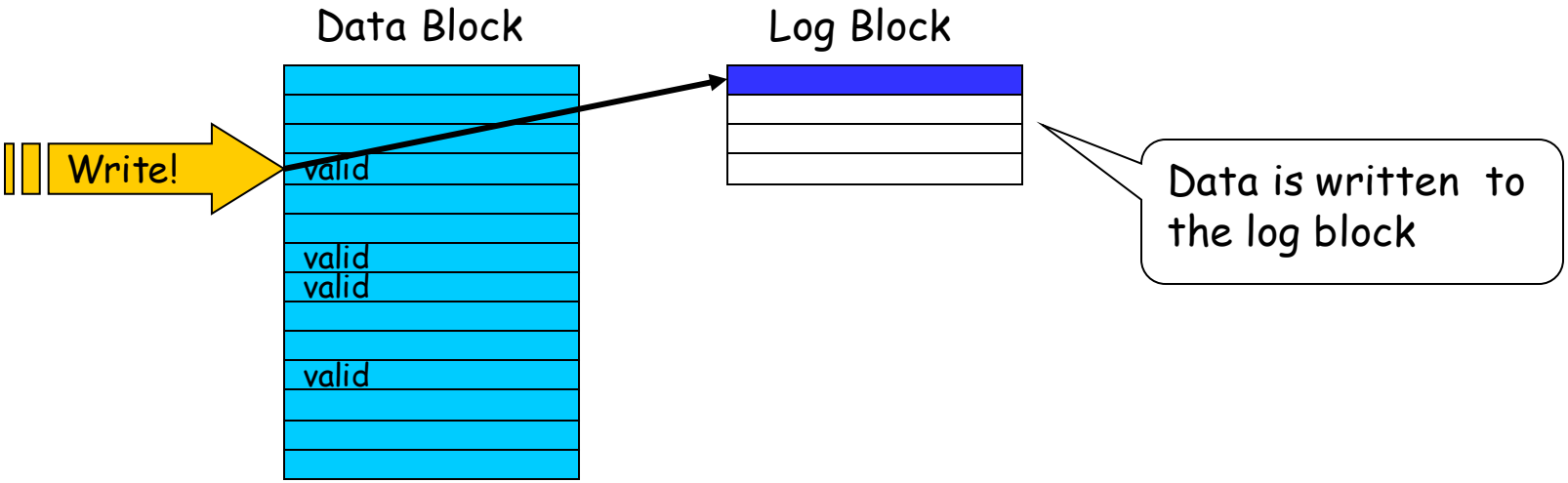
Example

- If the logical block 2002 is updated with contents c',
  - FTL must read in 2000, 2001, and 2003.
  - Write out all four logical blocks in a new location.
  - Update the mapping table.



Hybrid mapping

- In hybrid mapping, FTL maintains
  - Log blocks: page mapped
  - Data blocks: block-mapped
- When looking for a particular logical block, the FTL will consult the page mapping table and block mapping table in order.



Example

- In the following situation,

Log Table:
Data Table: 250 → 8 Memory

	Block:	0				1				2				
	Page:	00	01	02	03	04	05	06	07	08	09	10	11	Flash
	Content:									a	b	c	d	Chip
	State:	i	i	i	i	i	i	i	i	V	V	V	V	

- Update these pages (with data a', b', c', and d'). → write them to the log block. FTL updates the page mapping information.

Log Table: 1000 → 0 1001 → 1 1002 → 2 1003 → 3
Data Table: 250 → 8 Memory

	Block:	0				1				2				
	Page:	00	01	02	03	04	05	06	07	08	09	10	11	Flash
	Content:	a'	b'	c'	d'					a	b	c	d	Chip
	State:	V	V	V	V	i	i	i	i	V	V	V	V	

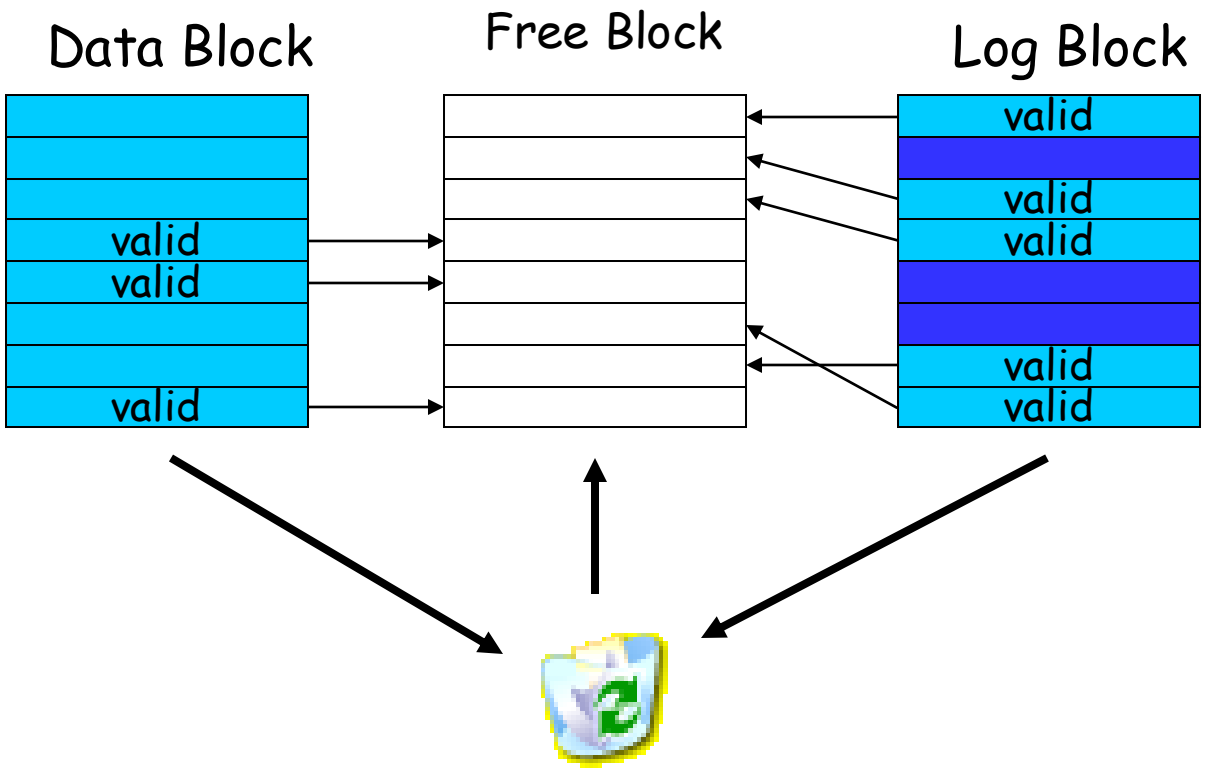
202

Module 7: 文件系统

1. 文件（File）和目录（Directory）
2. VSFS（Very Simple File System）简单文件系统
3. Locality and The FFS (Fast File System)
4. Crash Consistency: FSCK and Journaling
5. LFS日志结构FS (Log-structured File Systems)
6. **Flash-based SSD**
   a. **Flash基本原理（进阶）**
   b. **垃圾回收（进阶）**
   c. **地址映射（进阶）**
   d. **合并（进阶）**

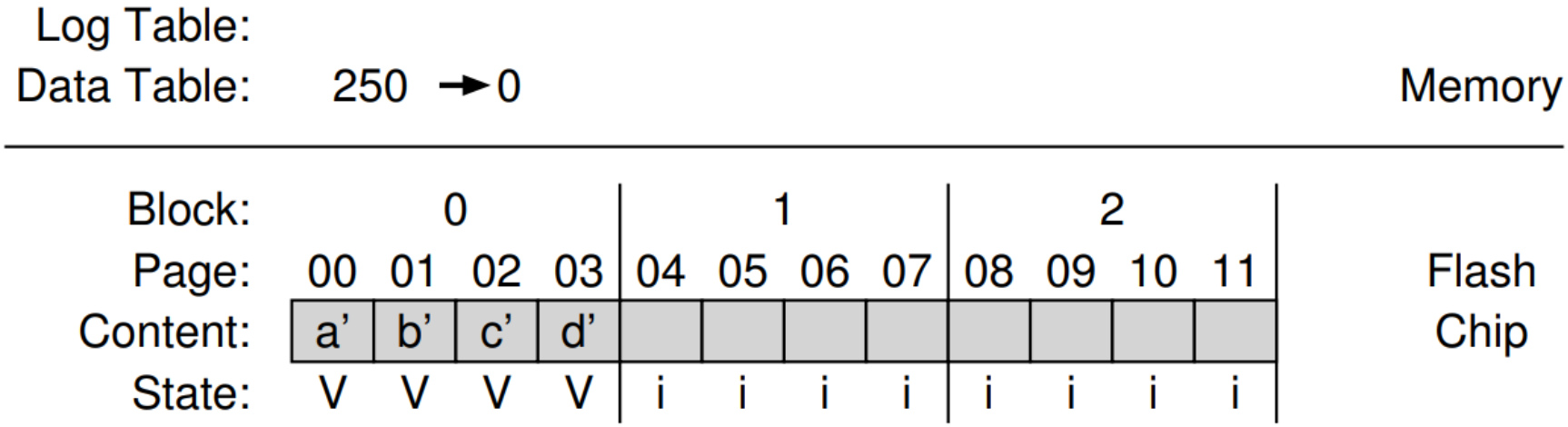
Merge

- When log block is full, perform merge.



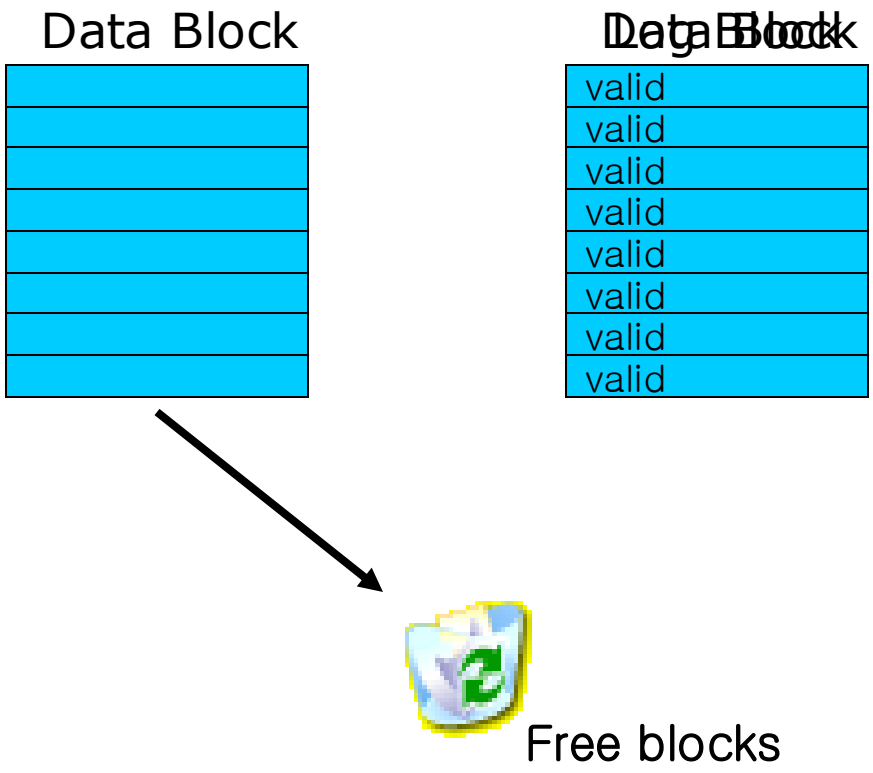
Switch Merge

- When log block is full, perform merge.
  - Switch merge



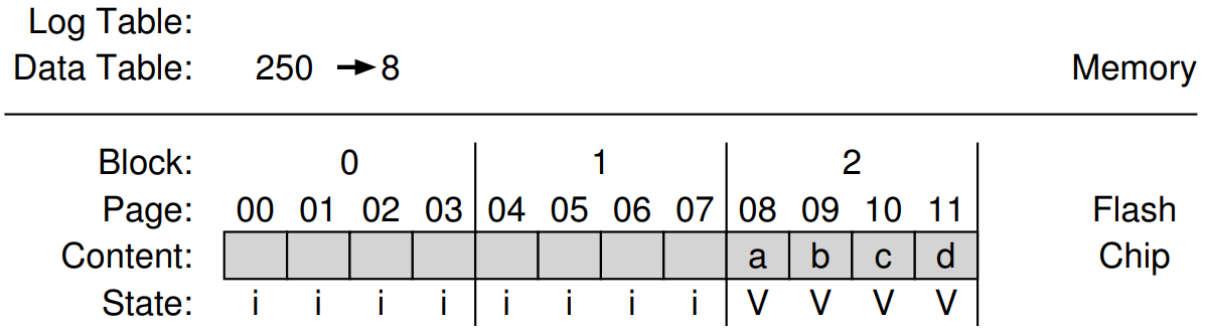
205

Switch merge

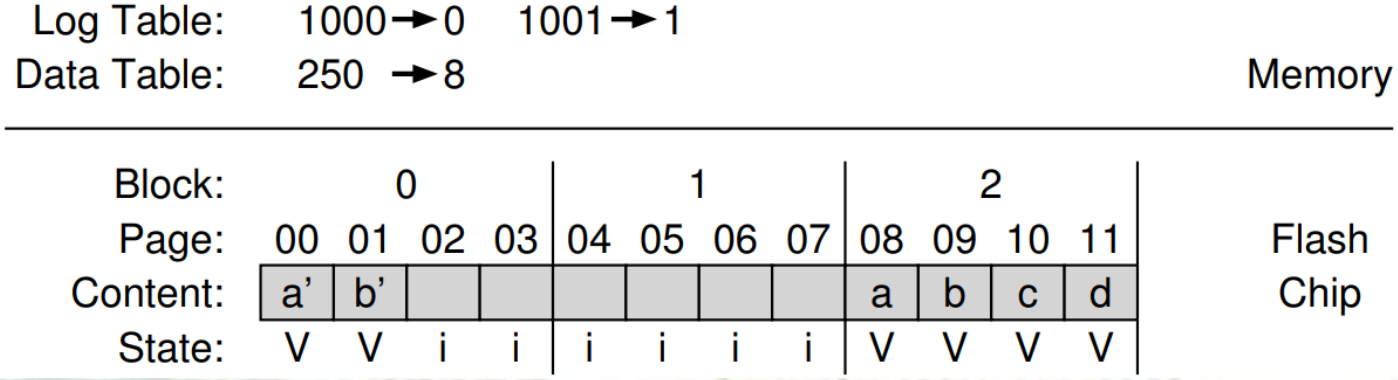


Partial Merge

- Client overwrites logical block 1000 and 1001 partially. What happened?

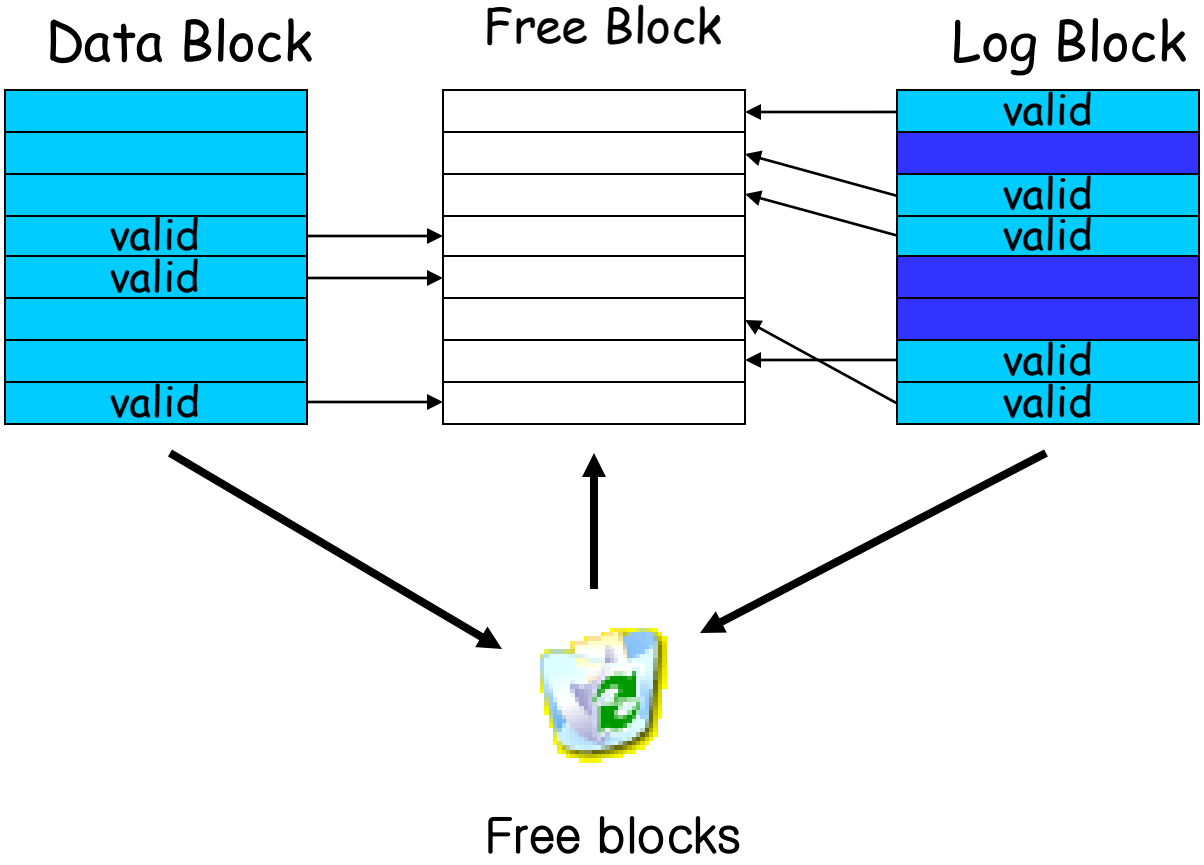


- The FTL writes logical block 1000 and 1001(contents a′ and b′) to the available pages in new block.
- And then, the FTL appends other live data.

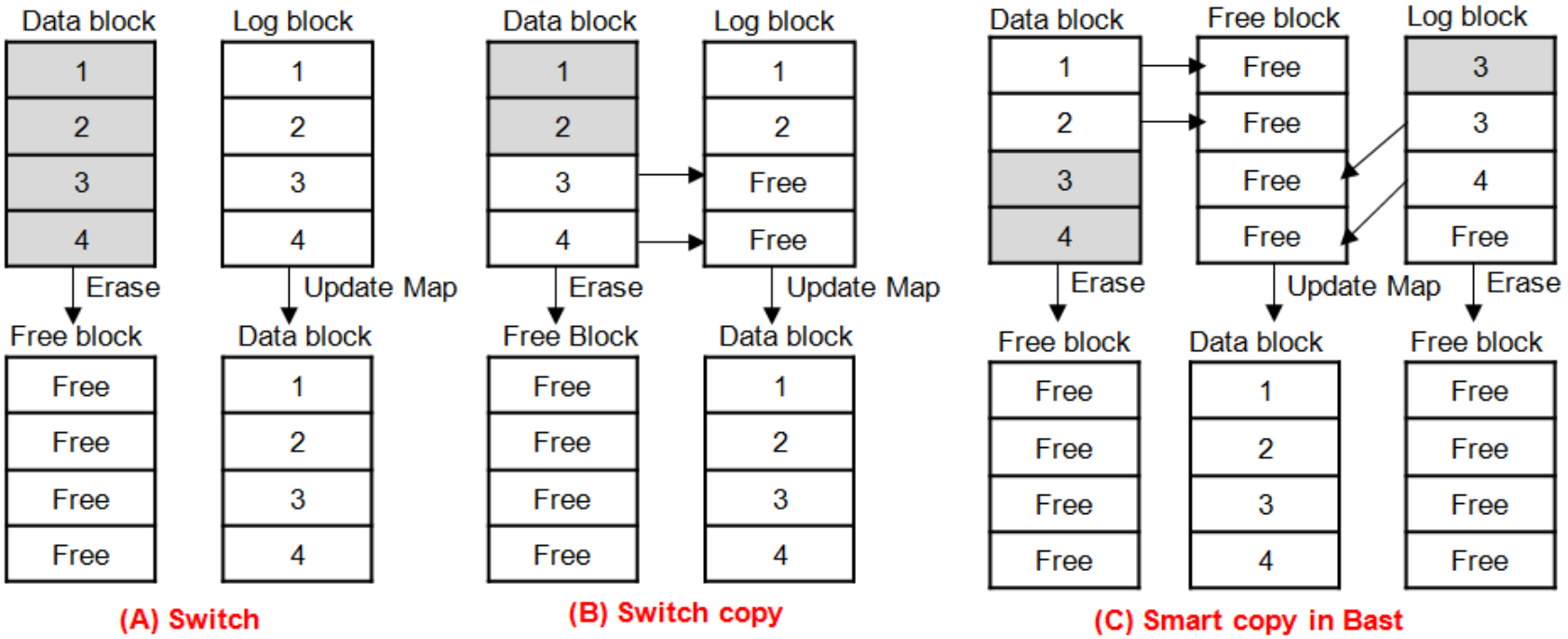


207

Partial merge



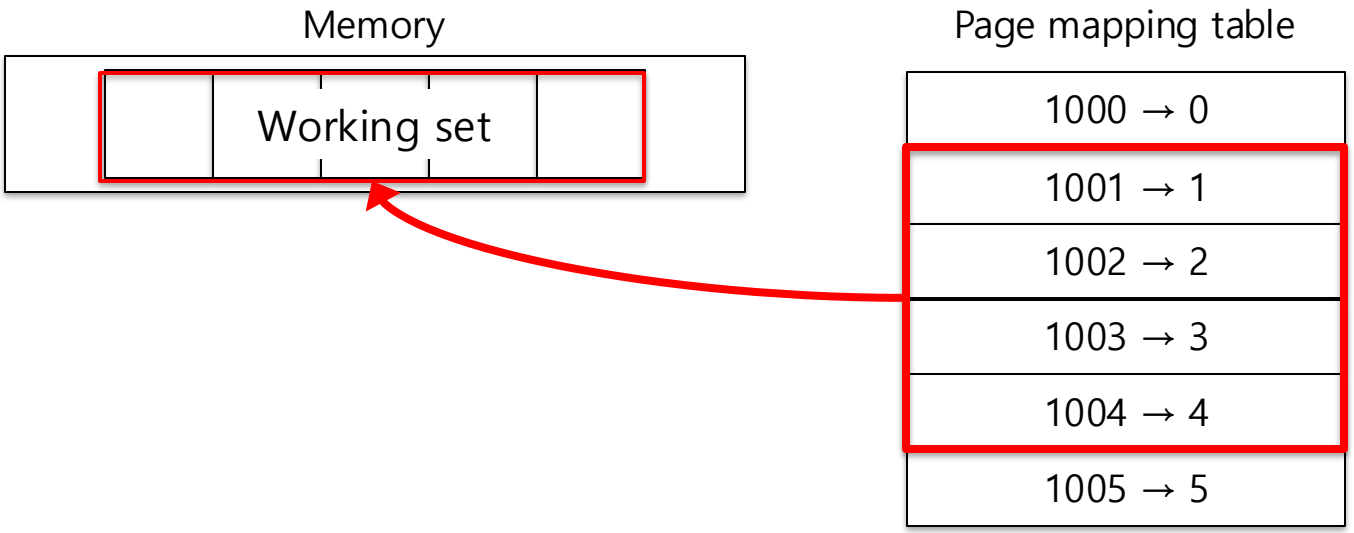
Switch and Merge



209

Page mapping plus caching

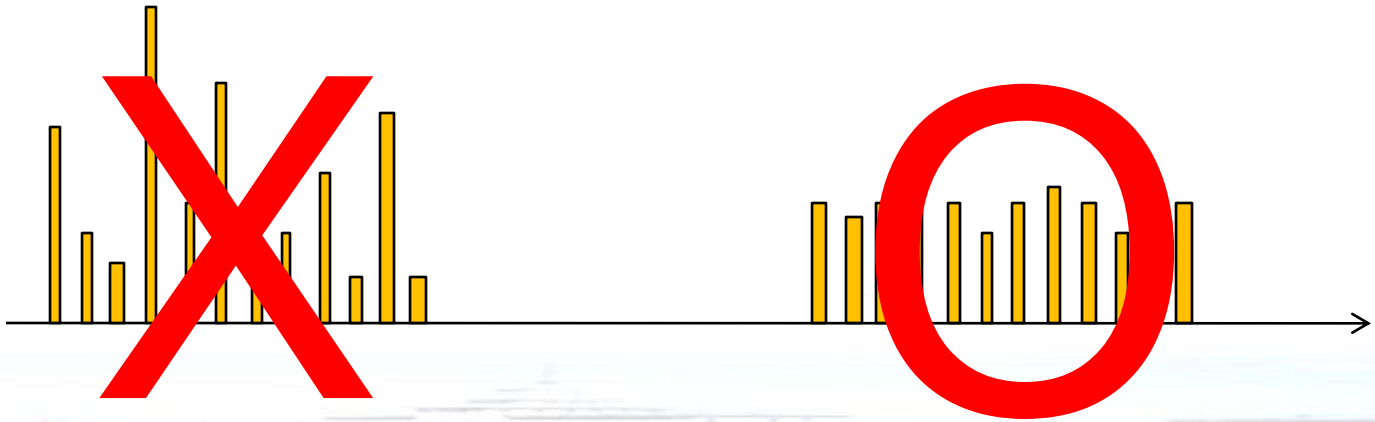
- Caching only the active part of the page-mapped FTL in memory.
  - If a given workload only accesses a small set of pages, the translations of those pages will be stored in the in-memory FTL.
- Performance will be excellent without high memory cost.
- Cache miss overhead exists.



Wear Leveling

- Erase/Write cycle is limited in Flash memory.
- If EW cycle is skewed, that shortens the lifespan of the entire Flash storage.

  All blocks should wear out at roughly the same time.
- A block may consist of cold data.
  - The FTL must periodically read all the live data out of such blocks and re-write it elsewhere.
  - Wear leveling increases the write amplification of the SSD and decreases performance.
- Sample Policy: Each Flash Block has EW cycle counter.
  - Maintain |Max(EW cycle) – Min(EW cycle)| < e
  - Maintain |Max(EW cycle) – Min(EW cycle)|/Max(EW cycle) < e



Summary

- Flash-based SSDs are becoming a common presence in …
  - laptops, desktops, and servers inside the datacenters.

- Flash Translation Layer
  - Address Translation
  - Wear Leveling
  - Garbage Collection

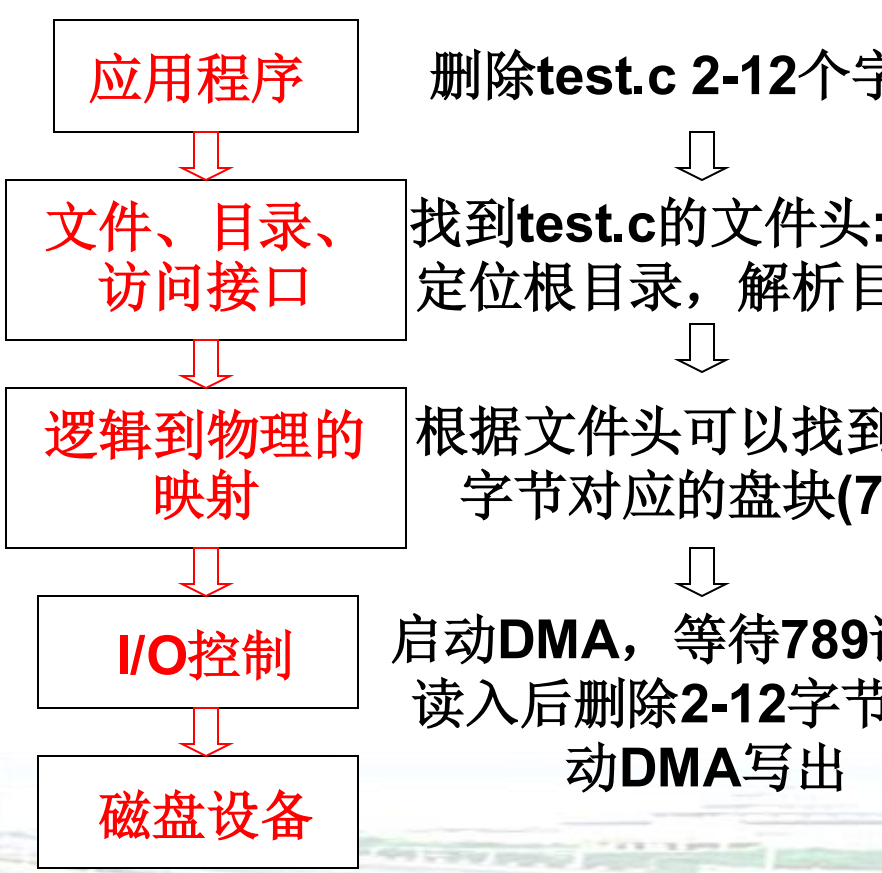
附录

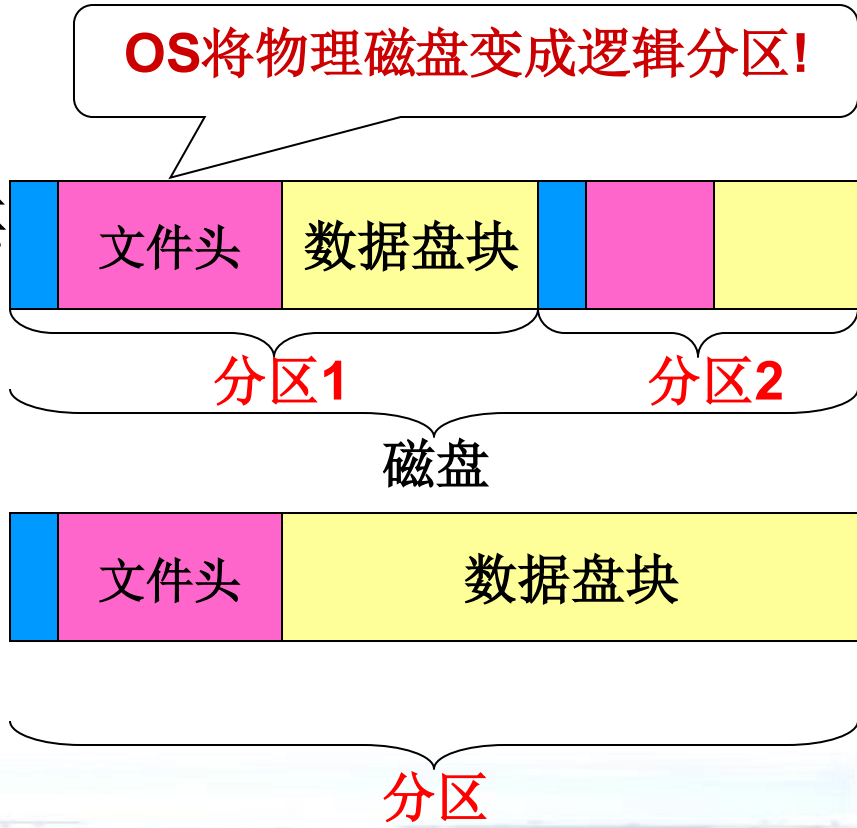
描述文件系统的实质(定义)

就像将CPU资源和地址空间封装成进程一样!

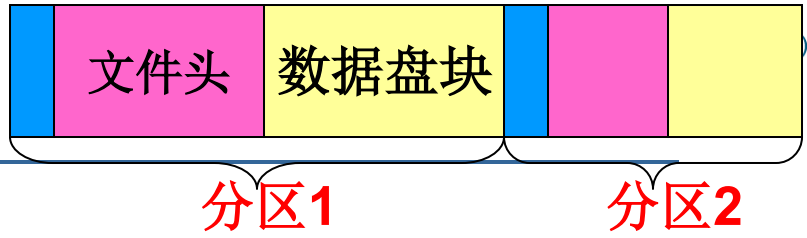
⑩ 文件系统: 将盘块“变”成文件集，方便用户访问

- 文件系统是“抽象盘块”的一层软件!





分区的详细结构

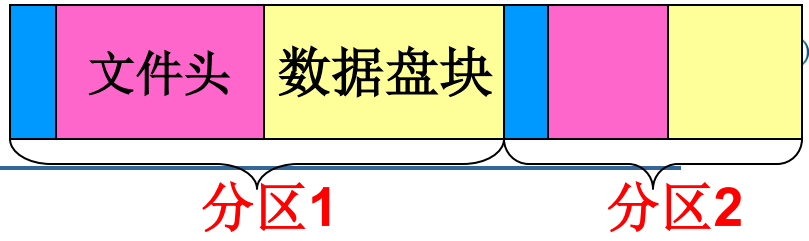


❿ 典型分区结构：UNIX分区的基本结构



- 引导块存放引导OS的信息，如果该分区中没有OS，则该块为空
- 超级块记录分区基本信息: 分区块数；块大小；空闲块数量、指针；空闲文件头数量、指针等
- 索引节点数组存放所有文件的文件头，UNIX root目录的索引节点号为2
- 数据块，文件内容

分区空闲盘块的管理

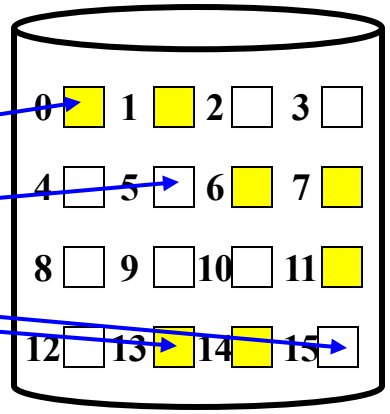


- ⑩ 有的盘块被文件使用，其它盘块如何管理？
  - 组织起来等待文件的使用！ 怎么组织？
  - 方法1：空闲位图(位向量)…

    0011110011101001

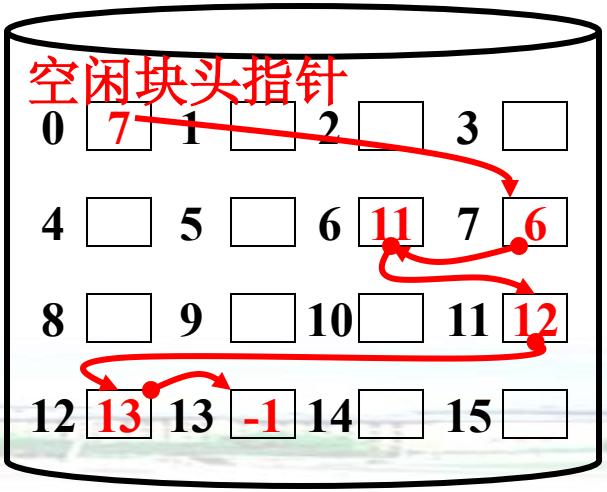
    表示磁盘块2,3,4,5,8,9,10,12已被占用

    可快速分配连续盘块组，但位向量很大(1G/1k=?)



  - 方法2：空闲链表

    分配一个(或少量的)空闲盘块是可以高效工作，但分配多个则慢！



良好运转的文件系统应该高效

⑩ 相比CPU和内存，磁盘读写非常慢!

```
int main(int argc, char* argv[])
{
    int  i , to, *fp, sum = 0;
    to = atoi(argv[1]);
    for(i=1; i<=to; i++)
    {
        sum = sum + i;
        fprintf(fp,"%d", sum);
    }
}
```

fprintf用一条其他计算语句代替

```
C:\>sum 10000000
0.015000 seconds
```

$0.015 * 10^{-7}$ S

用fprintf

```
C:\>sum 1000
0.859000 seconds
```

$0.859 * 10^{-3}$ S

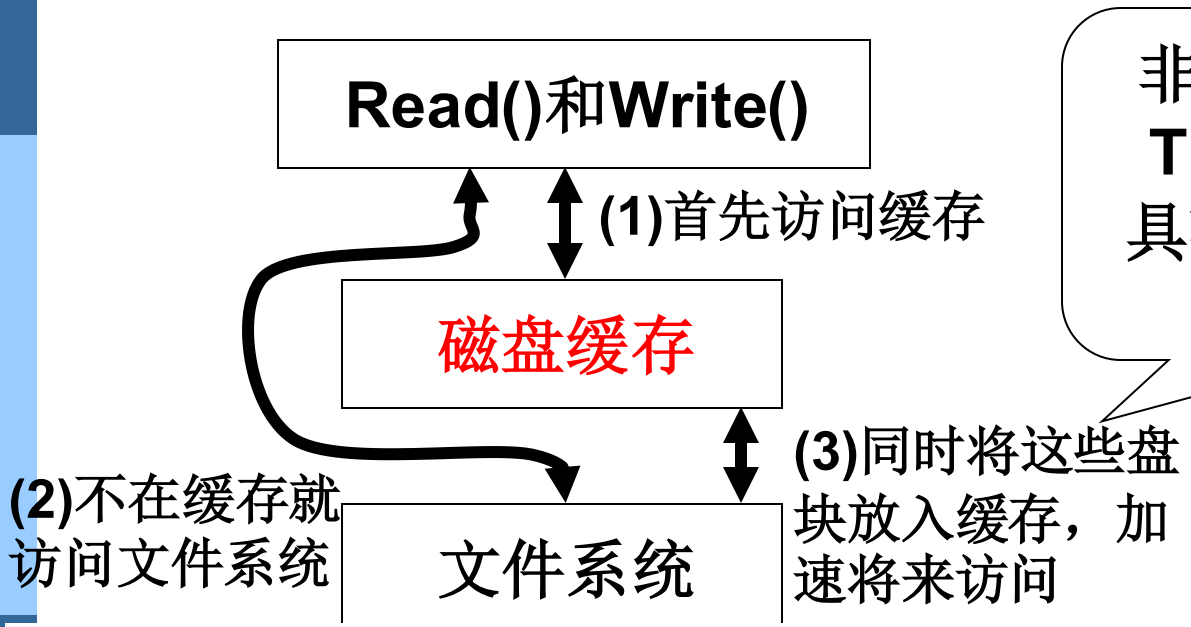
$5.7 \times 10^5 : 1$

解决速度差异问题的基本手段是?
引入缓存!(因为局部性)



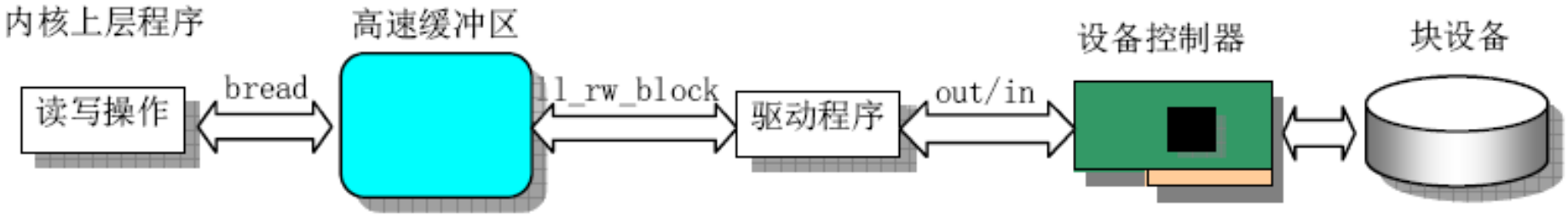
磁盘缓存

⑩ 在内存中缓存磁盘上的部分(很少部分)盘块



非常熟悉的过程?
TLB、虚拟内存!
具有局部性性质才有意义!

Linux中的sync命令
用于强制将修改的文件的内容立刻写入磁盘



Delayed Write or Write Through

其他的提高文件访问效率的技术…

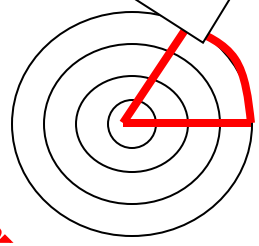
- 某些目录文件的FCB可以常驻内存
  - /的FCB常驻内存，因为许多目录解析都从此处开始
  - 当前目录(pwd)，如gcc 1.c。其FCB可驻内存?
- ls -l的使用非常频繁

一段时间大多数文件访问集中在一个目录中(局部性)

  - 怎么才能快速执行?

    同一目录中的文件inode在一个柱面(组)!

每个柱面组都有inode，空闲盘块…



  - 需要重新设计inode、目录文件分配算法…
- 显然，还有许多提高文件系统效率的技术……
  - 需要自己去整理，这是操作系统课程显著特点之一

良好运转的文件系统应该提供保护

- 文件用来存放用户的信息，用户应该能控制对文件的访问: 如只允许读
  - 文件关联权限，哪些权限? 放在哪里?
  - 权限: 读/写/执行(r/w/x)，当然是放在文件头中!
  - 一实例: drwxrwxrwx root staff test/

不同用户具有不同权限 | owner id | group id

  - 如何强制执行(enforce)?
  - 访问文件是由进程发起的，进程是由用户启动的:

(1)PCB中有uid和gid; (2)fork时设置; (3)这些信息在登录时收集(从tty); (4)文件访问时校验

又将许多东西联系在了一起…OS魅力所在!

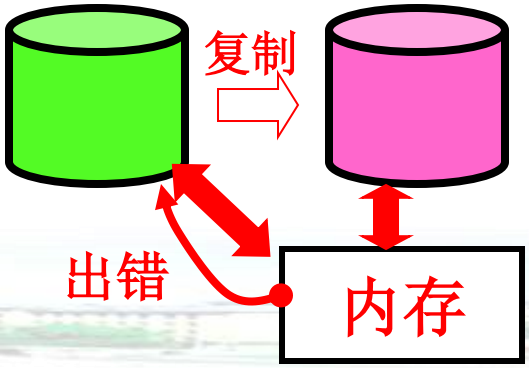
回忆:进程控制块PCB


```
struct task_struct {
/*-----------------------------------------------*/
    long state;    // 进程运行状态（-1 不可运行，0 可运行，>0 以停止）
    long counter; // 任务运行时间片，递减到 0 是说明时间片用完
    long priority;     // 任务运行优先数，刚开始是 counter＝priority
    long signal;  // 任务的信号位图，信号值=偏移+1
    struct sigaction sigaction[32];  //信号执行属性结构，对应信号将要执行的操作和标志信息
    long blocked; // 信号屏蔽码
/*--------------- various fields----------------------- */
    int exit_code;     // 任务退出码，当任务结束时其父进程会读取
    unsigned long start_code, end_code, end_data, brk, start_stack;
        // start_code      代码段起始的线性地址
        // end_code    代码段长度
        // end_data    代码段长度+数据段长度
        // brk         代码段长度+数据段长度+bss 段长度
        // start_stack     堆栈段起始线性地址
    long pid, father, pgrp, session, leader;
        // pid 进程号，father 父进程号，pgrp 父进程组号，session 会话号，leader 会话首领
    unsigned short uid, euid, suid;
        // uid 用户标 id，euid 有效用户 id，suid 保存的用户 id
    unsigned short gid, egid, sgid;
        // gid 组 id，egid 有效组 id，sgid 保存组 id
    long alarm;    // 报警定时值
```

良好运转的文件系统似乎也应该容错

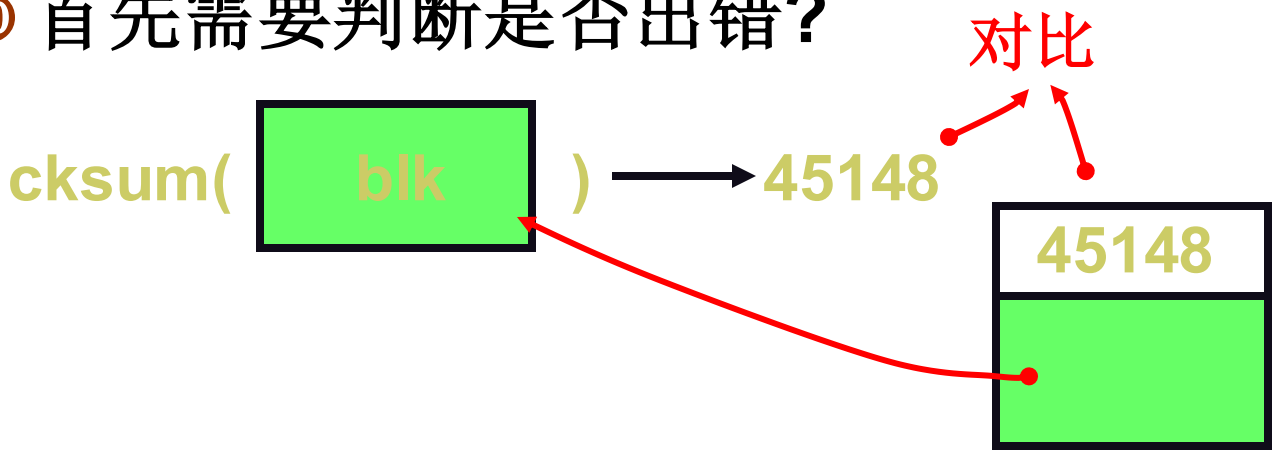


- 有过这样的经历?
  - 用户当然不希望: 早晨起来发现写了数月论文打不开了(如因下一块的link断了)…
  - 错误是难免的: 误操作、突然断电、无处不在的电磁干扰… 怎么办? 错误避免还是错误恢复…
  - **RAID**(Redundant Arrays of Inexpensive Disks)
  - RAID基本思想就是冗余(R): 如在镜像磁盘上备份数据, 发现错误时拷贝镜像磁盘(恢复)

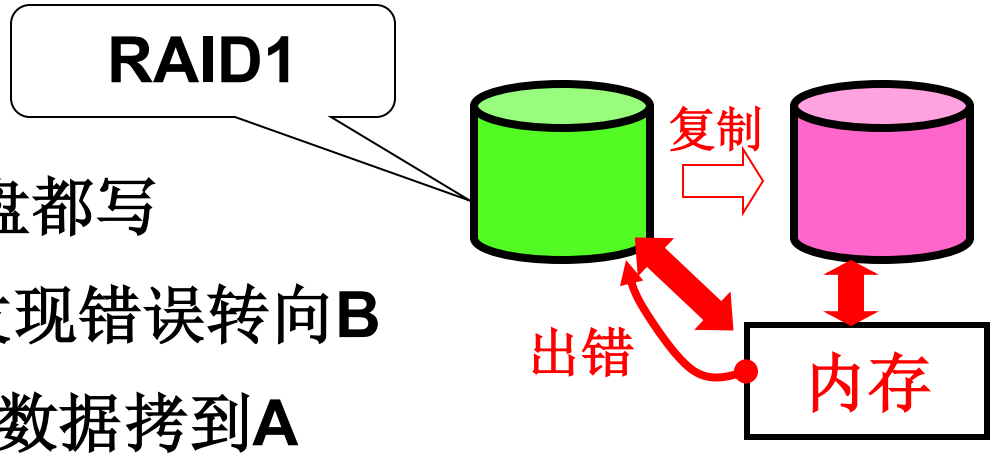


RAID的简单实现

- 首先需要判断是否出错?



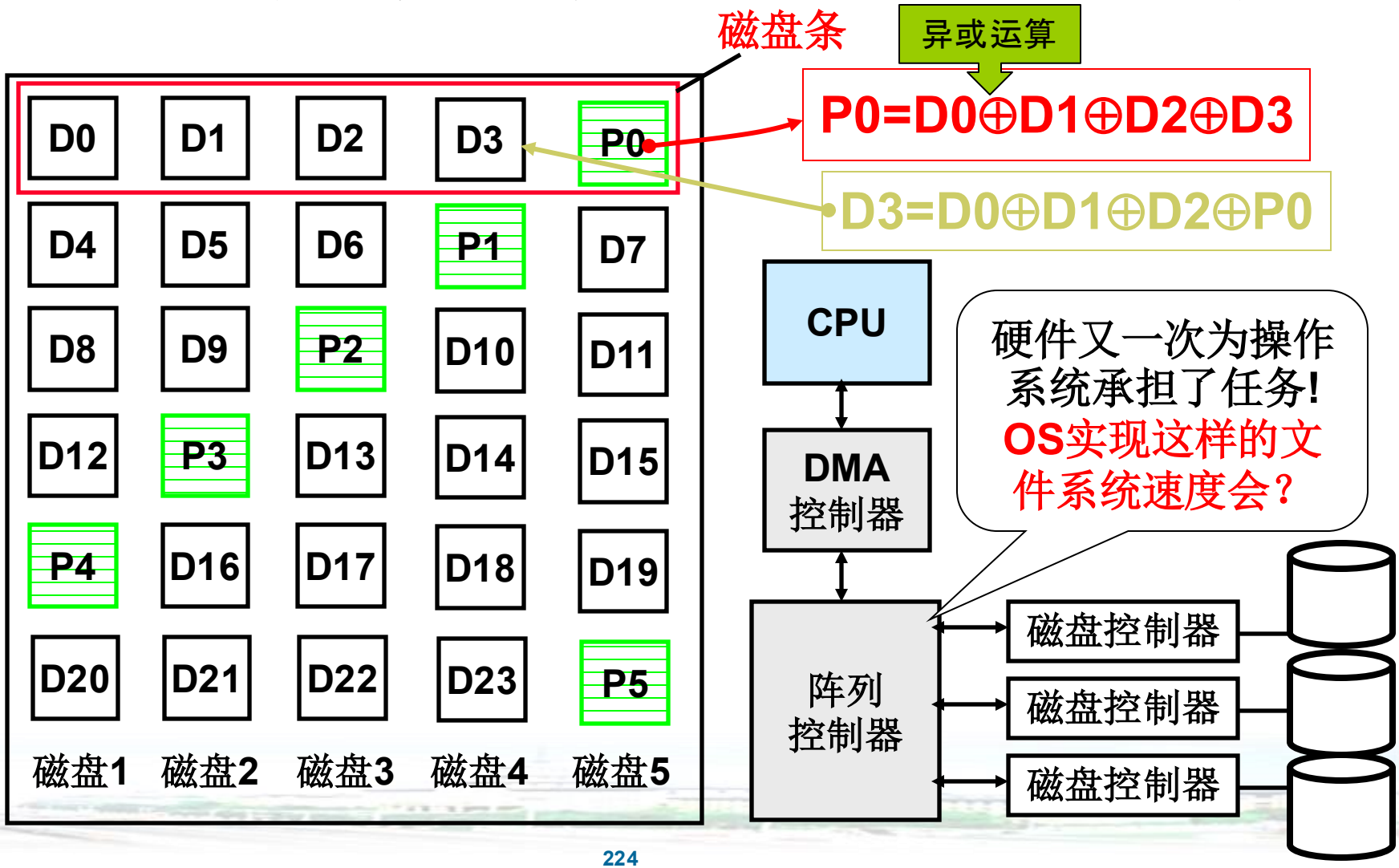
- 此时的磁盘读写



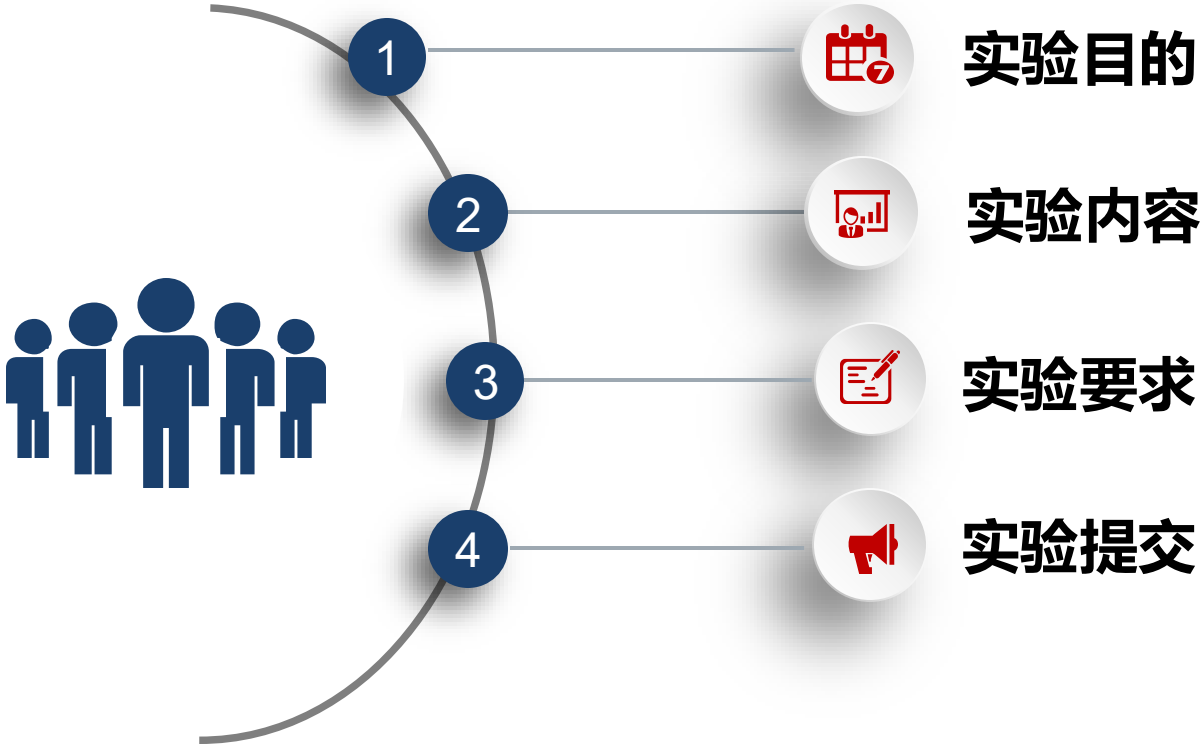
  - (1)每次写两个磁盘都写
    (2)从磁盘A读，发现错误转向B
    (3)A有错时将B的数据拷到A
  - 磁盘利用率50%

RAID5+

❿ 校验数据分布在多个盘上，磁盘互相恢复数据



实验五：文件系统



实验目的

- 了解磁盘和磁盘管理的基本概念，理解为什么需要文件系统
- 掌握文件系统的核心设计思想及其实现原理，熟悉文件系统内部数据结构的组织方式和基本处理流程
- 掌握EXT2文件系统的布局，并完整设计和实现一个青春版的EXT2文件系统

实验内容 | 2个实验任务

①文件系统demo

实现一个文件系统demo:

- 阅读 fs/demo 中源码
- 填充超级块的数据
- 实现文件属性的获取
- 实现 ls 命令查看文件

②青春版EXT2文件系统

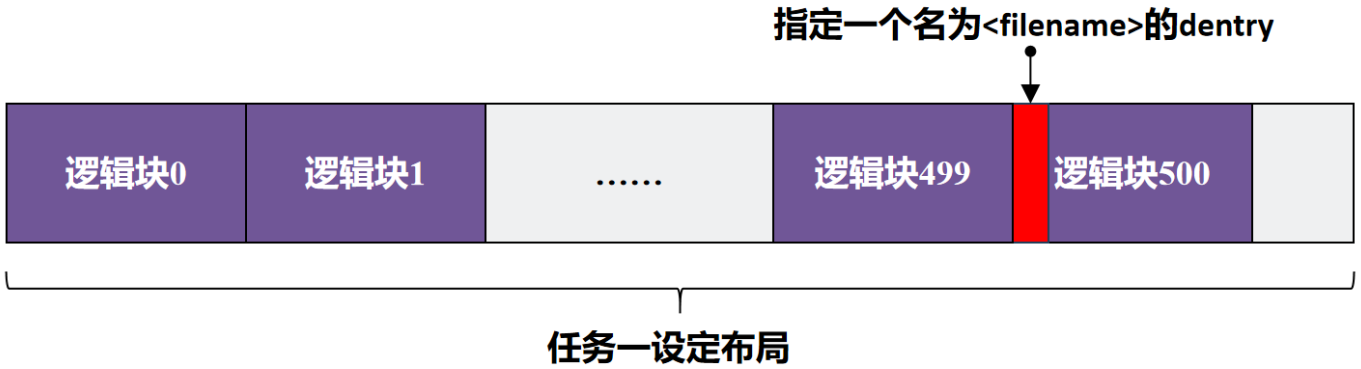
实现一个简化的EXT2文件系统:

- 阅读 fs/sfs 源码
- 实现文件系统存储布局
- 实现 Inode Map和分配
- 实现 Data Map和分配
- 实现文件系统挂载/卸载

实验任务一：文件系统 Demo

- 理解磁盘是按逻辑块进行读写
  - 磁盘IO块：512B
  - 磁盘逻辑块：1024B （2个IO块）
  - 磁盘读取需要提供 offset（按IO块对齐），每次读写都是整个IP块一起读写
  - 需要实现对特定 offset 的数据进行读取





实验任务一：文件系统 Demo

```c
/* 挂载文件系统 */
static void* demo_mount(struct fuse_conn_info * conn_info){
    // 打开驱动
    char device_path[128] = {0};
    sprintf(device_path, "%s/" DEVICE_NAME, getpwuid(getuid())->pw_dir);
    super.driver_fd = ddriver_open(device_path);

    printf("super.driver_fd: %d\n", super.driver_fd);


    /* 填充super信息 */
    super.sz_io = 512; /* TODO 磁盘IO大小，单位B */
    super.sz_disk = 4194304; /* TODO 磁盘容量大小，单位B 4MB磁盘*/
    super.sz_blks = 1024;  /* TODO 逻辑块大小，单位B */

    return 0;
}
```

文件系统
超级块 保存的内容

sz_io：磁盘IO读取的基本大小，一般是 512B
sz_blks：文件系统逻辑块大小，这里一个逻辑块是2个磁盘IO块大小

逻辑块使用多个IP块，主要目的是减少磁盘寻道时间，增加顺序读取数量，从而提高磁盘读取效率(增加了吞吐量)

移动磁盘到指定位置

从磁盘读取数据

填充目录文件名

```
static int demo_readdir(const char* path, void* buf, fuse_fill_dir_t filler, off_t offs
{
    // 此处任务一同学无需关注demo_readdir的传入参数，也不要使用到上述参数

    char filename[128]; // 待填充的

    /* 根据超级块的信息，从第500逻辑块读取一个dentry，ls将只固定显示这个文件名 */
    int logic_block_index = 500;

    /* TODO: 计算磁盘偏移off，并根据磁盘偏移off调用ddriver_seek移动磁盘头到磁盘偏移off处 */
    off_t disk_offset = logic_block_index * super.sz_blks;
    ddriver_seek(super.driver_fd, disk_offset, SEEK_SET);

    /* TODO: 调用ddriver_read读出一个磁盘块到内存，512B */
    char buffer[512];
    ddriver_read(super.driver_fd, buffer, super.sz_io);

    /* TODO: 使用memcpy拷贝上述512B的前sizeof(demo_dentry)字节构建一个demo_dentry结构 */
    struct demo_dentry dentry;
    memcpy(&dentry, buffer, sizeof(struct demo_dentry));

    /* TODO: 填充filename */
    memcpy(filename, dentry.fname, 128);

    // 此处大家先不关注filler，已经帮同学写好，同学填充好filename即可
    return filler(buf, filename, NULL, 0);
}
```

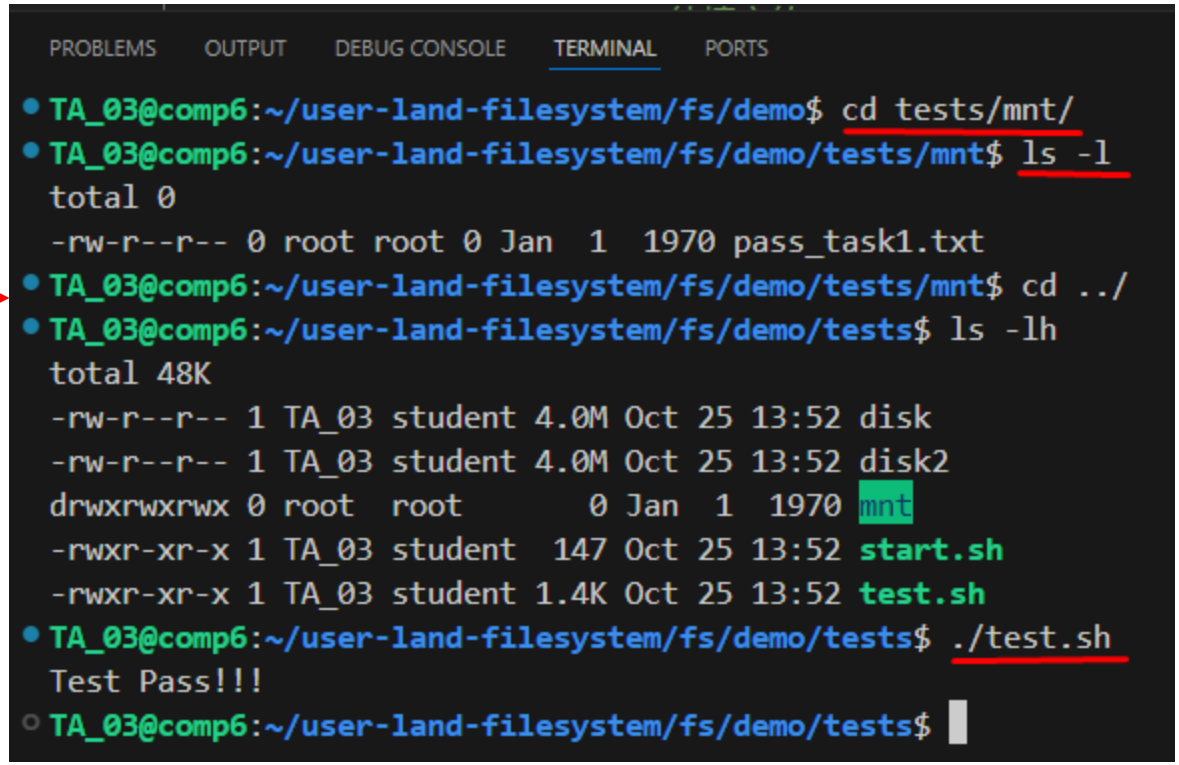
实验任务一：文件系统 Demo

挂载文件系统到指定目录

```
TA_03@comp6:~/user-land-filesystem/fs/demo/tests$ ../demo -f -d ./mnt/
FUSE library version: 2.9.9
nullpath_ok: 0
nopath: 0
utime_omit_ok: 0
unique: 2, opcode: INIT (26), nodeid: 0, insize: 56, pid: 0
INIT: 7.34
flags=0x33fffffb
max_readahead=0x00020000
super.driver_fd: 4
   INIT: 7.19
   flags=0x00000011
   max_readahead=0x00020000
   max_write=0x00020000
   max_background=0
   congestion_threshold=0
   unique: 2, success, outsize: 40
unique: 4, opcode: GETATTR (3), nodeid: 1, insize: 56, pid: 1142602
getattr /
   unique: 4, success, outsize: 120
```

实验任务一：文件系统 Demo

运行测试脚本
测试文件系统
通过



实验任务二：青春版Ext2文件系统

- 理解Ext2文件系统存储结构
  - 超级块：1个逻辑块 1024B
  - InodeMap： 1个逻辑块 1024B，对应8196个Inode节点
  - DataMap： 1个逻辑块 1024B，对应8196数据块
  - Inode节点：每个节点占一个逻辑块（为了简化设计）
  - Data 数据：每个数据占一个逻辑块



实验任务二：青春版Ext2文件系统

超级块的数据结构

包含了整个存储结构

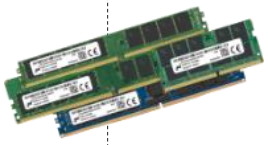
```
* SECTION: FS Specific Structure - Disk structure
* | Super(1) | Inode Map(1) | DATA Map(1) | INODE(1) | DATA(*) |
*************************************************************
struct sfs_super_d
{
    uint32_t            magic_num;
    uint32_t            sz_usage;

    uint32_t            max_ino;
    // Inode Map(1)
    uint32_t            map_inode_blks;
    uint32_t            map_inode_offset;
    // DATA Map(1)
    uint32_t            map_data_blks;
    uint32_t            map_data_offset;
    // INODE
    uint32_t            inode_offset;
    // DATA
    uint32_t            data_offset;
};
```



实验任务二：青春版Ext2文件系统

文件系统的挂载(mount)
读取超级块、Inode Map、Data Map，实现挂载



挂载本质：初始化管理区缓存

④ 初始化根目录的结构，作为后续路径解析入口
(内存)

③ 生成数据块/索引节点位图
(内存)

② 生成超级块
(内存)

完成挂载 ← 创建空根目inode及dentry ← 清零索引节点、数据块位图 ← 重新估算磁盘布局信息 ← 否 ← 是否已初始化? → 是 → 直接读取填充磁盘布局信息 → 读取索引节点、数据块位图 → 读取根目录inode，生成层级 → 完成挂载

参考 sfs_util.c 559-570

参考 sfs_util.c 547-552

参考 sfs_util.c 529-546

超级块

参考 sfs_util.c 547-552

参考 sfs_util.c 554-557

参考 sfs_util.c 564-570

① 读入超级块判断

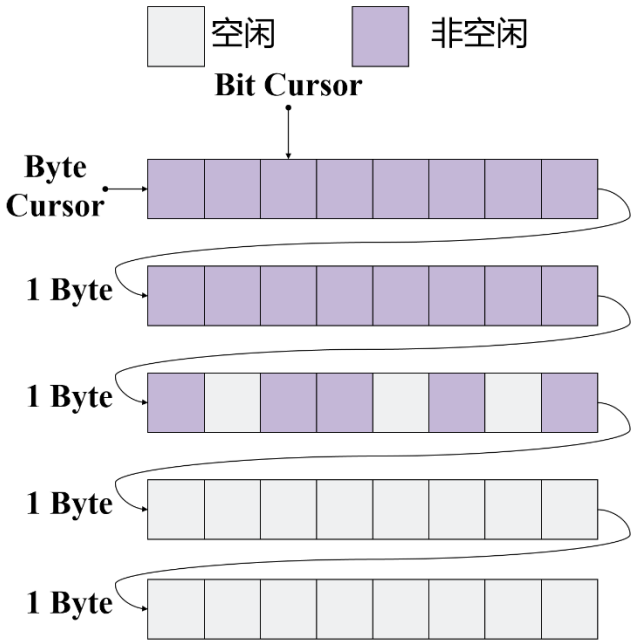
读取 读取 读取



超级块	索引节点位图	数据块位图	索引节点	索引节点	索引节点	索引节点	数据块	数据块

实验任务二：青春版Ext2文件系统

BitMap 的操作
在 Inode Map、Data Map 中寻找未分配空间




```
for (byte_cursor = 0; byte_cursor < BLKS_SZ(map_data_blks);
     byte_cursor++)
{
    for (bit_cursor = 0; bit_cursor < 8; bit_cursor++)
    {
        if((map_data[byte_cursor] & (0x1 << bit_cursor))
            == 0)
        {
            map_data[byte_cursor] |= (0x1 << bit_cursor);
            is_find_free = TRUE;
            break;
        }
    }
    if (is_find_free) {
        break;
    }
}
```

实验任务二：青春版Ext2文件系统

文件系统的卸载(umount)
把超级快、Inode Map、Data Map 写回磁盘中



管理区缓存

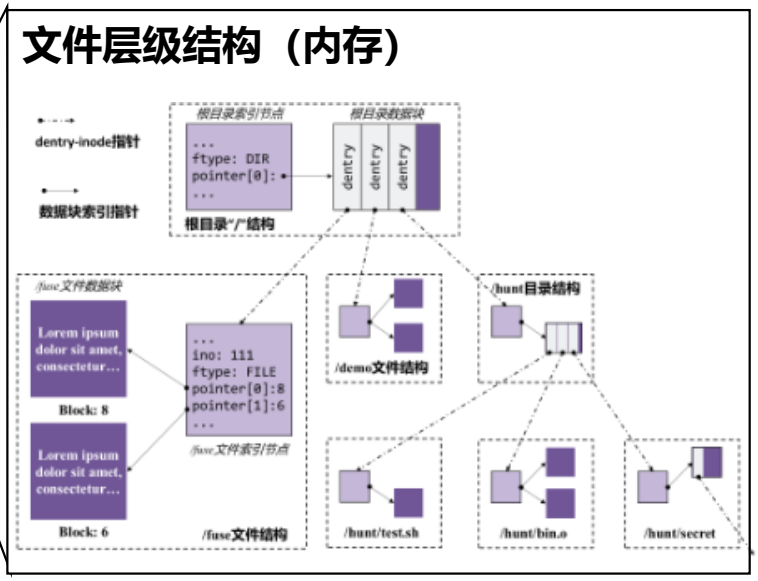
超级块
(内存)

文件层级结构
(内存)

索引节点位图
(内存)

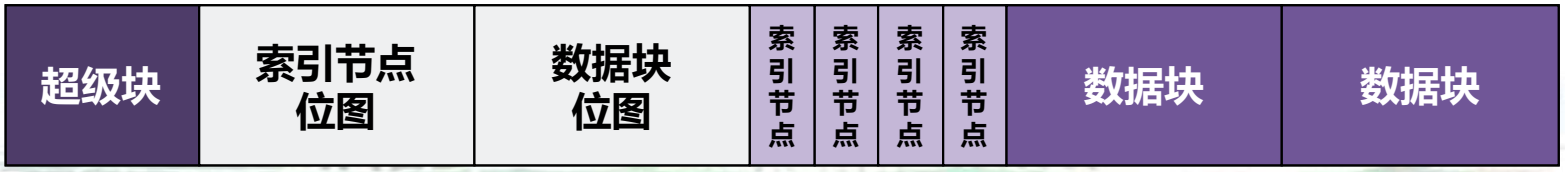
数据块位图
(内存)

文件层级结构（内存）



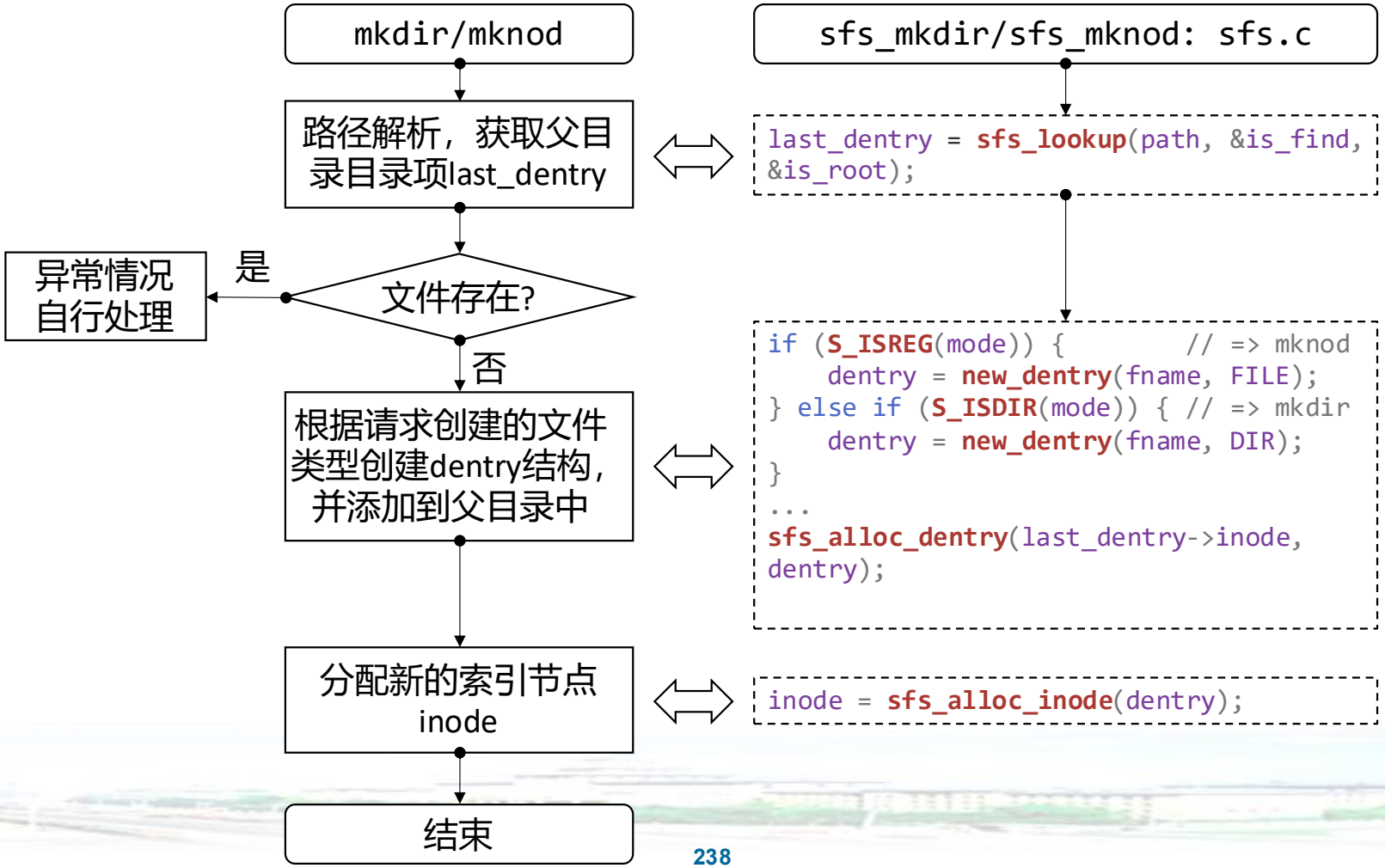
刷回磁盘





实验任务二：青春版Ext2文件系统

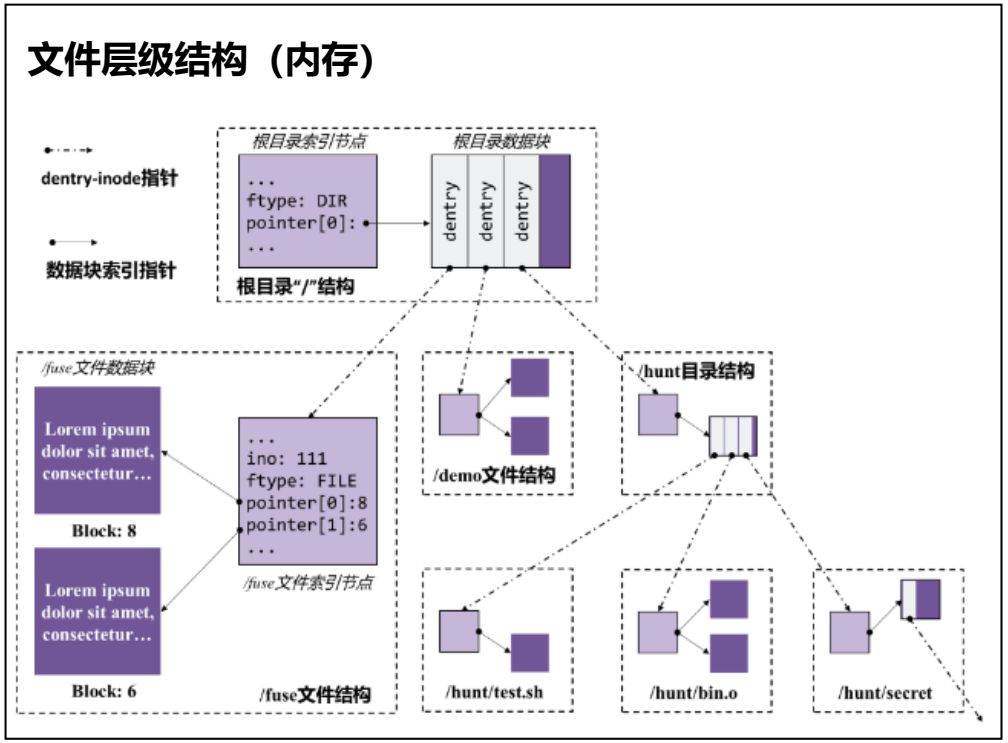
创建一个文件/目录的流程

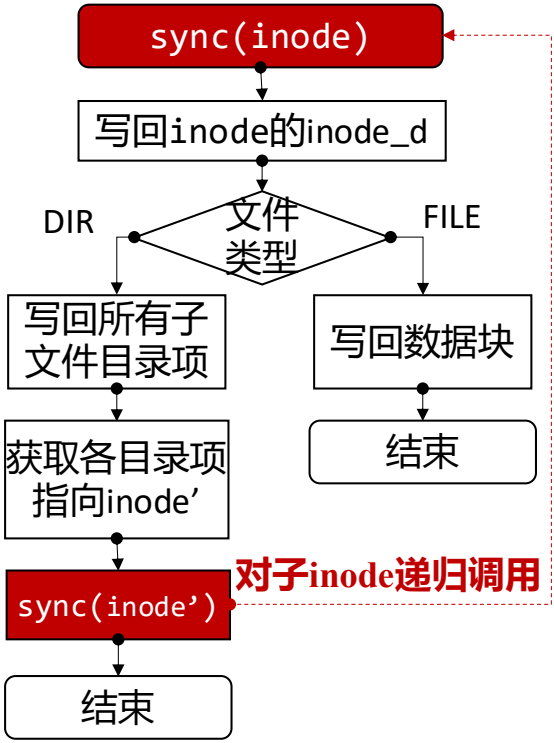


实验任务二：青春版Ext2文件系统

文件/目录 数据写入磁盘



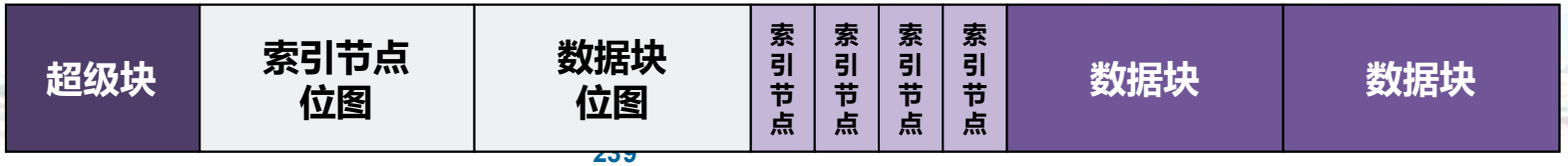




刷回磁盘

遍历所有文件/目录并刷回所有数据





实验提交

- 了解磁盘和磁盘管理的基本概念
- 掌握文件系统的核心设计思想及其实现原理，熟悉文件系统内部数据结构的组织方式和基本处理流程
- 设计和实现一个青春版的EXT2文件系统

完整代码和操作步骤见实验指导：https://github.com/hitsz-ids/OS-Kernel-system-software-tutorial/blob/main/Chapter07/Chapter07.md